



PICSimLab 0.9.3

Luis Claudio Gambôa Lopes <lcgamboa@yahoo.com>

[Download and support on Github](#)

[HTML version of documentation](#)

[PICSimLab on Instagram](#)

March 1, 2026

Contents

1	Introduction	6
2	Install	8
2.1	Stable version executables download	8
2.2	Unstable version executables download	8
2.3	Install from source	8
2.3.1	Linux	8
2.3.2	Windows	9
2.3.3	macOS	9
2.3.4	Experimental version	9
3	Simulator Interface	10
3.1	Main Window	10
3.2	Interaction with the Board	12
3.3	Command Line	13
3.4	Remote Control Interface	13
3.5	Picture Map Reference	16
4	Boards	18
4.1	Arduino Mega	20
4.2	Arduino Nano	20
4.3	Arduino Uno	21
4.4	Blue Pill	21
4.5	Breadboard	22
4.6	Curiosity	22
4.7	Curiosity HPC	23
4.8	ESP32-C3-DevKitC-02	23
4.9	ESP32-DevKitC	24
4.10	Franzininho DIY	24
4.11	K16F	25
4.12	McLab1	26
4.13	McLab2	26
4.14	PICGenios	27
4.15	PQDB	29

4.16 Remote TCP	29
4.17 STM32 H103	30
4.18 X	30
4.19 Xpress	31
4.20 gpboard	32
4.21 uCboard	32
5 Serial Communication	33
5.1 Com0com Installation and Configuration(Windows)	33
5.2 tty0tty Installation and Configuration (Linux)	35
5.3 Arduino IDE Integration (simavr)	36
6 Backend Simulators	37
6.1 PICsim	37
6.1.1 MPLABX Integrated Debug	37
6.2 simavr	37
6.2.1 avr-gdb Debug	38
6.2.2 MPLABX Int. Debug	38
6.3 qemu-stm32	39
6.3.1 arm-gdb Debug	39
6.4 qemu-ESP32	40
6.4.1 ESP32-gdb Debug	40
6.5 uCsim	41
6.5.1 uCsim Debug	41
6.6 gpsim	41
6.7 Remote	42
7 Tools	43
7.1 Serial Terminal	43
7.2 Serial Remote Tank	44
7.2.1 Actuators	44
7.2.2 Sensors	45
7.2.3 Communication Protocol	45
7.3 Esp8266 Modem Simulator	46
7.3.1 Supported Commands	47
7.4 Arduino Bootloader	47
7.5 MPLABX Debugger Plugin	47
7.6 Pin Viewer	47
7.7 picsimlab_tool	48
8 Oscilloscope	49
9 Spare Parts	50
9.1 Pin Alias	54
9.2 Inputs	55
9.2.1 ADXL345 (Accel)	55

9.2.2	BMP180 (Pressure I2C)	56
9.2.3	BMP280 (Pressure I2C)	57
9.2.4	DHT11 (Temp. Hum.)	57
9.2.5	DHT22 (Temp. Hum.)	58
9.2.6	DS1621 (Temperature I2C)	59
9.2.7	DS18B20 (Temperature)	59
9.2.8	Encoder	60
9.2.9	FM50 (Temperature)	60
9.2.10	Fixed Voltage	60
9.2.11	Gamepad	61
9.2.12	Gamepad (Analogic)	62
9.2.13	HX711 (Load Cell Amp)	63
9.2.14	Keypad	63
9.2.15	LDR	65
9.2.16	LM35 (Temperature)	66
9.2.17	MPU6050	66
9.2.18	Potentiometers	67
9.2.19	Potentiometers (Rotary)	68
9.2.20	Push Buttons	69
9.2.21	Push Buttons (Analogic)	69
9.2.22	SHT3X (Temp. Hum.)	69
9.2.23	Switches	70
9.2.24	Ultrasonic HC-SR04	71
9.3	Outputs	71
9.3.1	7 Segments Display	71
9.3.2	7 Segments Display (Decoder)	73
9.3.3	Buzzer	74
9.3.4	DC Motor	75
9.3.5	LCD hd44780	76
9.3.6	LCD ili9341	78
9.3.7	LCD pcf8833	79
9.3.8	LCD pcd8544	80
9.3.9	LCD ssd1306	80
9.3.10	LED Matrix	81
9.3.11	LEDs	81
9.3.12	RGB LED	82
9.3.13	RGB LED WS2812B	82
9.3.14	Servo Motor	83
9.3.15	Step Motor	84
9.4	Others	85
9.4.1	ETH w5500	85
9.4.2	IO 74xx573	86
9.4.3	IO 74xx595	86
9.4.4	IO MCP23017	87
9.4.5	IO MCP23S17	87
9.4.6	IO MM74C922	88

9.4.7	IO PCF8574	88
9.4.8	IO UART	89
9.4.9	Jumper Wires	89
9.4.10	Logic Block	90
9.4.11	MEM 24CXXX	90
9.4.12	RTC ds1307	91
9.4.13	RTC pfc8563	91
9.4.14	SD Card	92
9.4.15	Temperature System	93
9.5	Virtual	93
9.5.1	D. Transfer Function	93
9.5.2	IO Virtual Term	94
9.5.3	Signal Generator	94
9.5.4	Text Box	95
9.5.5	VCD Dump	95
9.5.6	VCD Dump (Analogic)	96
9.5.7	VCD Play	96
10	Troubleshooting	98
11	License	99
A	Online Simulator	100
B	Use with MPLABX	102
B.1	Installing the Necessary Tools	102
B.1.1	Install MPLABX IDE and XC8 Compiler	102
B.1.2	Install PICsimLab	103
B.1.3	How to Install PICSimLab MPLABX Debugger plugin	103
B.2	Configuring a New Project in MPLABX	108
B.2.1	Project Creation	108
B.2.2	File Creation	111
B.2.3	PIC Configuration Bits	112
B.2.4	Code Example	113
B.2.5	Building the Project	114
B.3	Program and Debug PICsimLab With MPLABX	115
B.3.1	Starting PICsimLab	115
B.3.2	Programming PICsimLab	115
B.3.3	Pausing the Program	116
B.3.4	Restarting the Program	116
B.3.5	Running Step by Step	117
B.3.6	Stopping Debugger	118
B.3.7	Disconnect Debugger	118
B.4	This Tutorial in Video	119

C	Creating New Boards	120
C.1	Creating a New Board	120
C.1.1	Board Hardware and Schematic	120
C.1.2	Board Picture	123
C.1.3	Picture map	125
C.1.4	Board code	129
C.1.5	Integration with PICsimLab	143
C.1.6	Final Result	144

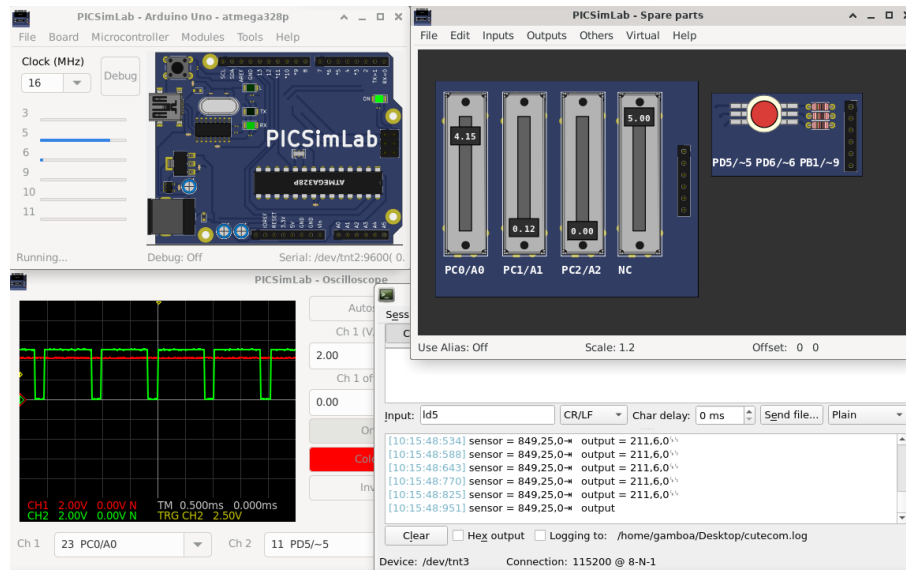
Chapter 1

Introduction

PICSimLab means Programmable IC Simulator Laboratory

PICSimLab is a realtime emulator of **development boards** with integrated MPLABX/avr-gdb debugger. PICSimLab supports microcontrollers of [picsim](#) , [simavr](#), [uCsim](#), [gpsim](#), [qemu-stm32](#) and [qemu-esp32](#) simulators. PICSimLab have integration with MPLABX/Arduino IDE for programming the boards microcontrollers. As the purpose of PICSimLab is to emulate real hardware it does not have any source code editing support. For code editing and debugging the same tools used for a real board should be used with PICSimLab, such as MPLABX, Arduino IDE or VSCode with PlatformIO.

PICSimLab supports several devices (spare parts) that can be connected to the boards for simulation. As for example LEDs and push buttons for simple outputs and inputs and some more complex ones like the ethernet shield w5500 for internet connection or the color graphic display ili9340 with touchscreen. The the complete list of parts can be accessed in the chapter **Spare Parts**.



Chapter 2

Install

2.1 Stable version executables download

If you are on Linux or Windows you can download the last version at:

<https://github.com/lcgamboa/picsimlab/releases>

If you are on macOS you can run PICSimLab using Wine:

1. Download and install ['xquartz'](https://www.xquartz.org)
2. Download and install [Wine](https://dl.winehq.org/wine-builds/macosx/download.html)
3. Download the executable and double-click it to run the installer

2.2 Unstable version executables download

The binaries of last code available on github can be downloaded at: [Sourceforge.net](https://sourceforge.net)

The unstable test version have the unreleased features of [Changelog_auto.md](#)

If you need a specific binary that is not available please contact me.

2.3 Install from source

2.3.1 Linux

In Debian Linux and derivatives Linux native:

Using a user with permission to run the sudo command:

In first time build:

```
git clone --depth=1 https://github.com/lcgamboa/picsimlab.git
cd picsimlab
bscripts/build_all_and_install.sh
```

To recompile use:

```
make -j4
```

2.3.2 Windows

Cross-compiling for Windows (from Linux or [WSL](#) on win10)

In first time build in Debian Linux and derivatives target Windows 64 bits:

```
git clone https://github.com/lcgamboa/picsimlab.git
cd picsimlab
bscripts/build_w64.sh
```

To recompile use:

```
make FILE=Makefile.cross -j4
```

For target Windows 32 bits:

```
git clone https://github.com/lcgamboa/picsimlab.git
cd picsimlab
bscripts/build_w32.sh
```

To recompile use:

```
make FILE=Makefile.cross_32 -j4
```

2.3.3 macOS

Theoretically it is possible to compile PICSimLab natively on macOS. But I do not have access to any computer with macOS to try to compile and until today nobody has communicated that they managed to do it. (help wanted)

2.3.4 Experimental version

Experimental version can be built using the parameter "exp" on scripts:

```
bscripts/build_all_and_install.sh exp
bscripts/build_w64.sh exp
bscripts/build_w32.sh exp
```

And recompiled using the parameter "exp" on Makefiles:

```
make -j4 exp
make FILE=Makefile.cross -j4 exp
make FILE=Makefile.cross_32 -j4 exp
```

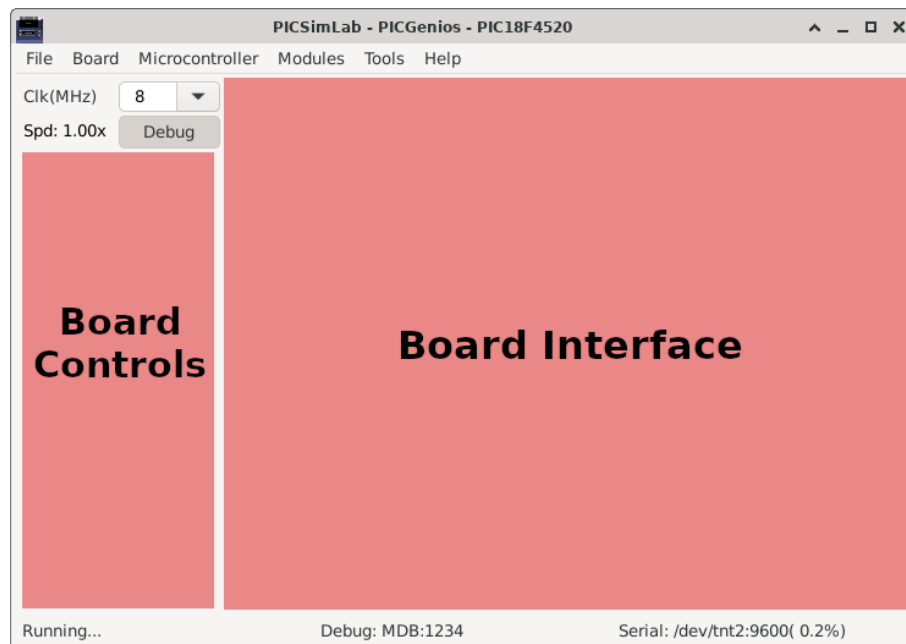
Chapter 3

Simulator Interface

3.1 Main Window

The main window consists of a menu, a status bar, a frequency selection combobox, an on/off button to trigger debugging, some board-specific controls and the part of the board interface itself.

In the title of the window is shown the name of the simulator PICSimLab, followed by the board and the microcontroller in use.



The frequency selection combobox directly changes the working speed of the mi-

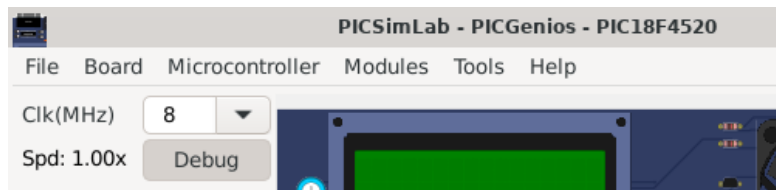
crocontroller. The “Spd” label show the ratio between simulation speed and real time. when the “Spd” label goes red indicates that the computer is not being able to run the program in real time for the selected clock. In this case the simulation may present some difference than expected and the CPU load will be increased.

The on/off button to enable debugging is used to enable debugging support, when active simulation load is increased.

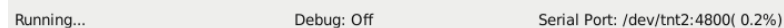
The menus and their functions are listed below:

- File
 - Load Hex - Load .hex files
 - Reload Last - Reload the last used .hex file
 - Save Hex - Save memory in a .hex file
 - Configure - Open the configuration windows
 - Save Workspace - Saves all current workspace settings to a .pzw file
 - Load Workspace - Loads saved settings from a .pzw file
 - Exit
- Board
 - Arduino Uno - Choose board Arduino Uno
 - Breadboard - Choose board Breadboard
 - Franzininho - Choose board Franzininho
 - K16F - Choose board K16F
 - McLab1 - Choose board McLab1
 - McLab2 - Choose board McLab2
 - PICGenios - Choose board PICGenios
 - PQDB - Choose board PQDB
 - uCboard - Choose board uCboard
- Microcontroller
 - xxxxx - Selects the microcontroller to be used (depends on the selected board)
- Modules
 - Oscilloscope - Open the oscilloscope window
 - Spare parts - Open the spare parts window
- Tools
 - Serial Terminal - Open the serial terminal (default **Cutecom**)
 - Serial Remote Tank - Open the **remote tank simulator**

- Esp8266 Modem Simulator - Open the [Esp8266 Modem Simulator](#)
- Arduino Bootloader - Load microcontroller with [Arduino serial bootloader](#)
- MPLABX Debugger Plugin - Open the web page to download the [MPLABX Debugger Plugin](#)
- Pin Viewer - Open the [Pin Viewer](#)
- Help
 - Contents - Open the Help window
 - Board - Open the Board Help window
 - Examples - Load the examples
 - About Board - Show message about author and version of board
 - About PICSimLab - Show message about author and version of PICSimLab



The first part of the status bar shows the state of the simulation, in the middle part the status of the debug support and in the last part the name of the serial port used, its default speed and the error in relation to the real speed configured in the microcontroller.



3.2 Interaction with the Board

On the interface area of the board it is possible to interact in some ways:

- Click in ICSP connector to load an .hex file.
- Click in PWR button to ON/OFF the emulator..
- The buttons can be activated through mouse or keys 1, 2, 3 e 4.
- Click and drag in potentiometers to change their values.
- Click on EEPROM memory to view its contents.

3.3 Command Line

PICSimLab supports two command lines format:

One for load a PICSimLab Workspace file (.pzw)

```
picsimlab file.pzw
```

And other for load .hex files

```
picsimlab boardname microcontroller [file.hex] [file.pcf]
```

3.4 Remote Control Interface

The remote control interface allows other programs to control the PICSimLab simulation through a TCP/IP socket using text formatted commands.

The PICSimLab remote control interface supports TCP connections using telnet or nc (netcat).

The default port is 5000 and can be changed in configuration windows.

The 'rlwrap' command can be used for best command edition support in telnet or nc:

```
rlwrap nc 127.0.0.1 5000
```

The supported commands can be shown using the “help” command:

```
help
List of supported commands:
  dumpe [a] [s]- dump internal EEPROM memory
  dumpf [a] [s]- dump Flash memory
  dumpr [a] [s]- dump RAM memory
  exit          - shutdown PICSimLab
  get ob        - get object value
  help          - show this message
  info          - show actual setup info and objects
  loadhex file  - load hex file (use full path)
  pins          - show pins directions and values
  pinsl         - show pins formatted info
  quit          - exit remote control interface
  reset         - reset the board
  set ob vl     - set object with value
  sim [cmd]     - show simulation status or execute cmd start/stop
  sync         - wait to synchronize with timer event
  version       - show PICSimLab version
Ok
```

The “info” command show all available "objects" and values:

```
info
Board:    Arduino Uno
```

```

Processor: atmega328p
Frequency: 16000000 Hz
Use Spare: 1
    board.out[00] LD_L= 254
part[00]: LEDs
    part[00].out[08] LD_1= 254
    part[00].out[09] LD_2= 30
    part[00].out[10] LD_3= 254
    part[00].out[11] LD_4= 254
    part[00].out[12] LD_5= 254
    part[00].out[13] LD_6= 254
    part[00].out[14] LD_7= 254
part[01]: Buzzer
    part[01].out[02] LD_1= 140
part[02]: Push buttons
    part[02].in[00] PB_1= 1
    part[02].in[01] PB_2= 0
    part[02].in[02] PB_3= 1
    part[02].in[03] PB_4= 1
    part[02].in[04] PB_5= 1
    part[02].in[05] PB_6= 1
    part[02].in[06] PB_7= 1
    part[02].in[07] PB_8= 1

```

Ok

The “pins” command show all pins directions and digital values:

```

pins
pin[01] ( PC6/RST ) < 0
pin[02] ( PD0/0 ) < 1
pin[03] ( PD1/1 ) < 1
pin[04] ( PD2/2 ) < 1
pin[05] ( PD3/~3 ) > 0
pin[06] ( PD4/4 ) < 1
pin[07] ( +5V ) < 1
pin[08] ( GND ) < 0
pin[09] ( PB6/X1 ) < 0
pin[10] ( PB7/X2 ) < 0
pin[11] ( PD5/~5 ) < 1
pin[12] ( PD6/~6 ) < 1
pin[13] ( PD7/7 ) < 1
pin[14] ( PB0/8 ) > 0
pin[15] ( PB1/~9 ) > 0
pin[16] ( PB2/~10 ) > 0
pin[17] ( PB3/~11 ) > 0
pin[18] ( PB4/12 ) < 0
pin[19] ( PB5/13 ) > 0
pin[20] ( +5V ) < 1
pin[21] ( AREF ) < 0
pin[22] ( GND ) < 0
pin[23] ( PC0/A0 ) < 0
pin[24] ( PC1/A1 ) < 0
pin[25] ( PC2/A2 ) < 0
pin[26] ( PC3/A3 ) < 0
pin[27] ( PC4/A4 ) > 0
pin[28] ( PC5/A5 ) > 0

```

Ok

The “pinsl” command show all pins info in text formatted output:

```

pinsl
28 pins [atmega328p]:
pin[01] D I 0 000 0.000 "PC6/RST "

```

```

pin[02] D I 1 200 0.000 "PD0/0  "
pin[03] D I 1 200 0.000 "PD1/1  "
pin[04] D I 1 200 0.000 "PD2/2  "
pin[05] D O 0 007 0.000 "PD3/~3  "
pin[06] D I 1 200 0.000 "PD4/4  "
pin[07] P I 1 200 0.000 "+5V    "
pin[08] P I 0 000 0.000 "GND    "
pin[09] D I 0 000 0.000 "PB6/X1  "
pin[10] D I 0 000 0.000 "PB7/X2  "
pin[11] D I 1 200 0.000 "PD5/~5  "
pin[12] D I 1 200 0.000 "PD6/~6  "
pin[13] D I 1 200 0.000 "PD7/7  "
pin[14] D O 0 000 0.000 "PB0/8  "
pin[15] D O 0 000 0.000 "PB1/~9  "
pin[16] D O 0 000 0.000 "PB2/~10 "
pin[17] D O 0 006 0.000 "PB3/~11 "
pin[18] D I 0 000 0.000 "PB4/12  "
pin[19] D O 0 000 0.000 "PB5/13  "
pin[20] P I 1 200 0.000 "+5V    "
pin[21] R I 0 000 0.000 "AREF    "
pin[22] P I 0 000 0.000 "GND    "
pin[23] A I 0 000 0.875 "PC0/A0  "
pin[24] A I 0 000 1.925 "PC1/A1  "
pin[25] A I 0 000 2.700 "PC2/A2  "
pin[26] A I 0 000 4.275 "PC3/A3  "
pin[27] D O 1 179 0.000 "PC4/A4  "
pin[28] D O 1 186 0.000 "PC5/A5  "

```

Ok

You can view one input/output state using the “get” command:

```

get board.out[00]

get part[02].in[01]

```

Its possible use the “get” command to view individual pins state:

```

#digital state
get pin[19]
pin[19]= 0
Ok

#digital mean value (0-200)
get pinm[19]
pin[18]= 100
Ok

#analog state
get apin[25]

```

```
apin[25]= 2.700
Ok

#all info
get pinl[13]
pin[13] D I 1 200 0.000 "PD7/7  "
Ok
```

And set value of one input using the “set” command:

```
set part[02].in[01] 0
set part[02].in[01] 1
```

Or set value of one pin using the “set” command:

```
#digital
set pin[10] 2

#analog
set apin[20] 2.345
```

For windows users [putty telnet client](#) is a good option to access the remote control interface.

3.5 Picture Map Reference

Names used in .map files for boards and parts are standardized and used by the remote control interface.

The names must start with **I_** if it is an input, **O_** if it is an output or **B_** if it is bidirectional. And be followed by one of the two-letter types in the table below before the area name.

Function	Type	Description	RControl In	RControl Out
I	MD	Memory Dump	-	-
I	KB	Keyboard Key	0 or 1	0 or 1
I	PG	Program	-	-
I	CN	Connector	-	-
B	PO	Potentiometer	0 to 200	0 to 200
B	JP	Jumper	0 or 1	0 or 1
B	VS	Value short	-32768 to 32767	-32768 to 32767
B	PB	Push button	0 or 1	0 or 1
B	DP	Dip switch	0 or 1	0 or 1
B	SW	Switch	0 or 1	0 or 1
B	RT	Rotary encoder	0 to 200	0 to 200
B	AJ	Dip switch	-	-
O	MC	Motor Cooler	-	0 to 200
O	PN	Pin name	-	-
O	ST	Status	-	-
O	IC	IC name	-	-
O	LR	LED RGB	-	-
O	LM	LED Matrix	-	-
O	DI	Display Info	-	-
O	LD	LED	-	0 to 200
O	DS	Display	-	if alphanumeric show text
O	MT	DC motor	-	dir 0 or 1, spd. 0 to 200, pos. 0 to 200
O	DG	Degree	-	float angle
O	SS	seven segment	-	decoded number

For example area named **B_PB_Start**, which describes the position of a push button named "Start". The **B_** bidirectional indicates that the mapped area serves as user action input and drawing output.

Chapter 4

Boards

PICSimLab currently supports five backend simulators: `picsim`, `simavr`, `uCsim`, `gpsim` and `qemu` (`stm32` and `esp32`).

The Figure below shows which boards are based on which backend simulator:

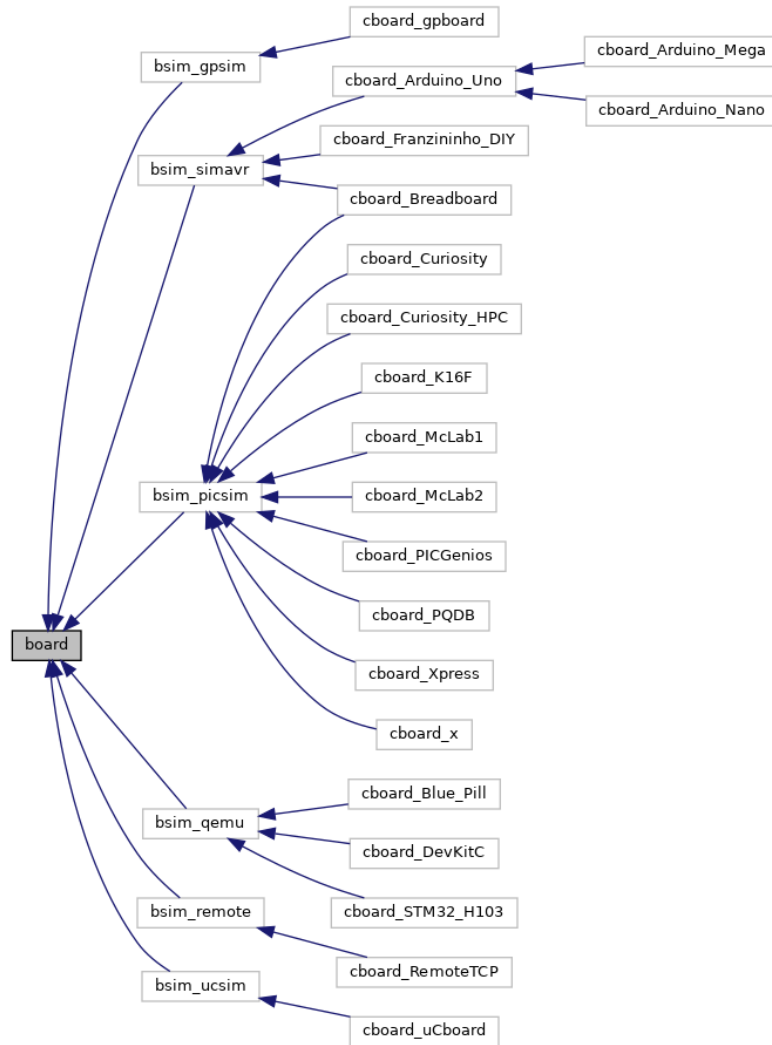


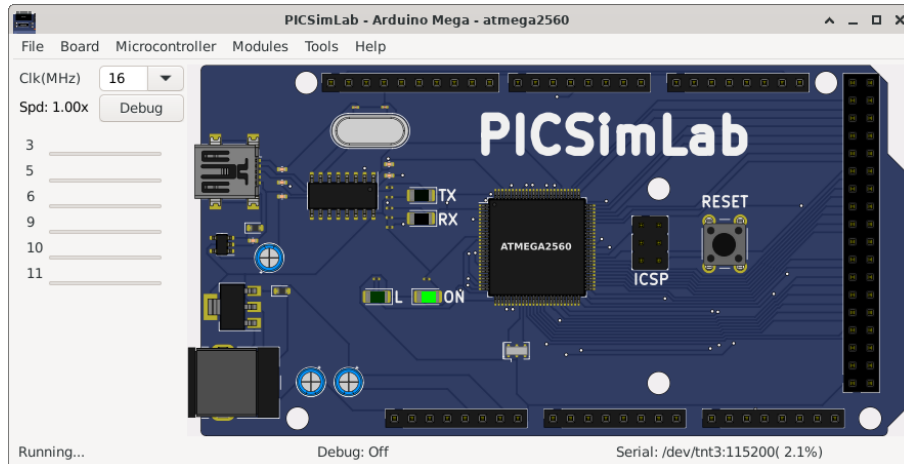
Figure 4.1: Boards backend simulators

The below table show the supported debug interface of each simulator:

Backend	Debug Support
picsim	see the section MPLABX Integrated Debug
simavr	see the sections MPLABX Integrated Debug and remote avr-gdb Debug
qemu-stm32	see the section remote arm-gdb Debug
qemu-esp32	see the section remote esp32-gdb Debug
uCsim	see the section uCsim remote console (telnet) Debug
gpsim	none yet

4.1 Arduino Mega

It emulates the Arduino Mega development board that uses one ATMEGA2560 micro-controller of [simavr backend simulator](#).



The code examples can be loaded in PICSIMLab menu **Help->Examples**.

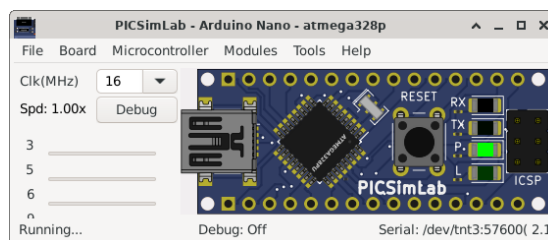
The source code of board Arduino Mega examples using the [Arduino IDE with avr-gcc](#) are in the link: [board_Arduino_Mega](#).

More information about the Arduino in www.arduino.cc

Information on how to configure the PICSIMLab integration with the Arduino IDE can be found in the section: [Arduino IDE Integration](#).

4.2 Arduino Nano

It emulates the Arduino Nano development board that uses one ATMEGA328P micro-controller of [simavr backend simulator](#).



The code examples can be loaded in PICSIMLab menu **Help->Examples**.

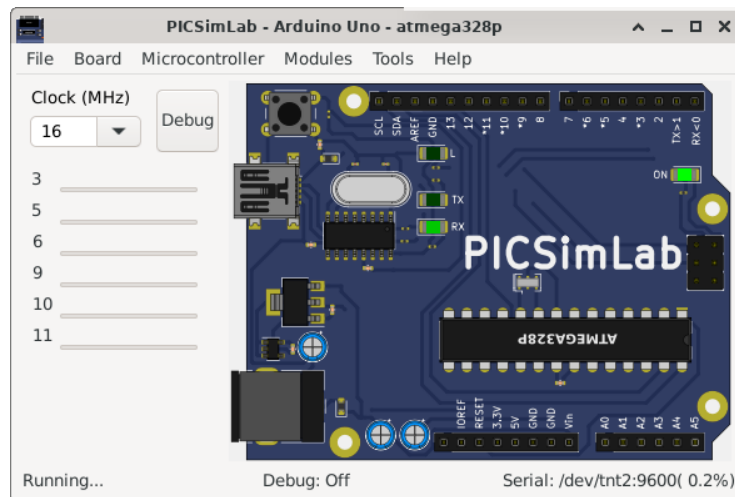
The source code of board Arduino Nano examples using the [Arduino IDE with avr-gcc](#) are in the link: [board_Arduino_Nano](#).

More information about the Arduino in www.arduino.cc

Information on how to configure the PICSimLab integration with the Arduino IDE can be found in the section: [Arduino IDE Integration](#).

4.3 Arduino Uno

It emulates the Arduino Uno development board that uses one ATMEGA328P microcontroller of [simavr backend simulator](#).



[Board Arduino Uno schematics](#).

The code examples can be loaded in PICSimLab menu **Help->Examples**.

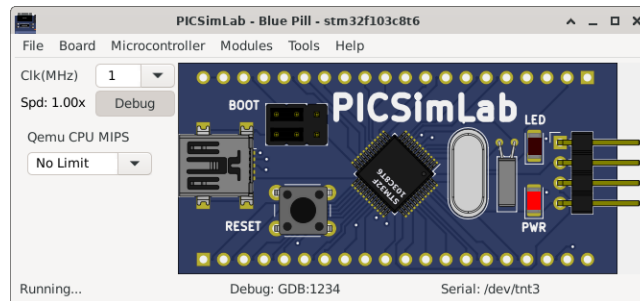
The source code of board Arduino Uno examples using the [Arduino IDE with avr-gcc](#) are in the link: [board_Arduino_Uno](#).

More information about the Arduino in www.arduino.cc

Information on how to configure the PICSimLab integration with the Arduino IDE can be found in the section: [Arduino IDE Integration](#).

4.4 Blue Pill

It is a generic board only with reset, serial and crystal circuits and support to stm32f103c8t6 microcontroller of [qemu-stm32 backend simulator](#).

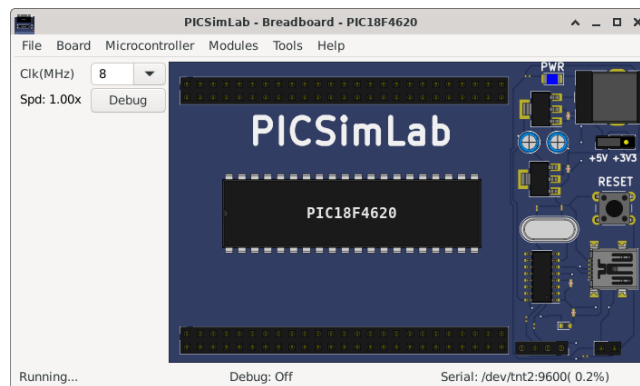


[Board Blue Pill schematics.](#)

[Examples](#)

4.5 Breadboard

It is a generic board only with reset, serial and crystal circuits and support to multiple microcontrollers of **PICSim** and **simavr** backend simulators.

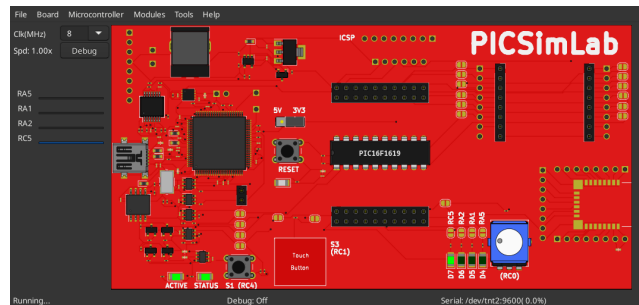


[Board Breadboard schematics.](#)

[Examples](#)

4.6 Curiosity

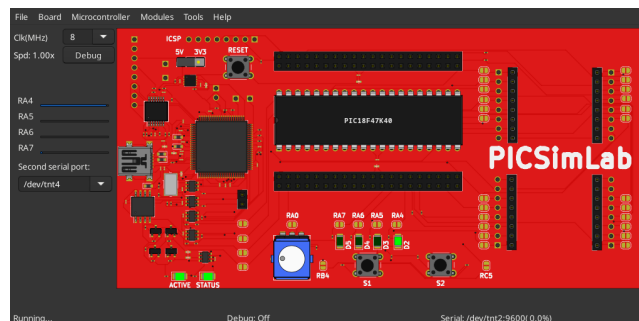
This is a simple PIC microcontroller development board that uses the **PICSim backend simulator**.



Examples

4.7 Curiosity HPC

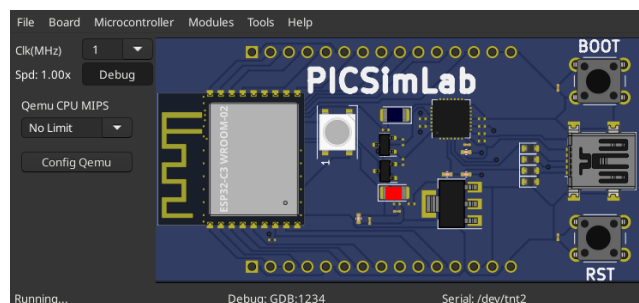
This is a simple PIC microcontroller development board that uses the **PICSim backend simulator**.



Examples

4.8 ESP32-C3-DevKitC-02

It is a simple board only with reset, serial and crystal circuits and support for ESP32-C3 microcontroller of **qemu-esp32 backend simulator**.

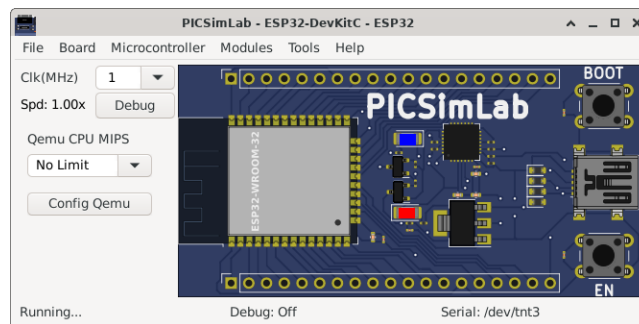


[Board DevKitC-02 schematics.](#)

[Examples](#)

4.9 ESP32-DevKitC

It is a simple board only with reset, serial and crystal circuits and support for ESP32 microcontroller of [qemu-esp32 backend simulator](#).

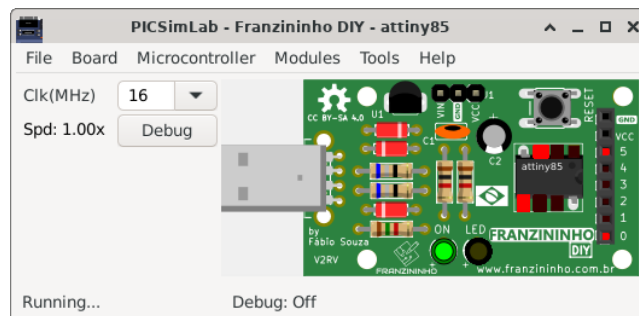


[Board DevKitC schematics.](#)

[Examples](#)

4.10 Franzininho DIY

The Franzininho DIY board is an openhardware project, more info at <https://franzininho.com.br/>. It was developed to be used with the microcontroller ATtiny85 of [simavr backend simulator](#).

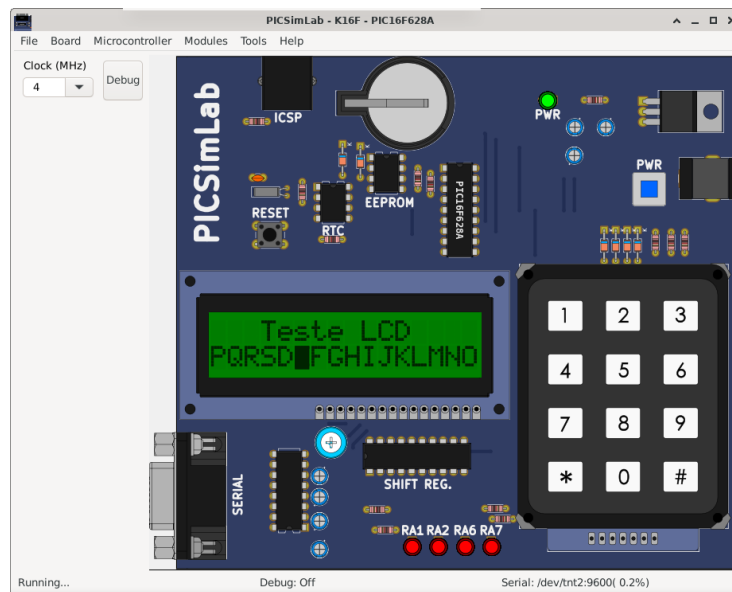
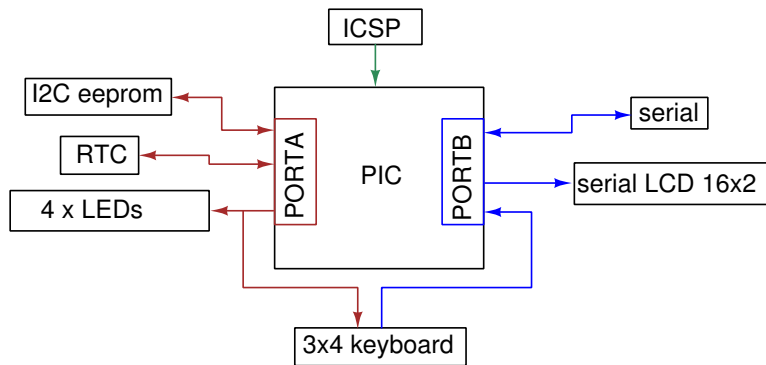


[Board Franzininho DIY schematics.](#)

[Examples](#)

4.11 K16F

It emulates an didactic board developed by author that uses one PIC16F84, PIC16F628 or PIC16F648 of **PICSim backend simulator**.



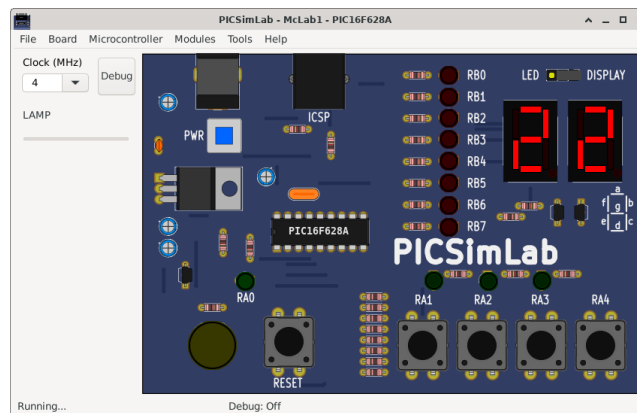
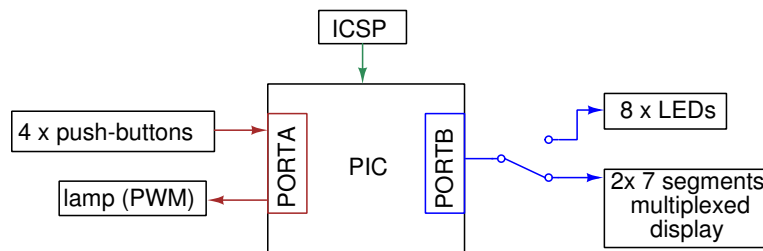
[Board K16F schematics.](#)

The code examples can be loaded in PICSimLab menu **Help->Examples**.

The source code of board K16F examples using **MPLABX** and **XC8** compiler are in the link: [board_K16F](#).

4.12 McLab1

It emulates the Labtools development board McLab1 that uses one PIC16F84, PIC16F628 or PIC16F648 of **PICSim backend simulator**.



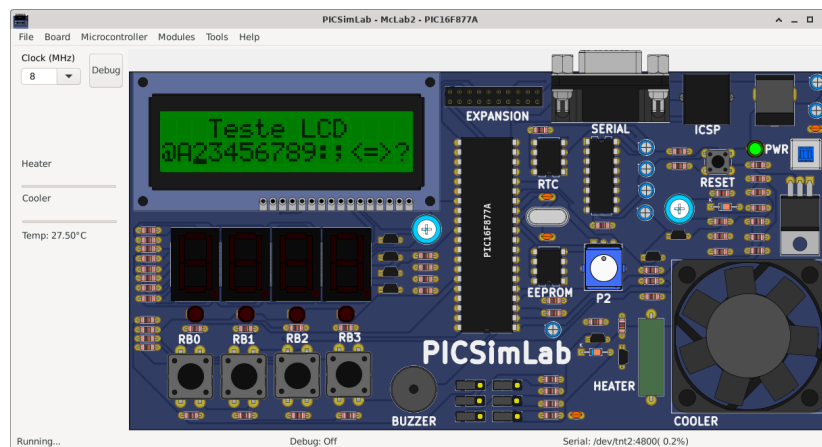
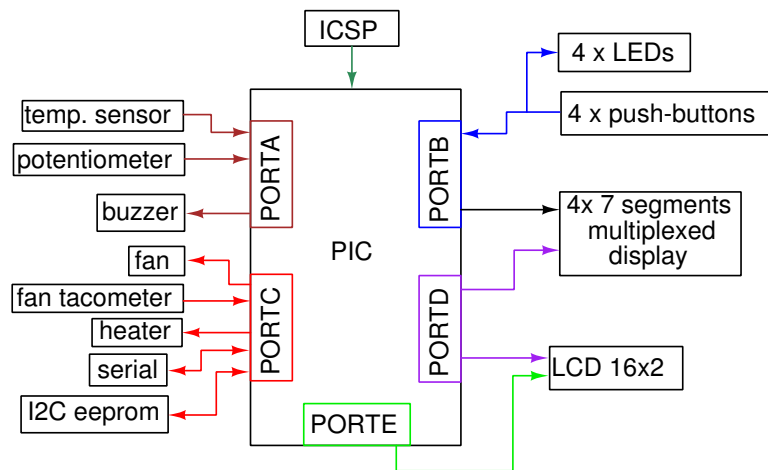
[Board McLab1 schematics.](#)

The code examples can be loaded in PICSimLab menu **Help->Examples**.

The source code of board McLab1 examples using [MPLABX](#) and [XC8](#) compiler are in the link: [board_McLab1](#).

4.13 McLab2

It emulates the Labtools development board McLab2 that uses one PIC16F777, PIC16F877A, PIC18F452, PIC18F4520, PIC18F4550 or PIC18F4620 of **PICSim backend simulator**..



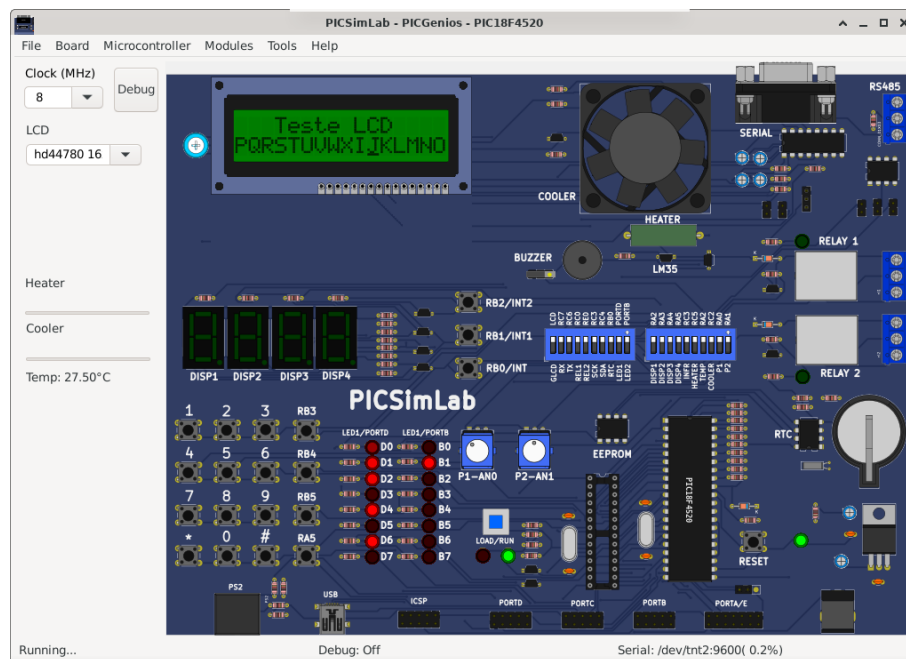
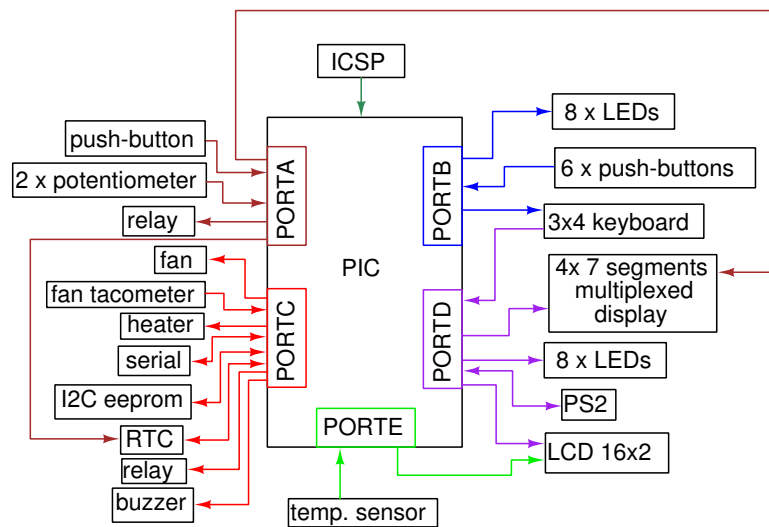
[Board McLab2 schematics.](#)

The code examples can be loaded in PICSimLab menu **Help->Examples**.

The source code of board McLab2 examples using [MPLABX](#) and [XC8](#) compiler are in the link: [board_McLab2](#).

4.14 PICGenios

It emulates the microgenius development board PICGenios PIC18F e PIC16F Microchip that uses one PIC16F777, PIC16F877A, PIC18F452, PIC18F4520, PIC18F4550 or PIC18F4620 of [PICSim backend simulator](#).



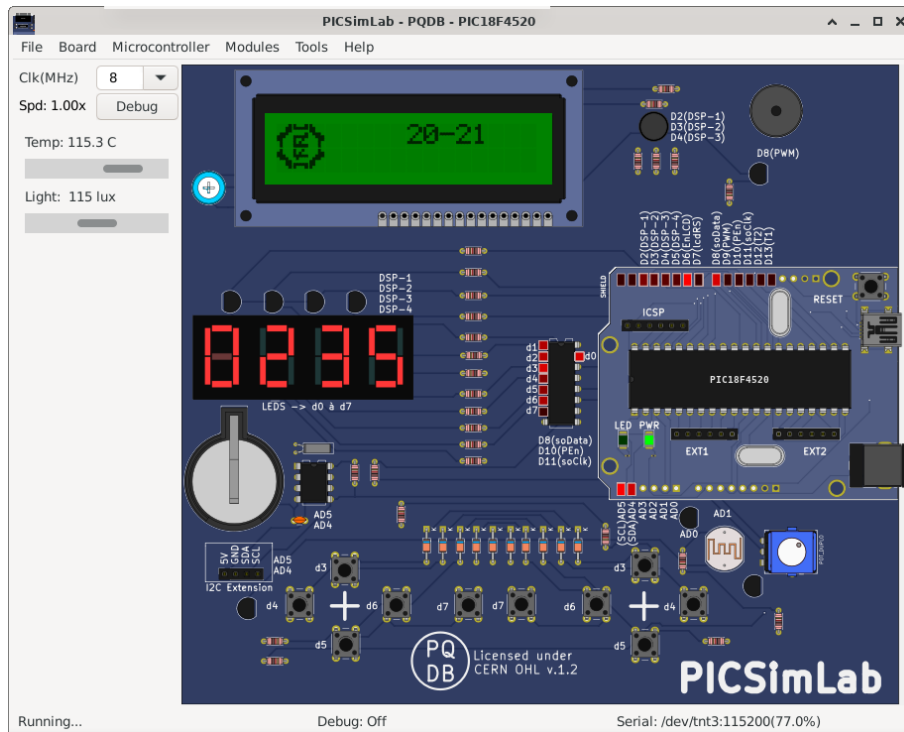
Board PICGenios schematics.

The code examples can be loaded in PICSimLab menu **Help->Examples**.

The source code of board PICGenios examples using **MPLABX** and **XC8** compiler are in the link: [board PICGenios](#).

4.15 PQDB

The PQDB board is an opensource/openhardware project, more info at <https://github.com/projetopqdb/>. It was developed to be used with arduino/freedom boards, but adapted to use the microcontroller PIC18F4520 of **PICSim backend simulator** on PICSimLab.



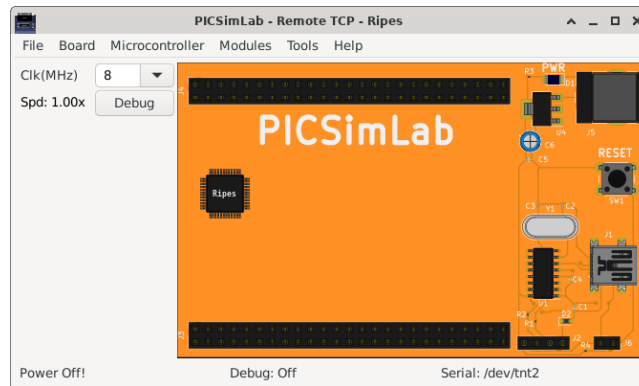
[Board PQDB schematics.](#)

[Hat board PIC18F schematics.](#)

[Examples](#)

4.16 Remote TCP

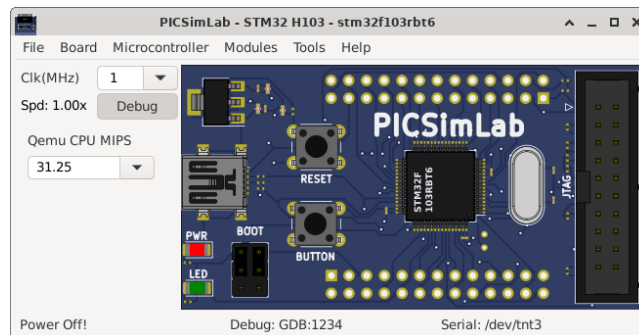
It is a virtual board controlled through one TCP connection. Currently only supports the Risc-V simulator [Ripes](#) and uses the **remote backend simulator**.



Examples

4.17 STM32 H103

It is a generic board only with reset, one push button, serial and crystal circuits and support to stm32f103rht6 microcontroller of [qemu-stm32 backend simulator](#).

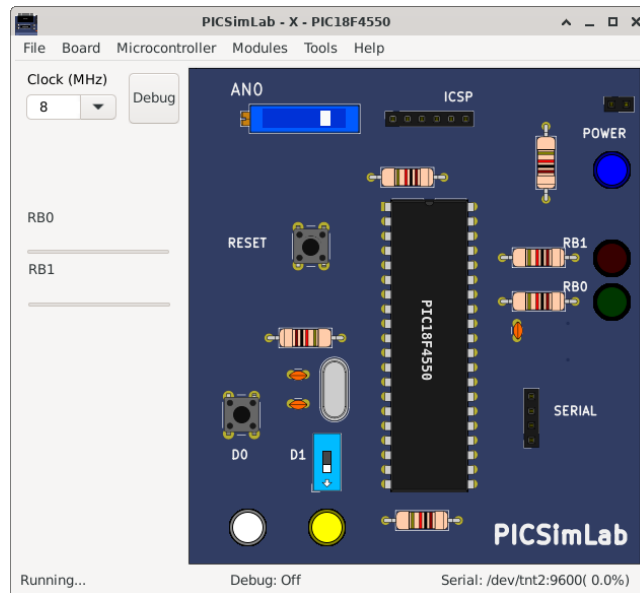


[Board STM32 H103 schematics.](#)

Examples

4.18 X

It is a generic board, used as example in the tutorial [Creating New Boards](#). This board uses one PIC16F877A, PIC18F4550 or PIC18F4620 of [PICSim backend simulator](#).

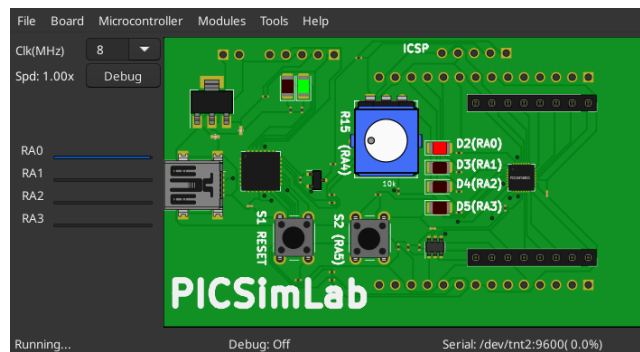


[Board X schematics.](#)

[Examples](#)

4.19 Xpress

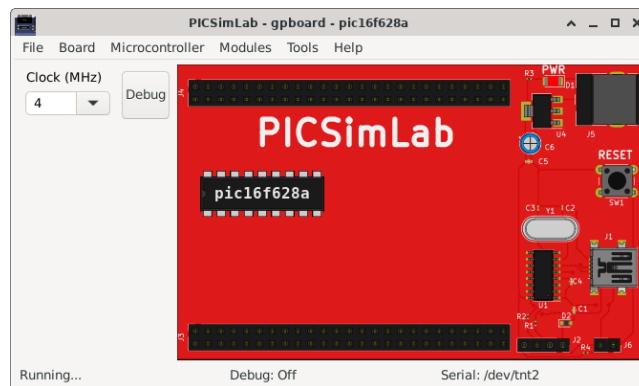
This is a simple PIC microcontroller development board that uses the [PICSim backend simulator](#).



[Examples](#)

4.20 gpboard

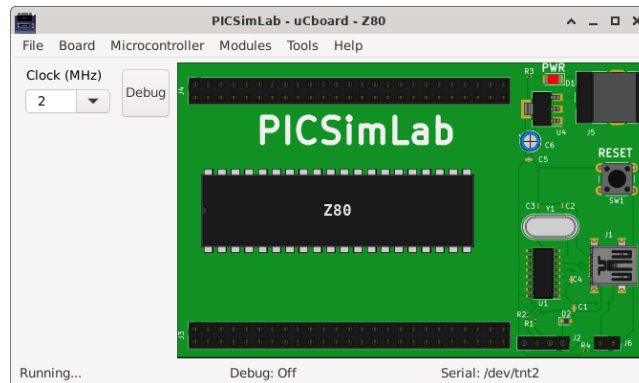
It is a generic board only with reset, serial and crystal circuits and support to multiple microcontrollers of **gpsim backend simulator**..



[Examples](#)

4.21 uCboard

It is a generic board only with reset, serial and crystal circuits and support to multiple microcontrollers (initially C51, Z80 and STM8S103)of **ucsim backend simulator**..



[Examples](#)

Chapter 5

Serial Communication

To use the simulator serial port emulation, you must install a NULL-MODEM emulator:

- Windows: com0com <http://sourceforge.net/projects/com0com/>
- Linux: tty0tty <https://github.com/lcgamboa/tty0tty>

For communication the PICSimLab should be connected in one port of the NULL-MODEM emulator and the other application connected in the other port. Configuration examples linking PICSimLab to [Cutecom](#) for serial communication:

OS	PicsimLab port	Cutecom port	NULL-Modem prog.	Connection
Windows	com1	com2	com0com	com1<=>com2
Linux	/dev/tnt2	/dev/tnt3	tty0tty	/dev/tnt2<=>/dev/tnt3

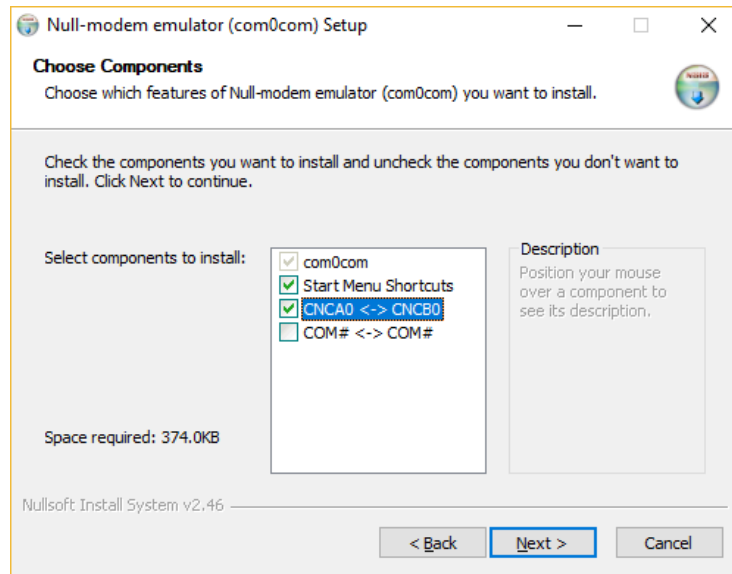
The PICSimLab serial communication uses the 8N1 format.

5.1 Com0com Installation and Configuration(Windows)

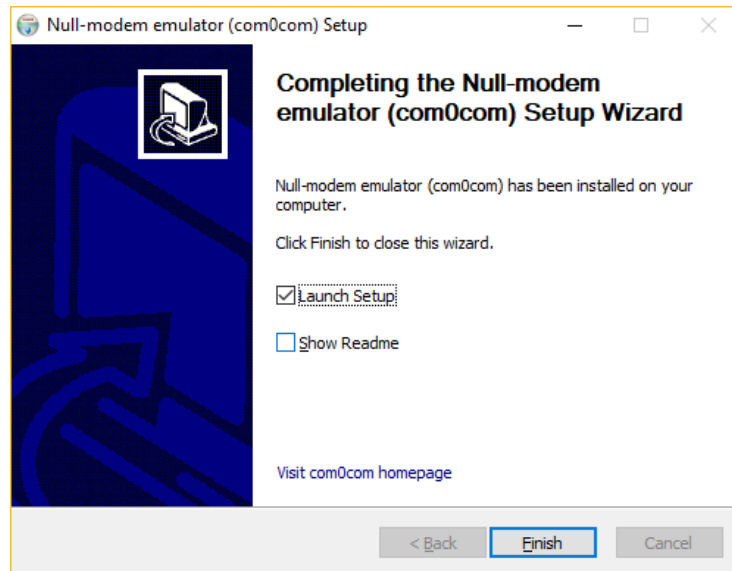
Download the signed version of [com0com](#).

Unzip the downloaded .zip file and run the specific installer of your operating system, x86 for windows 32-bit or x64 for windows 64-bit.

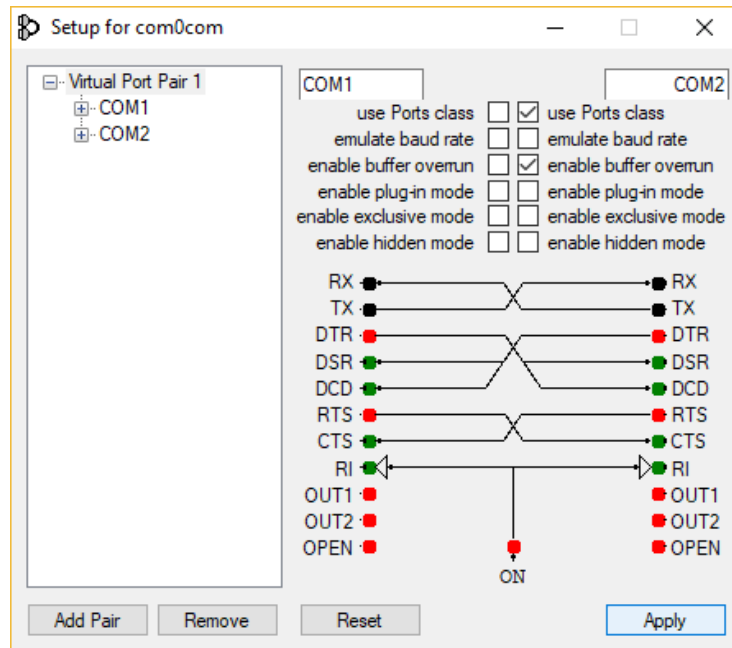
Configure the “choose components” window as the figure below:



In the last configuration window, check the “Launch setup” option:



In the setup window, change the port names to COM1, COM2, COM3 Just check the “enable buffer overrun” and “use Port class” options on the port used to CuteCom, click in the “Apply” button and close the setup. In the configuration shown in the figure below, the port COM1 (with buffer overrun disabled) must be used by the PICSimLab and COM2 (with buffer overrun enabled) by the application with serial communication.



5.2 tty0tty Installation and Configuration (Linux)

Download the [tty0tty](#). Unzip the downloaded folder.

Open a terminal and enter in the `tty0tty/module/` folder and enter the following commands:

```
sudo apt-get update
sudo apt-get -y upgrade
sudo apt-get -y install gcc make linux-headers-`uname -r`
sudo ./dkms-install.sh
sudo modprobe tty0tty
```

The user must be in the **dialout** group to access the ports. To add your user to **dialout** group use the command:

```
sudo usermod -a -G dialout your_user_name
```

after this is necessary logout and login to group permissions take effect.

Once installed, the module creates 8 interconnected ports as follows:

```
/dev/tnt0 <=> /dev/tnt1
/dev/tnt2 <=> /dev/tnt3
/dev/tnt4 <=> /dev/tnt5
/dev/tnt6 <=> /dev/tnt7
```

the connection between each pair is of the form:

```
TX    ->  RX
RX    <-   TX
RTS   ->  CTS
CTS   <-   RTS
DSR   <-   DTR
CD    <-   DTR
DTR   ->  DSR
DTR   ->  CD
```

Any pair of ports form a NULL-MODEM connection, where one port must be used by the PICSimLab and another by the application with serial communication.

5.3 Arduino IDE Integration (simavr)

For integrated use with the Arduino IDE, simply configure the serial port as explained in the Chapter [Serial Communication](#) and load the Arduino bootloader. The bootloader can be loaded from the “Tools->Arduino bootloader” menu.

In Windows, considering com0com making a NULL-MODEM connection between COM1 and COM2, simply connect the PICSimLab on the COM1 port (defined in configuration window) and the Arduino IDE on the COM2 port or vice versa.

On Linux the operation is the same, but using for example the ports /dev/tnt2 and /dev/tnt3.

In Linux for the virtual ports to be detected in Arduino IDE 1.8.xx it is necessary to replace the library lib/liblistSerials.j.so of the Arduino with a version which support the detection of ttyOtty ports, that can be downloaded in the link [listSerialC with ttyOtty support](#). The Arduino IDE 2.0 does not yet support virtual port detection, consider using IDE version 1.8 or vscode.

Chapter 6

Backend Simulators

PICSimLab currently supports five backend simulators: [picsim](#), [simavr](#), [uCsim](#), [gpsim](#) and [qemu](#) ([stm32](#) and [esp32](#)).

The type of debug interface depends on the backend simulator utilized.

6.1 PICsim

“[PICsim](#) emulates some PIC microcontroller and peripherals such as USART and timers, the simulator architecture permit easy implementation of external elements in c language. It can be used as a standalone simulator (picsim executable) or as a library in other programs (As in PICSimLab).”

6.1.1 MPLABX Integrated Debug

To use the [MPLABX](#) IDE for debug and program the [PicsimLab](#), install the plugin [com-picsim-picsimlab.nbm](#) in MPLABX.

The plugin connect to PICSimLab through a TCP socket using port 1234 (or other defined in configuration window), and you have to allow the access in the firewall.

[Tutorial: how to use MPLABX to program and debug PICsimLab.](#)

6.2 simavr

“[simavr](#) is a new AVR simulator for linux, or any platform that uses avr-gcc. It uses avr-gcc’s own register definition to simplify creating new targets for supported AVR devices. The core was made to be small and compact, and hackable so allow quick prototyping of an AVR project. The AVR core is now stable for use with parts with <= 128KB flash, and with preliminary support for the bigger parts. The simulator loads ELF files directly, and there is even a way to specify simulation parameters directly in the emulated code using an .elf section. You can also load multipart HEX files.”

6.2.1 avr-gdb Debug

With debug support enabled you can use avr-gdb to debug the code used in the simulator. Use the configuration window to choose between MDB (MPLABX) or GDB to debug AVR microcontrollers.

Use avr-gdb with the .elf file as the parameter:

```
avr-gdb compiled_file.elf
```

and the command below to connect (1234 is the default port):

```
target remote localhost:1234
```

Graphic debug mode can be made using [eclipse IDE](#) with [Sloeber Arduino plugin](#).

It is also possible to debug using [platformIO in VSCode](#), just add the configuration lines below in the project's **platformio.ini** file:

```
;upload_protocol = arduino
;upload_port = COM7
;upload_port = /dev/tnt3
;monitor_port = /dev/tnt3

upload_protocol = custom
upload_command = C:\"Program Files"\PicsimLab\picsimlab_tool.exe loadhex "$BUILD
;upload_command = /usr/bin/picsimlab_tool loadhex "$BUILD_DIR/firmware.hex"

build_type = debug
debug_tool = custom
debug_port = localhost:1234
debug_build_flags = -O2 -g
debug_init_break = tbreak setup
debug_init_cmds =
    define pio_reset_halt_target
    end
    define pio_reset_run_target
    end
    target extended-remote $DEBUG_PORT
    $LOAD_CMDS
    pio_reset_halt_target
    $INIT_BREAK
```

6.2.2 MPLABX Int. Debug

PICSimLab also supports using MPLABX to debug simavr. The configuration is the same as described in PICSim section: [MPLABX Integrated Debug](#). It's possible import and debug a Arduino sketch into MPLABX using the [Arduino import plugin](#).

6.3 qemu-stm32

“[Qemu STM32](#): QEMU with an STM32 microcontroller implementation”

6.3.1 arm-gdb Debug

With debug support enabled you can use arm-none-eabi-gdb (or gdb-multiarch) to debug the code used in the simulator.

Use arm-none-eabi-gdb with the .elf file as the parameter:

```
arm-none-eabi-gdb compiled_file.elf
```

and the command below to connect (1234 is the default port):

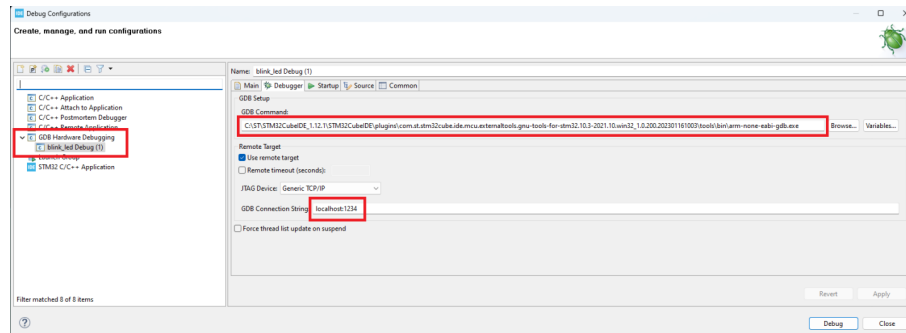
```
target extended-remote localhost:1234
```

Graphic debug mode can be made using [eclipse IDE](#) with [Eclipse Embedded CDT](#) or using [platformIO in VSCode](#), just add the configuration lines below in the project's **platformio.ini** file:

```
upload_protocol = custom
upload_command = C:\"Program Files"\PicsimLab\picsimlab_tool.exe loadbin "$BUILD
;upload_command = /usr/bin/picsimlab_tool loadbin "$BUILD_DIR/firmware.bin"

build_type = debug
debug_tool = custom
debug_port = localhost:1234
debug_build_flags = -O2 -g
debug_init_break = tbreak main
debug_init_cmds =
    define pio_reset_halt_target
        monitor system_reset
    end
    define pio_reset_run_target
        monitor system_reset
    end
target extended-remote $DEBUG_PORT
$LOAD_CMDS
pio_reset_halt_target
$INIT_BREAK
```

It's possible to configure STM32CubeIDE to connect and debug using PICSimLab:



6.4 qemu-ESP32

“**Qemu ESP32:** Qemu Emulator for TTGO TDisplay esp32 board. ”

For integrated use with the Arduino IDE or IDF `esptool.py` , simply configure the serial port as explained in the Chapter [Serial Communication](#) to flash PICSIMLAB ESP32-DevKitC as a real ESP32 board.

Attention! Qemu ESP32 don't support the QIO and QOUT flash modes, use only DIO or DOUT flash modes.

6.4.1 ESP32-gdb Debug

With debug support enabled you can use `xtensa-esp32-elf-gdb` to debug the code used in the simulator.

Use `xtensa-esp32-elf-gdb` with the `.elf` file as the parameter:

```
xtensa-esp32-elf-gdb compiled_file.elf
```

and the command below to connect (1234 is the default port):

```
target extended-remote localhost:1234
```

Graphic debug mode can be made using [platformIO in VSCode](#), just add the configuration lines below in the project's `platformio.ini` file:

```
;upload_protocol = esptool
;upload_port = COM7
;upload_port = /dev/tnt2
```

```
upload_protocol = custom
upload_command = C:\"Program Files"\PicsimLab\picsimlab_tool.exe loadbin "$BUILD
;upload_command = /usr/bin/picsimlab_tool loadbin "$BUILD_DIR/firmware.bin"
```

```
build_type = debug
```

```
debug_tool = custom
debug_port = localhost:1234
debug_build_flags = -O2 -g
debug_init_break = tbreak main
debug_init_cmds =
    define pio_reset_halt_target
        monitor system_reset
    end
    define pio_reset_run_target
        monitor system_reset
    end
    target extended-remote $DEBUG_PORT
    $LOAD_CMDS
    pio_reset_halt_target
    $INIT_BREAK
```

Compile, and upload the code to PICSimLab before starting Debug.

6.5 uCsim

“[uCsim](#) Software simulator for microcontrollers. uCsim can be used to simulate microcontrollers. It supports MCS51 family, AVR core, Z80, HC08, ST7, STM8, TLCS90, XA51 and Pdauk. It can run on Linux, Windows, OSX, BSD, and other systems.”

6.5.1 uCsim Debug

The uCsim debug console can be accessed with the telnet (1234 is the default port):

```
telnet localhost 1234
```

All [uCsim commands](#) are supported.

For windows users [putty telnet client](#) is a good option to access the uCsim console.

6.6 gpsim

“[gpsim](#) is a full-featured software simulator for Microchip PIC microcontrollers distributed under the GNU General Public License, Version 2 or higher, and some of its libraries under GNU Lesser General Public License, Version 2 or higher.

gpsim has been designed to be as accurate as possible. Accuracy includes the entire PIC - from the core to the I/O pins and including ALL of the internal peripherals. Thus it's possible to create stimuli and tie them to the I/O pins and test the PIC the same PIC the same way you would in the real world.”

6.7 Remote

This is experimental support to allow other simulators acting as microcontrollers to control PICSimLab remotely (TCP/IP).

Chapter 7

Tools

7.1 Serial Terminal

Open the serial terminal (with system application associated with .sterm file extension), the default application is the [CuteCom](#).

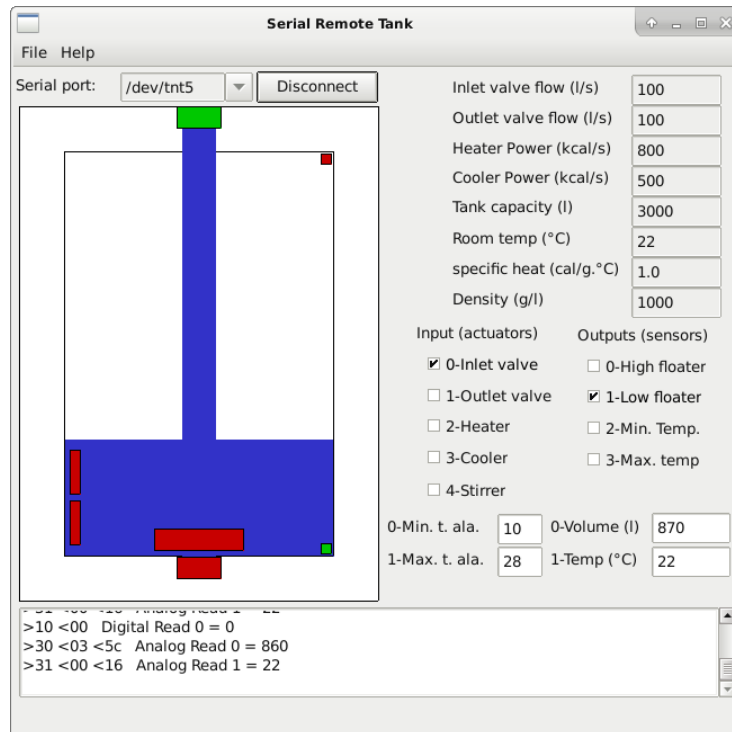


A serial terminal is used to send and receive data over a serial communication channel. The use of this terminal can be replaced by others like the Arduino IDE serial monitor.

To use this tool with PICSimLab you first need to configure a virtual serial port as described in Chapter: [Serial Communication](#). It is possible to use this tool with a real serial port connected to a real device.

7.2 Serial Remote Tank

The serial remote tank is a tank simulator controlled by a serial communication protocol. The tank has several sensors and actuators that can be read and controlled using the communication protocol. The parameters of the serial communication port must be 19200 8N1.



To use this tool with PICSimLab you first need to configure a virtual serial port as described in Chapter: [Serial Communication](#). It is possible to use this tool with a real serial port connected to a real device.

7.2.1 Actuators

Digital inputs:

1. Inlet valve
2. Outlet valve
3. Heater
4. Cooler
5. Stirrer

Analog inputs:

1. Minimal temperature alarm trigger level
2. Maximal temperature alarm trigger level

7.2.2 Sensors

Digital outputs:

1. High floater
2. Low floater
3. Minimal temperature
4. Maximal temperature

Analog outputs:

1. Volume
2. Temperature

7.2.3 Communication Protocol

Writing on Digital Input

Sent one byte in 0x0N hexadecimal format where N is the number of input followed by a second byte with value 0x00 for disable or 0x01 for enable.

Example to turn on the input 2:

```
Serial_write(0x02);  
Serial_write(0x01);
```

Reading Digital Output

Sent one byte in 0x1N hexadecimal format where N is the number of output and read one byte. The byte readed have value 0x00 for disable or 0x01 for enable.

Example to read output 3:

```
Serial_write(0x13);  
valor=Serial_read(0);
```

Writing on Analog Input

Sent one byte in 0x2N hexadecimal format where N is the number of input followed by two bytes with the 16 bits value.

Example to write the value 230 on analog input 1:

```
Serial_write(0x21);  
valor=230;  
Serial_write((valor&0xFF00)>>8);  
Serial_write(valor&0x00FF);
```

Reading Analog Output

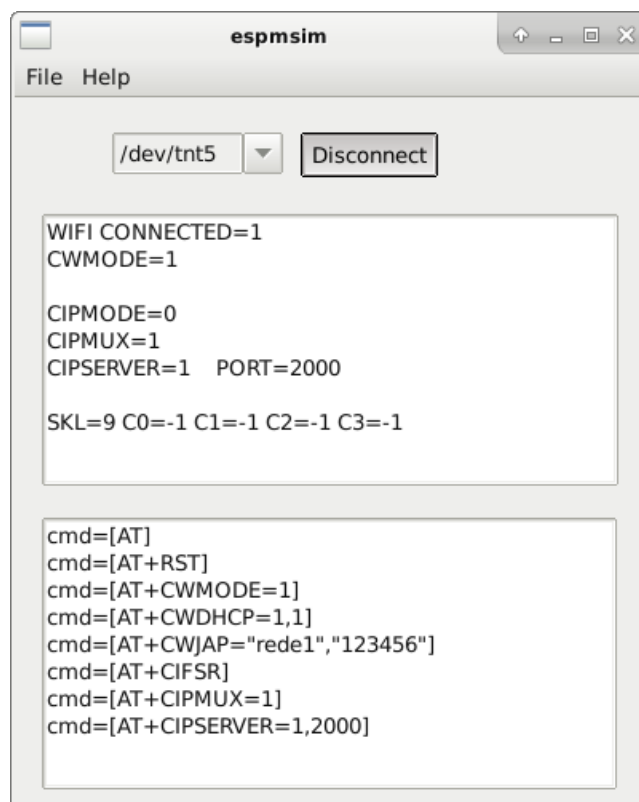
Sent one byte in 0x3N hexadecimal format where N is the number of output and read two bytes to form the 16 bits value.

Example to read analog output 2:

```
Serial_write(0x32);  
valorh=Serial_read(0);  
valorl=Serial_read(0);  
valor=(valorh<<8)|valorl;
```

7.3 Esp8266 Modem Simulator

The ESP8266 modem simulator emulates the operation of an esp8266 with wifi modem firmware. Communication is done using a serial channel via AT modem commands. The parameters of the serial communication port must be 115200 8N1.



To use this tool with PICSimLab you first need to configure a virtual serial port as described in Chapter: [Serial Communication](#). It is possible to use this tool with a real serial port connected to a real device.

7.3.1 Supported Commands

- AT
- AT+RST
- AT+GMR
- AT+CWMODE=1
- AT+CWDHCP=1,1
- AT+CWLAP
- AT+CWJAP="rede1","123456"
- AT+CIFSR
- AT+CIPMUX=1
- AT+CIPSERVER=1,2000
- AT+CIPSEND=0,10
- AT+CIPCLOSE=0

7.4 Arduino Bootloader

This menu option load PICSimLab microcontroller with Arduino serial bootloader. The microcontroller with the bootloader loaded can be programmed directly by the Arduino IDE or using the avrdude program.

To use this tool with PICSimLab you first need to configure a virtual serial port as described in Chapter: [Serial Communication](#).

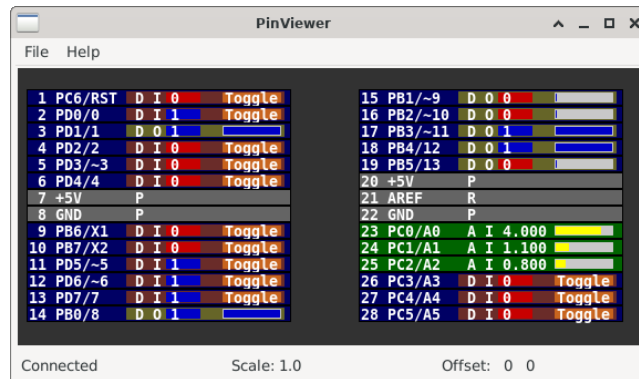
7.5 MPLABX Debugger Plugin

This menu option open the web page to download the MPLABX Debugger Plugin.

The plugin must be installed on MPLABX to allow debugging and programming PICSimLab (PICs and AVR) from the IDE, like a real tool for debugging and programming.

7.6 Pin Viewer

The PinViewer connects to PICSimLab through the [rcontrol interface](#) and allows viewing the status and direction of all microcontroller pins. It is also possible to change the state of the digital pins and adjust the voltage value on the analog pins configured as input. Pins configured as outputs also show the average value, useful for evaluating the functioning of PWM outputs.



7.7 picsimlab_tool

The **picsimlab_tool** is a simple command line tool to update the firmware running in the PICSimLab simulator. It can be used integrated in development tools (like platformio).

After open the PICSimLab you can use the **loadhex** with boards based on AVR or PICs:

```
picsimlab_tool loadhex file.hex
```

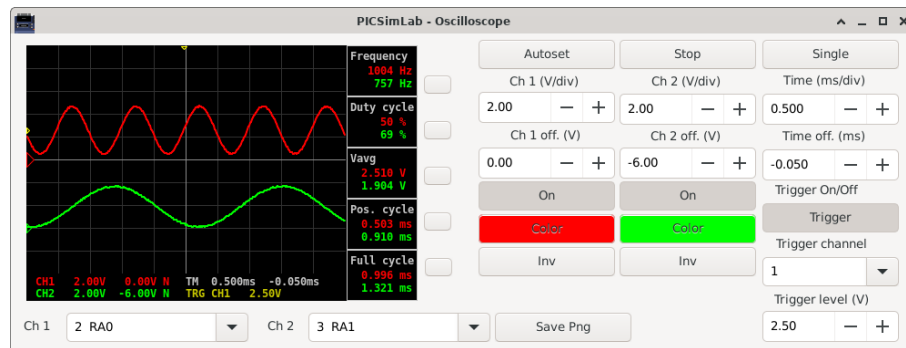
And use the **loadbin** with boards based on ESP32 or STM32.

```
picsimlab_tool loadbin file.bin
```

Chapter 8

Oscilloscope

The PICSimLab has a basic two-channel oscilloscope that can be used to view the signal on any pin of the microcontroller. The oscilloscope can be accessed through the “Modules->Oscilloscope” menu.



The oscilloscope supports up to 4 simultaneous measurements of the type:

- Vmax
- Vmin
- Vavg
- Vrms
- Frequency
- Duty cycle
- Pos. cycle
- Neg. cycle
- Full cycle

Chapter 9

Spare Parts

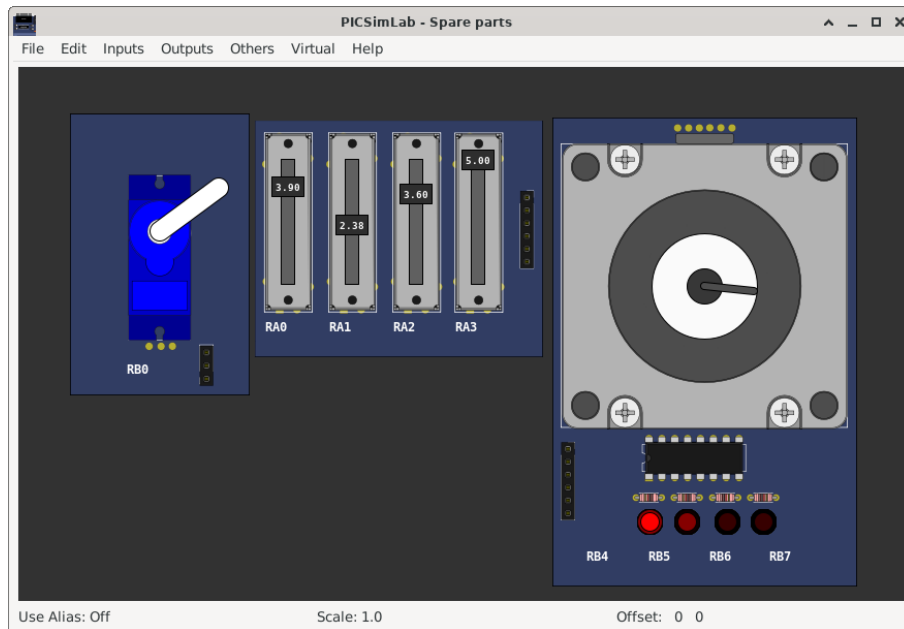
The PICSimLab has a window that allows the connection of spare parts to the micro-controller, it can be accessed through the menu “ Modules-> Spare parts ”.

The main window has the menu with the following functions:

- File
 - New configuration - Clear the spare parts window
 - Save configuration - Saves the current settings of the spare parts into .pcf file
 - Load configuration - Loads the settings from .pcf file
 - Save pin alias - Saves the current pin alias to .ppa text file
 - Load pin alias - Loads the pin alias from .ppa file
- Edit
 - Clear pin alias - Clear the pin alias
 - Toggle pin alias - Enable/Disable pin alias use
 - Edit pin alias - Open current pin alias .ppa file in text editor
 - Reload pin alias - Reload the current .ppa pin alias file (need after edit .ppa file)
 - Zoom in - Increase draw scale
 - Zoom out - Decrease draw scale
- Inputs
 - ADXL345 (Accel) - Adds a I2C/SPI accelerometer (only raw values)
 - BMP180 (Pressure I2C) - Adds a I2C pressure and temperature sensor
 - BMP280 (Pressure I2C) - Adds a I2C/SPI pressure and temperature sensor
 - DHT11 (Temp. Hum.) - Adds a humidity and temperature sensor

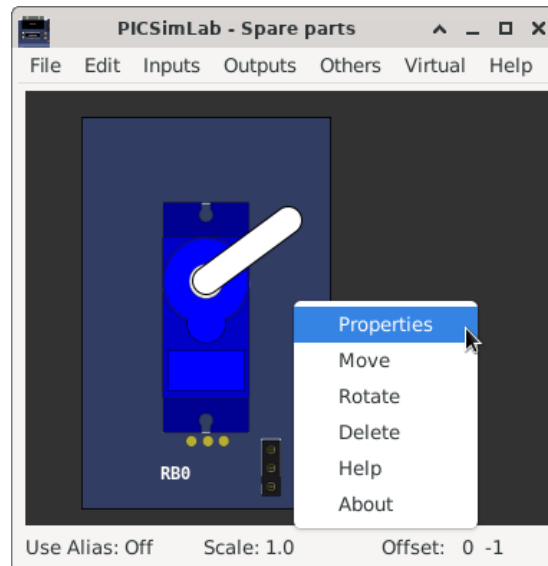
- DHT22 (Temp. Hum.) - Adds a humidity and temperature sensor
 - DS1621 (Temperature I2C) - Adds a I2C temperature sensor
 - DS18B20 (Temperature) - Adds a 1-Wire temperature sensor
 - Encoder - Adds a rotary quadrature encoder with push button
 - FM50 (Temperature) - Adds a analog temperature sensor
 - Fixed Voltage - Adds a analog fixed voltage reference
 - Gamepad - Adds a gamepad
 - Gamepad (Analogic) - Adds a gamepad with one analogic output
 - HX711 (Load Cell Amp) - Adds a Load Cell with HX711 amplifier
 - Keypad - Adds one matrix keypad
 - LDR - Adds a light dependent resistor
 - LM35 (Temperature) - Adds a analog temperature sensor
 - MPU6050 - Adds a accelerometer and gyroscope (only raw values)
 - Potentiometers - Adds 4 potentiometers
 - Potentiometers (Rotary) - Adds 4 rotary potentiometers
 - Push Buttons - Adds 8 push buttons
 - Push Buttons (Analogic) - Adds 8 push buttons with analog output
 - SHT3X - Adds a analog temperature and humidity sensor
 - Switches - Adds eight switches
 - Ultrasonic HC-SR04 - Adds a ultrasonic range sensor
- Outputs
 - 7 Segments Display - Adds four multiplexed (or single) 7 segments displays
 - 7 Segments Display (w/dec) - Adds four multiplexed 7 segments displays with decoder
 - Buzzer - Adds a active/passive buzzer
 - DC Motor - Adds a DC motor with H-bridge and quadrature encoder
 - LCD hd44780 - Adds a text display hd44780
 - LCD hd44780 I2C - Adds a text display hd44780 connect to one PCF8574 I2C converter
 - LCD ili9340 - Adds a color graphic display ili9340 with touchscreen
 - LCD pcd8544 - Adds a monochrome graphic display pcd8544 (Nokia 5110)
 - LCD pcf8833 - Adds a color graphic display pcf8833
 - LCD ssd1306 - Adds a monochrome graphic display ssd1306
 - LED Matrix - Adds a 8x8 LED matrix with MAX72xx controller

- LEDs - Adds 8 red LEDs
 - RGB LED - Adds one RGB LED
 - RGB LED WS2812B - Adds one or multiple addressable RGB LED
 - Servo Motor - Adds a servo motor
 - Step Motor - Adds a step motor
- Others
 - ETH w5500 - Adds a ethernet shield w5500
 - IO 74xx573 - Adds a 74xx573 octal latch
 - IO 74xx595 - Adds a 74xx595 SIPO 8 bit shift register
 - IO MCP23017 - Adds a MCP23017 serial I2C IO expander
 - IO MCP23S17 - Adds a MCP23S17 serial SPI IO expander
 - IO MM74C922 - Adds a MM74C922 key encoder
 - IO PCF8574 - Adds a PCF8574 serial I2C IO expander
 - IO UART - Adds a UART serial port
 - Jumper Wires - Adds sixteen jumper wires
 - Logic Block - Adds one logic gate block
 - MEM 24CXXX - Adds a 24CXXX serial I2C EEPROM memory
 - RTC ds1307 - Adds a ds1307 real time clock
 - RTC pfc8563 - Adds a pfc8563 real time clock
 - SD Card - Adds a SD card shield
 - Temperature System - Adds a temperature control system
- Virtual
 - D. Transfer Function - Adds a discrete transfer function mathematical model
 - IO Virtual term - Adds a virtual serial terminal
 - Signal Generator - Adds a virtual signal generator
 - Text Box - Adds a static text box
 - VCD Dump - Adds a digital value file dump recorder
 - VCD Dump (Analogic) - Adds a analog value file dump recorder
 - VCD Play - Adds a digital value file dump player
- Help
 - Contents - Open Help window
 - About - Show message about author and version



After adding the part, with a right click of the mouse you can access the options menu of the part with the options:

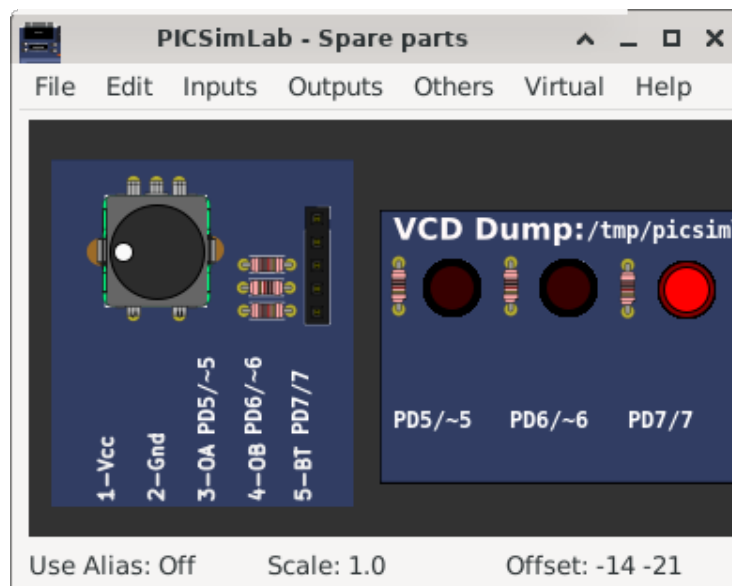
- Properties - Opens the connection settings window
- Move - Unlocks the part to move
- Rotate - Change the orientation of part
- Delete - Remove part
- Help - Open Help window of part
- About - Show message about author and version of part



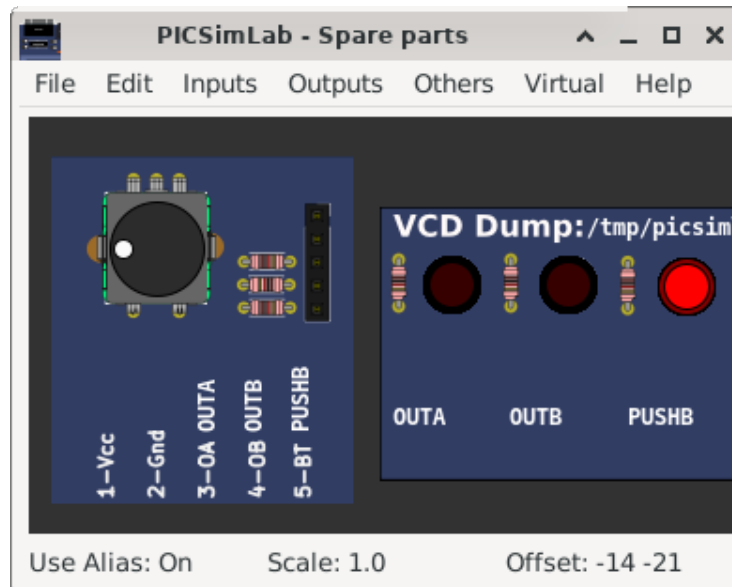
9.1 Pin Alias

The pin alias support allows the user to place custom names on the pins making it easy to identify according to the project.

When off the normal names are shown:



When on the alias names are shown:



To use:

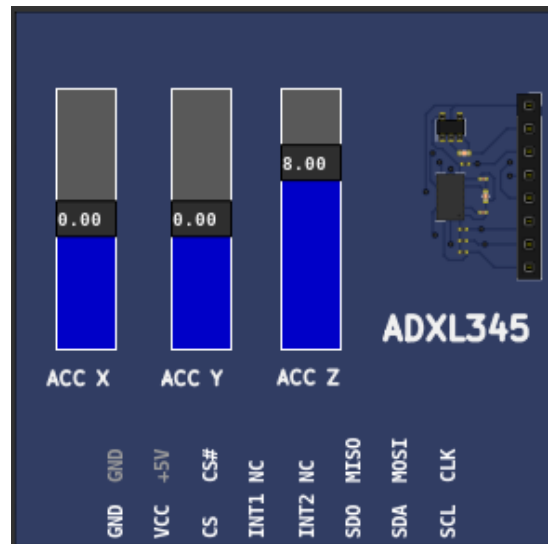
1. active the menu “Edit->Clear pin alias” to reset the pin alias file
2. active the menu “Edit->Edit pin alias” to open pin alias file, change the names, save and close.
3. active the menu “Edit->Reload pin alias” to load new alias
4. active the menu “Edit->Toggle pin alias” to show new alias

9.2 Inputs

9.2.1 ADXL345 (Accel)

This part is ADXL345 accelerometer with I2C/SPI interface. Only raw values are available.

To select SPI mode put low level in CS pin. In I2C mode (CS pin high), the address is 0x1D when SDO is NC or high and 0x53 when SDO pin is held in low.

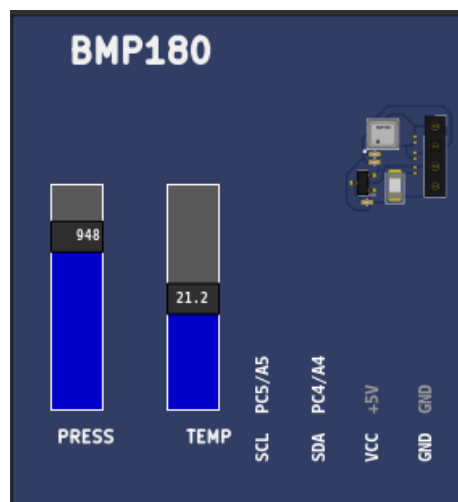


[Examples](#)

[Datasheet](#)

9.2.2 BMP180 (Pressure I2C)

This part is BMP180 I2C pressure and temperature sensor (address 0x77). The temperature range is -40 to 85 °C and the pressure range is 300 hPa to 1100 hPa.



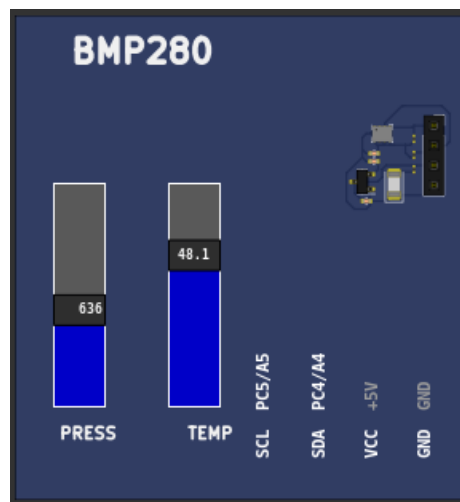
[Examples](#)

[Datasheet](#)

9.2.3 BMP280 (Pressure I2C)

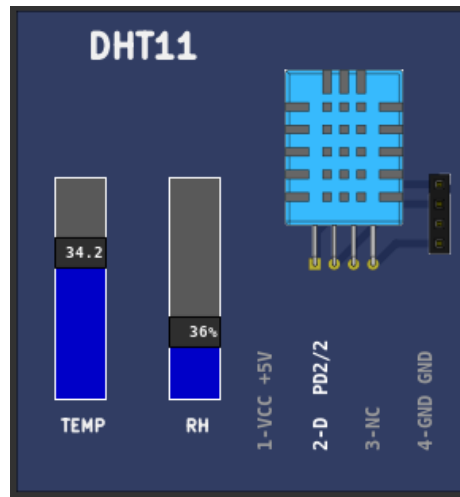
This part is BMP280 I2C/SPI pressure and temperature sensor. The temperature range is -40 to 85 °C and the pressure range is 300 hPa to 1100 hPa.

To select SPI mode put low level in CSB pin. In I2C mode (CSB pin high), the address is 0x77 when SDO is NC or high and 0x76 when SDO pin is held in low.

[Examples](#)[Datasheet](#)

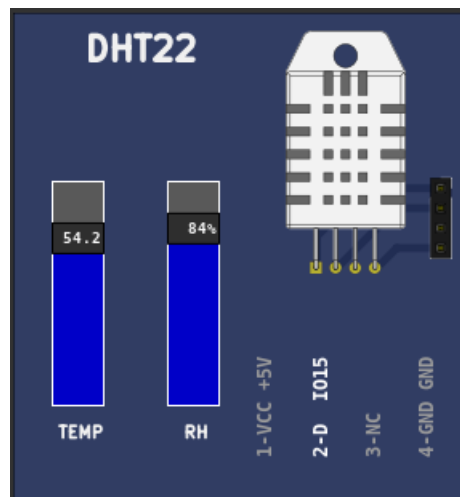
9.2.4 DHT11 (Temp. Hum.)

This part is DHT11 digital temperature and humidity sensor. The temperature range is 0 to 50 °C and the relative humidity range is 20 to 80 %.

[Examples](#)[Datasheet](#)

9.2.5 DHT22 (Temp. Hum.)

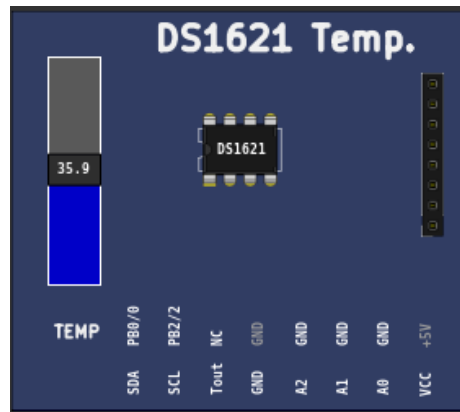
This part is DHT22 (AM2302) digital temperature and humidity sensor. The temperature range is -40 to 80 °C and the relative humidity range is 0 to 100 %.

[Examples](#)[Datasheet](#)

9.2.6 DS1621 (Temperature I2C)

This part is DS1621 I2C temperature sensor. The measurement range is -55 to 125 °C.

The I2C address range is 0x48 to 0x4F. If pins A2, A1 and A0 are not connect (NC) the default address is 0x48.

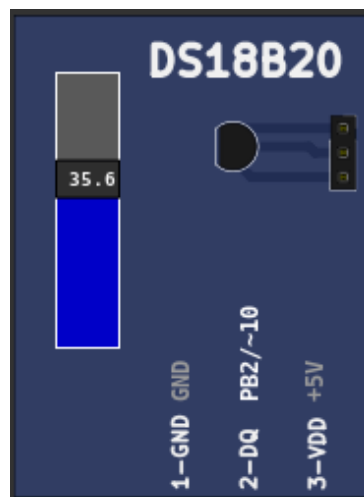


[Examples](#)

[Datasheet](#)

9.2.7 DS18B20 (Temperature)

This part is DS18B20 1-Wire temperature sensor. The measurement range is -40 to 80 °C.

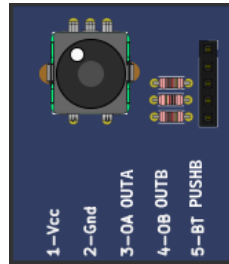


[Examples](#)

[Datasheet](#)

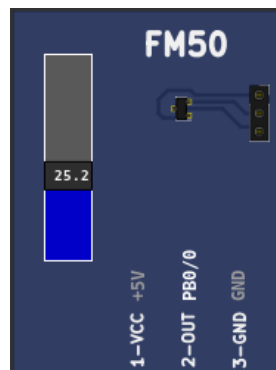
9.2.8 Encoder

This part is a rotary quadrature encoder (Ky-40) with push button. The output is twenty pulses per revolution.

[Examples](#)[Datasheet](#)

9.2.9 FM50 (Temperature)

This part is FM50 analog temperature sensor. The measurement range is -40 to 125 °C and voltage output is $10\text{mV}/^{\circ}\text{C} + 500\text{mV}$.

[Examples](#)[Datasheet](#)

9.2.10 Fixed Voltage

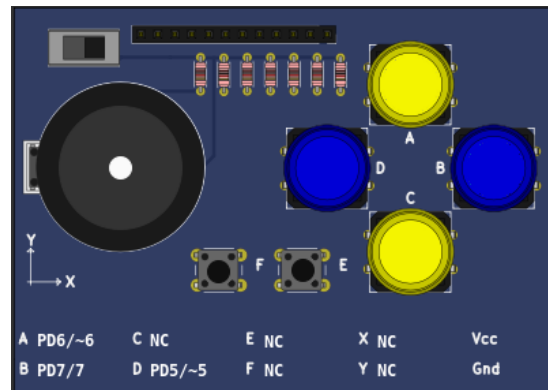
This part is analog fixed voltage reference. The value range is 0 to 5V.

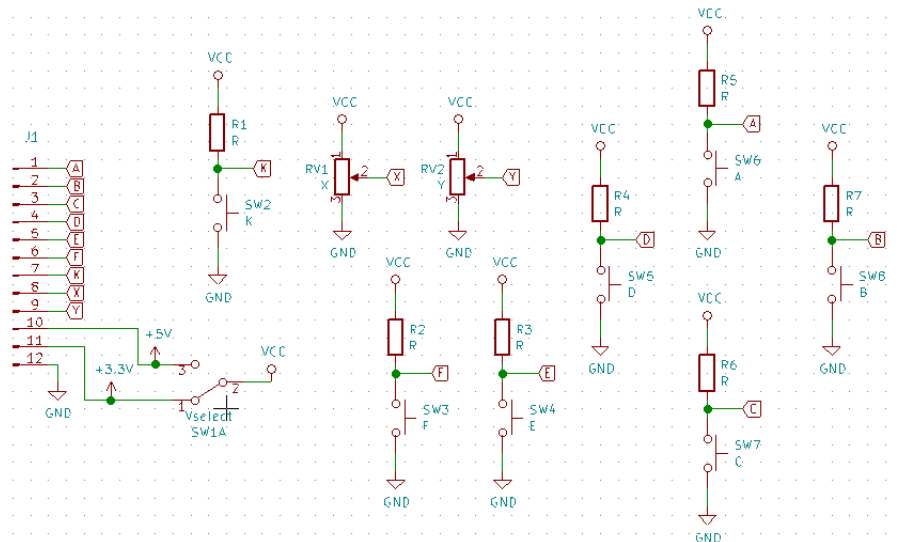


Examples

9.2.11 Gamepad

This part is a gamepad with two analog axis and 7 push buttons.





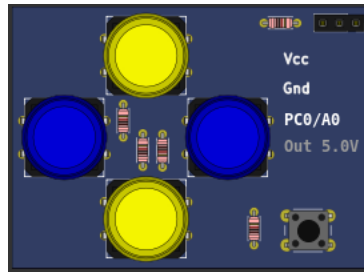
The gamepad can be controlled by keyboards keys:

- X axis - keys 'A' and 'D'
- Y axis - keys 'W' and 'S'
- Button A - key 'I'
- Button B - key 'L'
- Button C - key 'K'
- Button D - key 'J'
- Button E - key 'E'
- Button F - key 'O'
- Button K - key 'R'

Examples

9.2.12 Gamepad (Analogic)

This part is a gamepad with 5 push buttons and one analogic output.



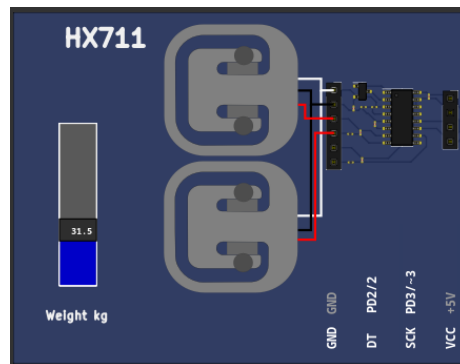
The gamepad can be controlled by keyboards keys:

- Button A - key 'L'
- Button B - key 'I'
- Button C - key 'K'
- Button D - key 'J'
- Button E - key 'O'

[Examples](#)

9.2.13 HX711 (Load Cell Amp)

This part is a 100Kg Load Cell with HX711 amplifier.



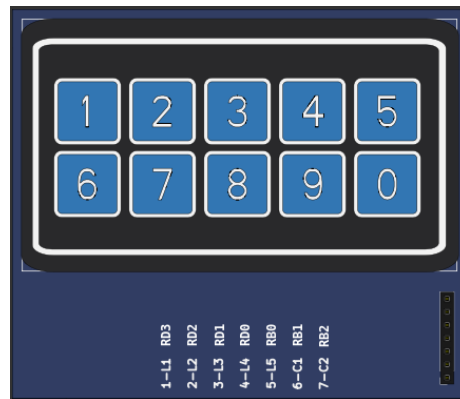
[Examples](#)

[Datasheet](#)

9.2.14 Keypad

It is a matrix keyboard configurable to 4x3 , 4x4 or 2x5 rows/columns.



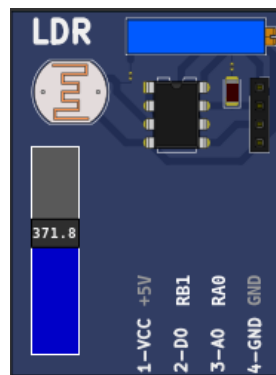


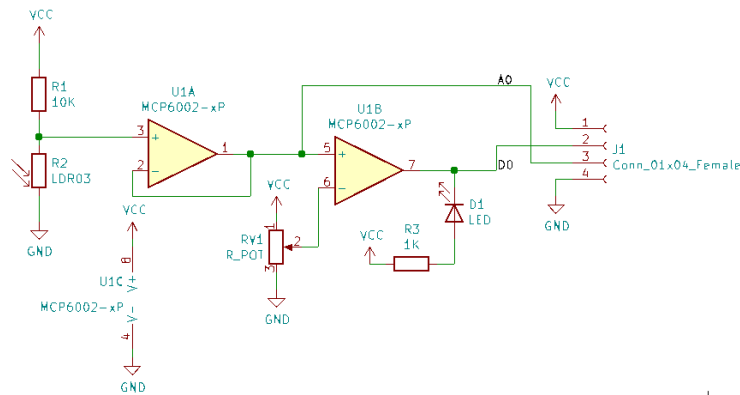
Examples

9.2.15 LDR

This part is light dependent resistor (LDR) connected in series with one 10K resistor. The analog output of the voltage divider is applied to one voltage follower and can be read directly from pin A0. The analog value from voltage follower is compared with one voltage threshold, the digital output of comparator and can be read directly from pin D0.

LDR Characteristics	Value
Gamma value at 100-10Lux	0.7
Light Resistance at 10Lux (25°C)	20K Ω



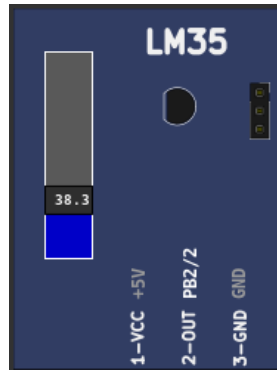


[Examples](#)

[Datasheet](#)

9.2.16 LM35 (Temperature)

This part is LM35 analog temperature sensor. The measurement range is 2 to 150 °C and voltage output is 10mV/°C.



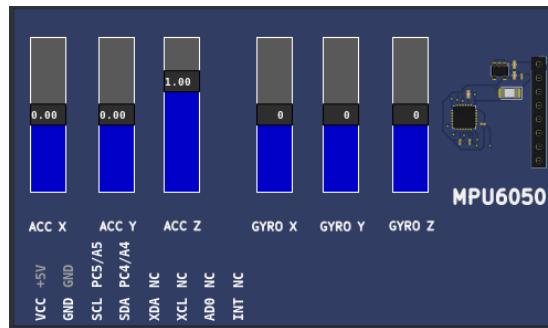
[Examples](#)

[Datasheet](#)

9.2.17 MPU6050

This part is MPU6050 accelerometer and gyroscope with I2C interface. Only raw values are available, DMP is not supported.

The I2C address is 0x68 when AD0 is NC or low and 0x69 when AD0 pin is held in high.

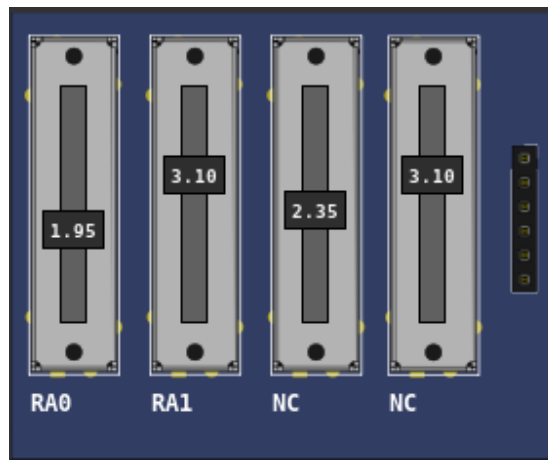


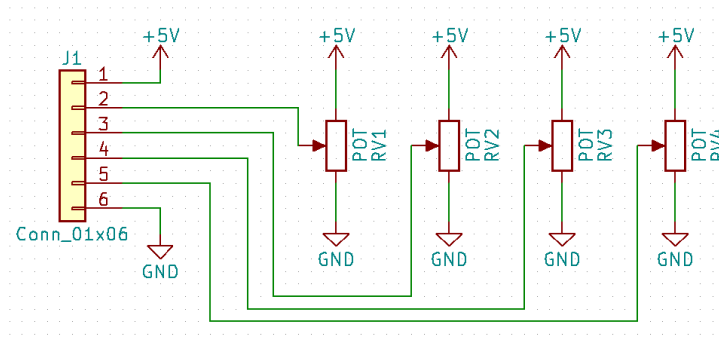
[Examples](#)

[Datasheet](#)

9.2.18 Potentiometers

This part is formed by 4 potentiometers connected between 0 and 5 volts, the output is connected to the cursor and varies within this voltage range.

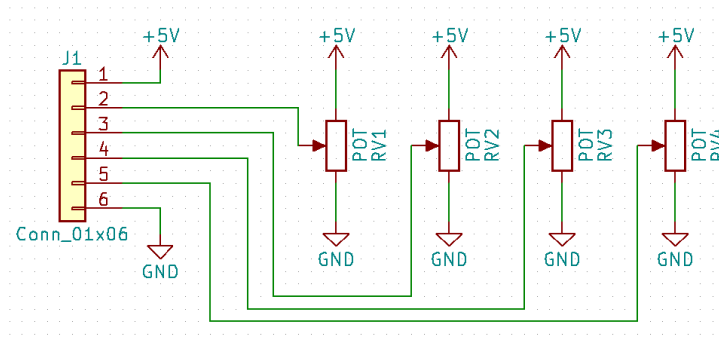
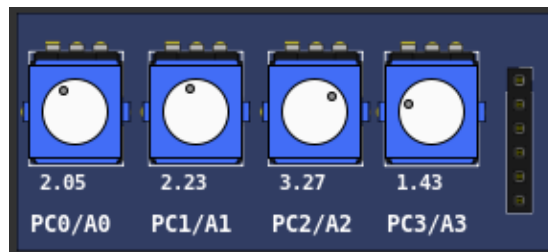




Examples

9.2.19 Potentiometers (Rotary)

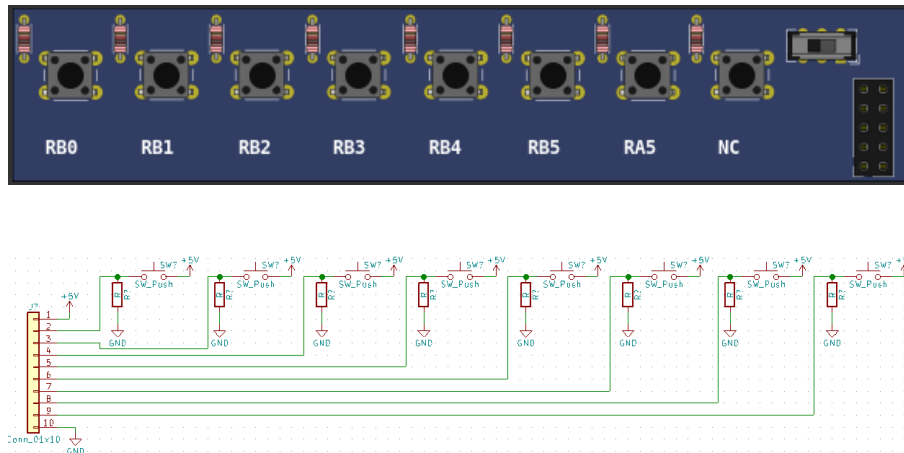
This part is formed by 4 rotary potentiometers connected between 0 and 5 volts, the output is connected to the cursor and varies within this voltage range.



Examples

9.2.20 Push Buttons

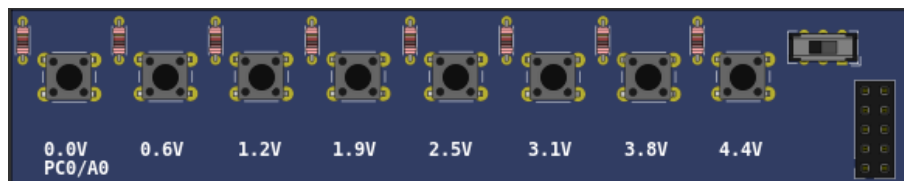
This part consists of 8 push buttons. The output active state can be configurable. The buttons have activation bounce effect emulation.



Examples

9.2.21 Push Buttons (Analogic)

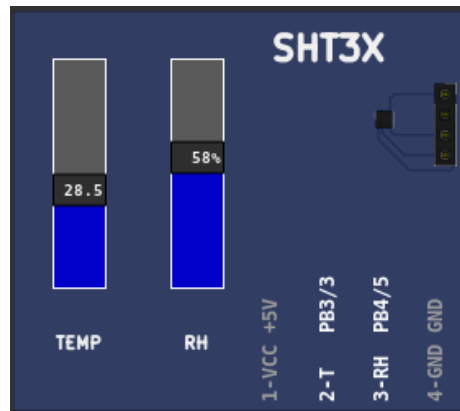
This part consists of 8 push buttons connected in a resistive ladder.



Examples

9.2.22 SHT3X (Temp. Hum.)

This part is SHT3X analog temperature and humidity sensor. The temperature range is -40 to 125 °C and voltage output is $22.85\text{mV}/^{\circ}\text{C} + 1.53\text{V}$. The relative humidity range is 0 to 100 % and voltage output is $40\text{mV}/\% + 500\text{mV}$.

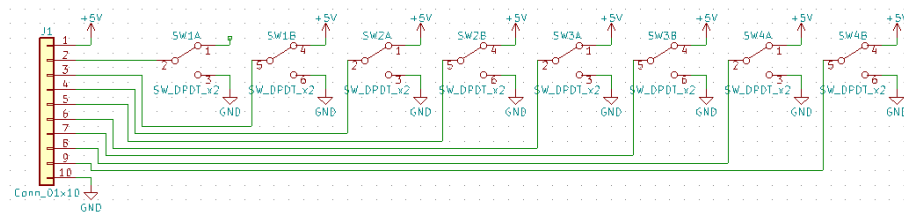


[Examples](#)

[Datasheet](#)

9.2.23 Switches

This part consists of 8 switches with on or off position (0 or 1). The switches have activation bounce effect emulation.



[Examples](#)

9.2.24 Ultrasonic HC-SR04

This part is ultrasonic range meter sensor.



[Examples](#)

[Datasheet](#)

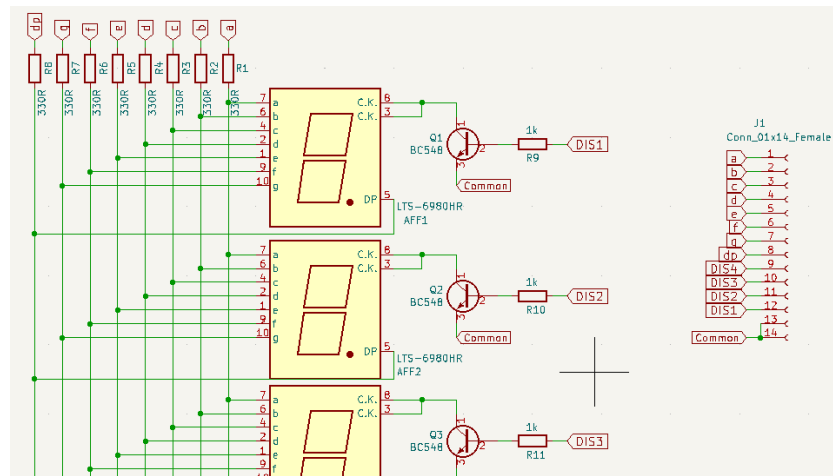
9.3 Outputs

9.3.1 7 Segments Display

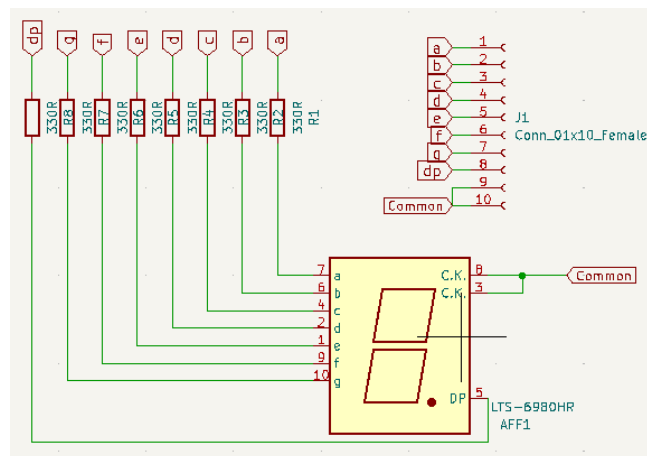
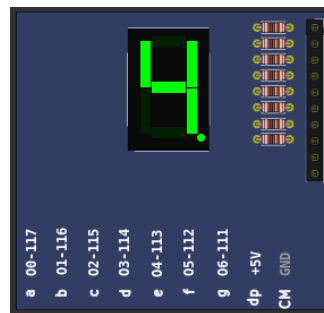
This part can be configured as four multiplexed or one single 7 segments display.

Four Multiplexed





Single Display

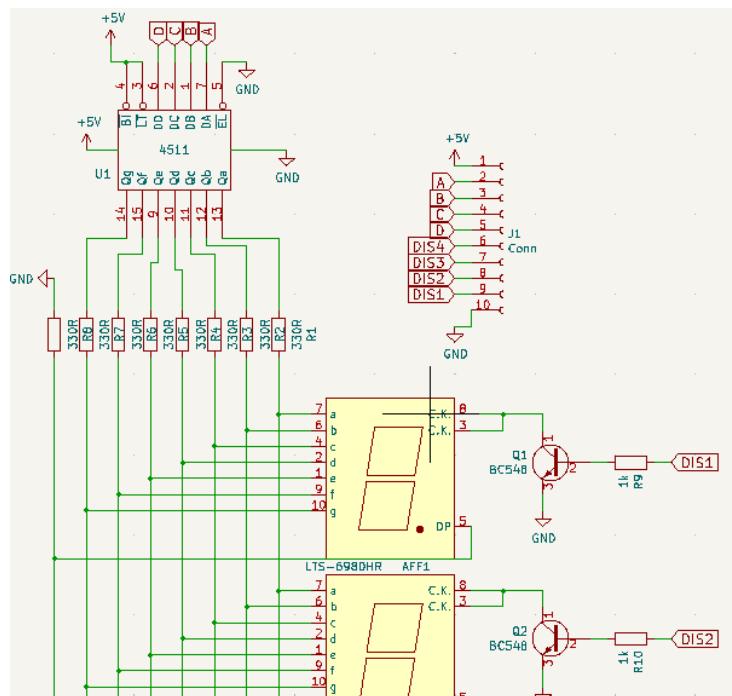
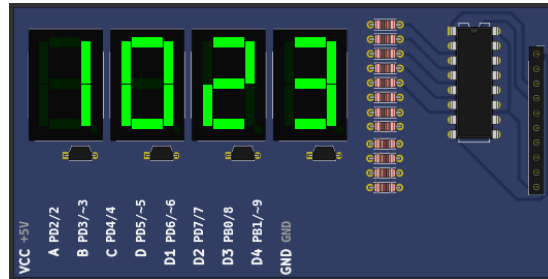


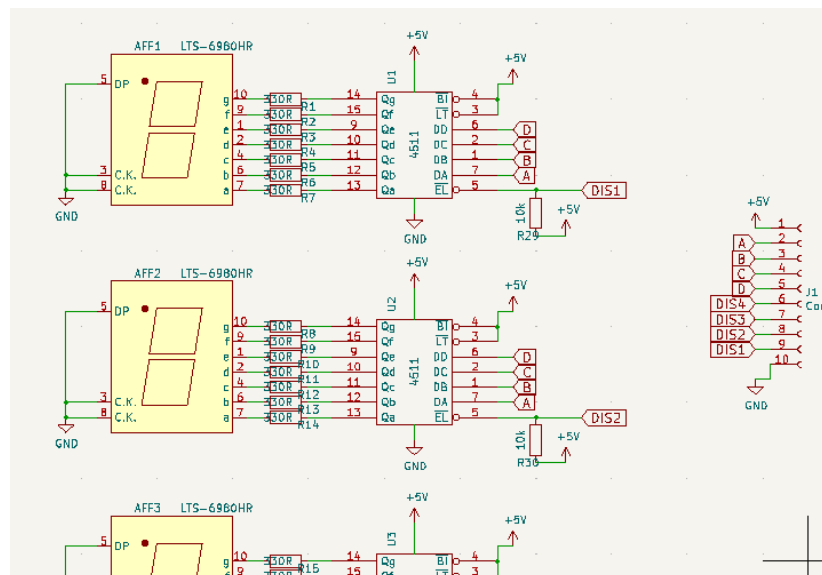
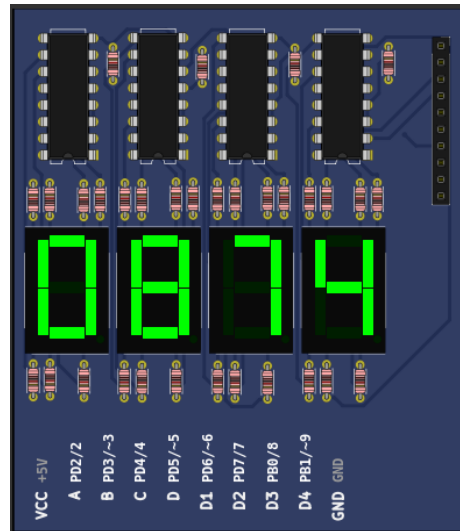
Examples

9.3.2 7 Segments Display (Decoder)

This is a four multiplexed 7 segments displays with BCD to 7 segments decoder (CD4511).

Four Multiplexed



Four with Latch**Examples****9.3.3 Buzzer**

This is a active/passive buzzer. The buzzer has 3 operating modes:

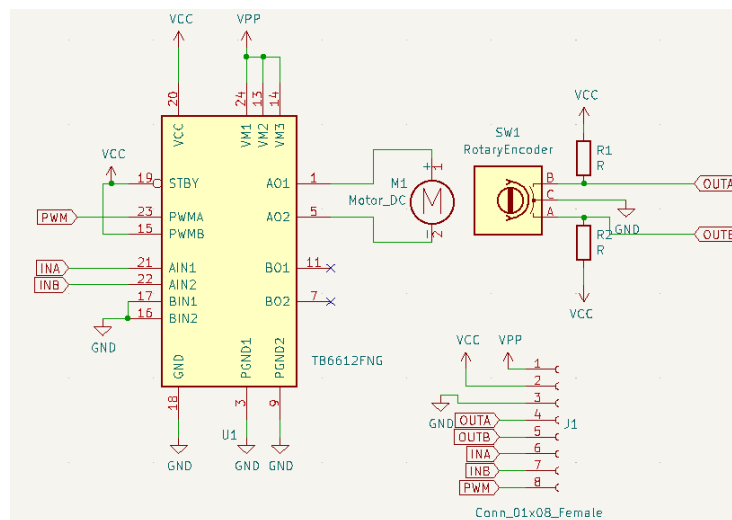
- **Active:** When powered, the buzzer emits a frequency of 440Hz.
- **Passive:** In this mode the values read from the input pin are sent directly to the sound card, this mode only works well if the simulation is in real time.
- **Tone:** This second passive mode measures the frequency on the input and updates the frequency of the buzzer every 100ms, its accuracy is less than the normal passive mode but it works better when the simulation is not in real time.



Examples

9.3.4 DC Motor

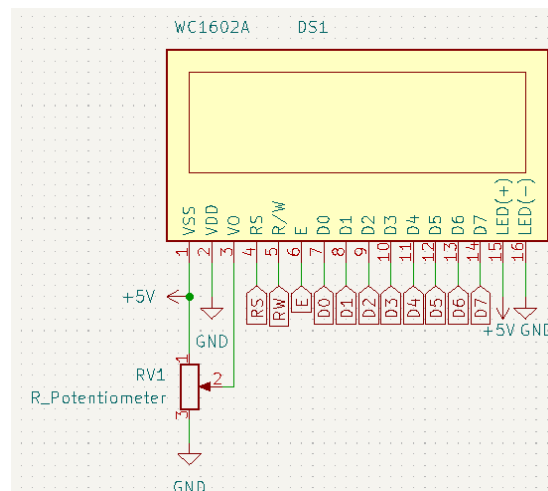
This part is DC motor with H-bridge driver and quadrature encoder.



Examples

9.3.5 LCD hd44780

This part is a text display with 2 (or 4) lines by 16 (or 20) columns.





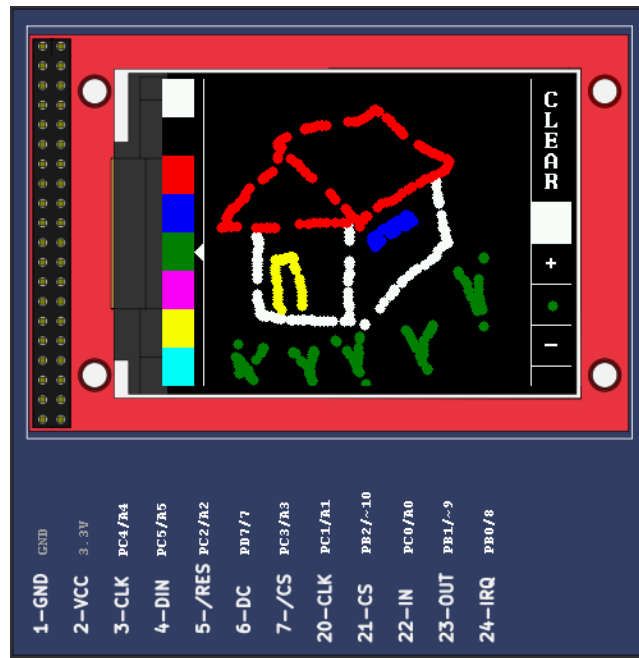


[Examples](#)

[Datasheet](#)

9.3.6 LCD ili9341

This part is a color graphic display with 240x320 pixels with touchscreen (xpt2046 controller). Only 4 SPI mode and 8 bits parallel mode is available.



[Examples](#)

[Datasheet ILI9341](#)

[Datasheet xpt2046](#)

9.3.7 LCD pcf8833

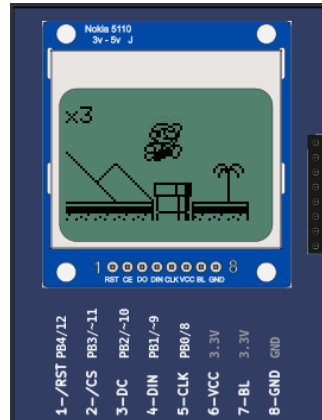
This part is a color graphic display with 132x132 pixels.



[Examples](#)

9.3.8 LCD pcd8544

This part is a monochrome graphic display with 48x84 pixels. (Nokia 5110)



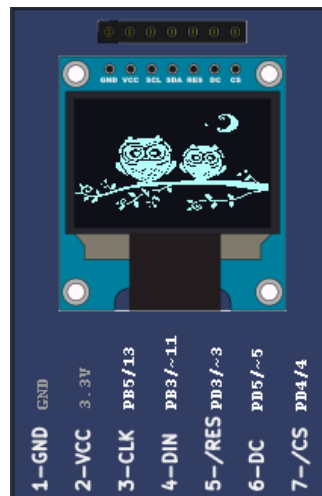
[Examples](#)

[Datasheet](#)

9.3.9 LCD ssd1306

This part is a monochrome oled graphic display with 128x64 pixels. The part suport I2C and 4 SPI serial mode.

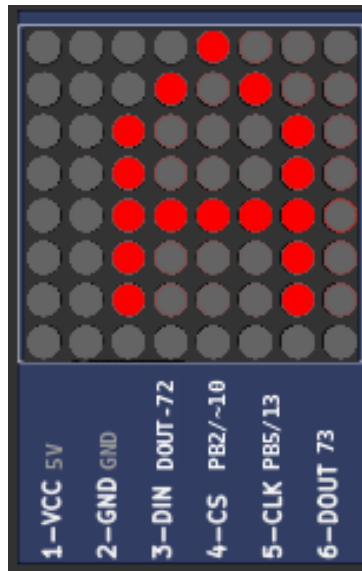
In the I2C mode address is 0x3c when DC pin is NC or low and 0x3d when DC pin is held in high.



[Examples](#)[Datasheet](#)

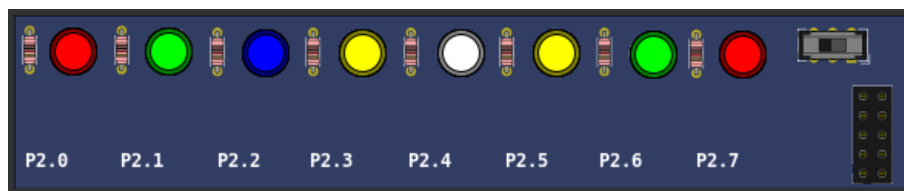
9.3.10 LED Matrix

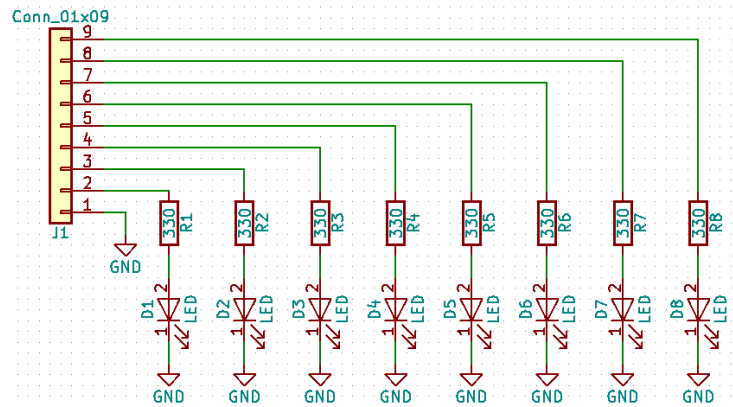
It is a 8x8 LED matrix with MAX72xx controller.

[Examples](#)

9.3.11 LEDs

This part is a bar of 8 independent colored LEDs.

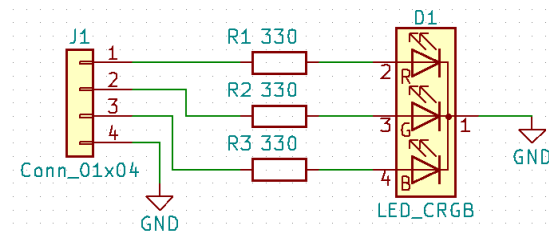
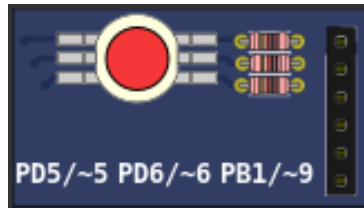




Examples

9.3.12 RGB LED

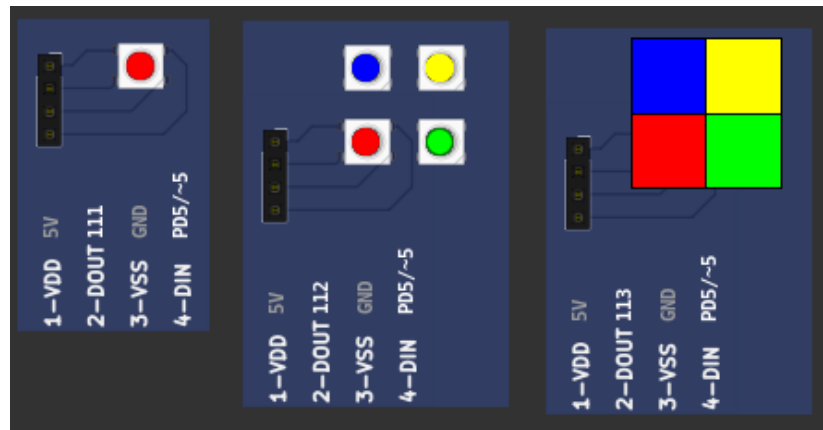
This part consists of a 4-pin RGB LED. Each color can be triggered independently. Using PWM it is possible to generate several colors by combining the 3 primary colors.



Examples

9.3.13 RGB LED WS2812B

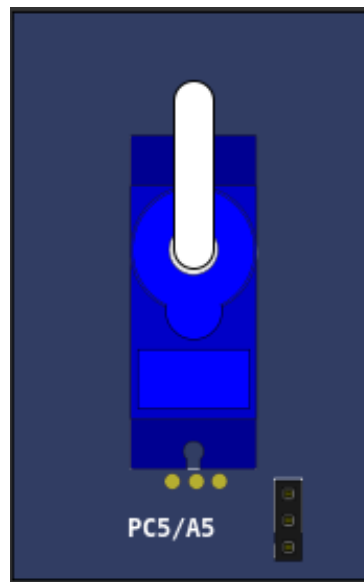
This part consists of a addressable RGB LED WS2812B. It is possible to set the number of rows and columns in the configuration window. The LED can be used with or without diffuser.

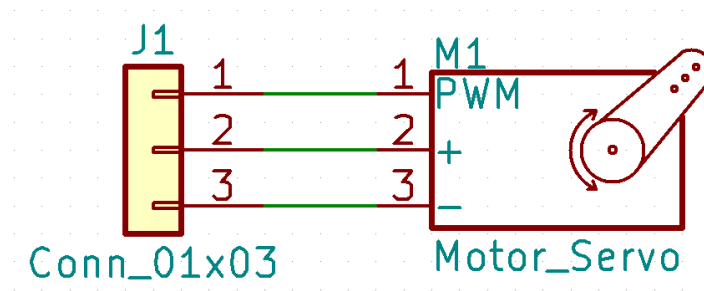


Examples

9.3.14 Servo Motor

The servo motor is a component that must be activated with a pulse of variable width from 1ms to 2ms every 20 ms. A pulse of 1ms positions the servo at -90° , one from 1.5ms to 0° and one from 2ms to 90° .

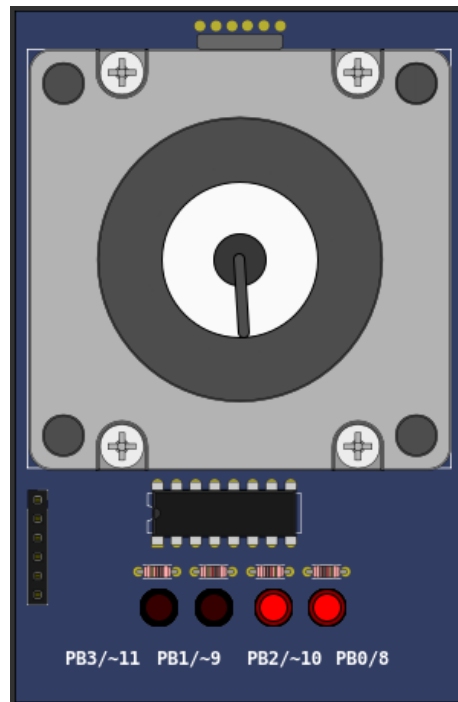


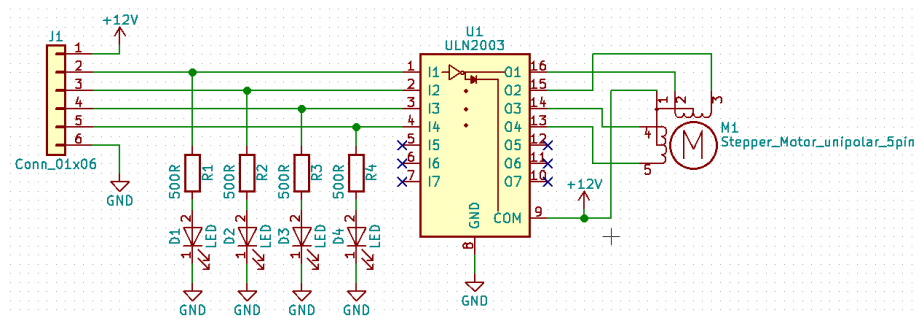


Examples

9.3.15 Step Motor

The stepper motor is a component with 4 coils that must be driven in the correct order to rotate the rotor. Each step of the motor is 1.8° .





Examples

9.4 Others

9.4.1 ETH w5500

This part is a ethernet shield w5500 with support to 8 sockets simultaneously.

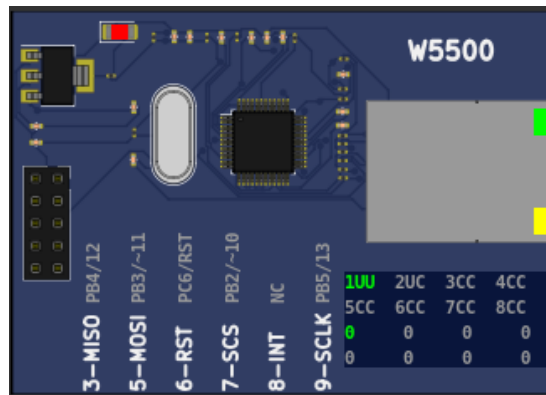
Only TCP/UDP unicast address sockets is supported. DHCP is emulated and return a fake ipv4 address.

All listening ports below 2000 are increased by 2000 to avoid operational system services ports. For example listening on port 80 becomes 2080.

w5500 Status Legend:

1° Letter - Type	2° Letter - Status	3° Letter - Error
C - Closed	C - Closed	B - Bind
T - TCP	I - Initialized	S - Send
U - UDP	L - Listen	R - Receive
M - MACRAW (don't supported)	S - Syn sent	L - Listen
	E - Established	U - Reuse
	W - Close wait	C - Connecting
	U - UDP	D - Shutdown
	M - MACRAW (don't supported)	

Click on connector to toggle link status.

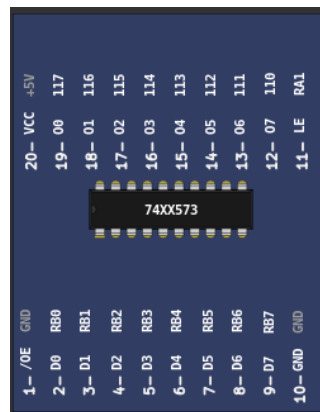


[Examples](#)

[Datasheet](#)

9.4.2 IO 74xx573

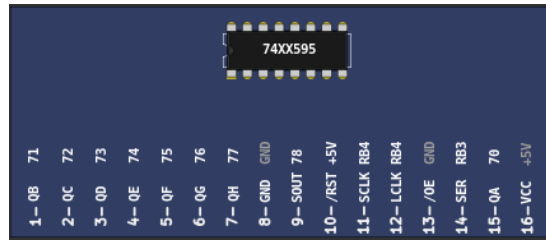
This is one 74xx573 octal latch.



[Examples](#)

9.4.3 IO 74xx595

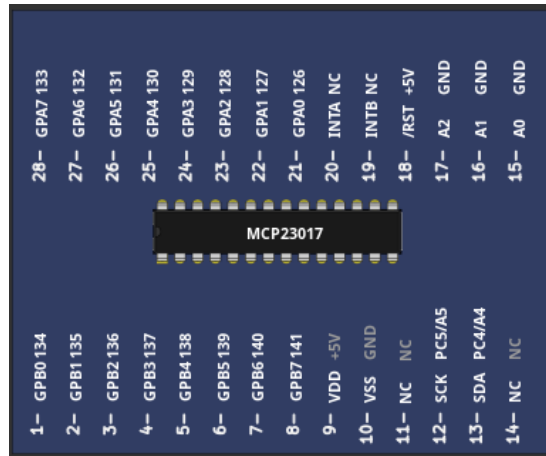
This is one 74xx595 serial input and parallel output 8 bit shift register.



Examples

9.4.4 IO MCP23017

It is a MCP23017 serial I2C IO expander part.

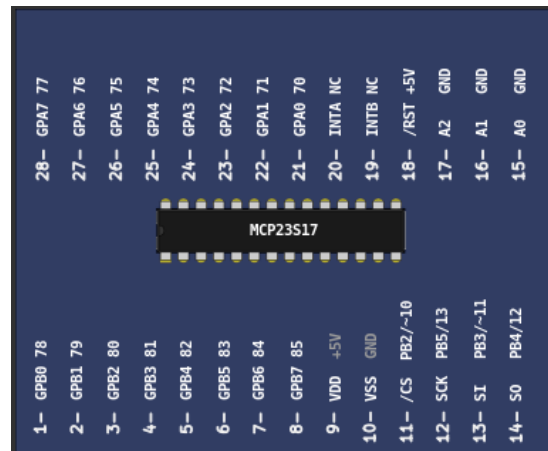


Examples

Datasheet

9.4.5 IO MCP23S17

It is a MCP23S17 serial SPI IO expander part.

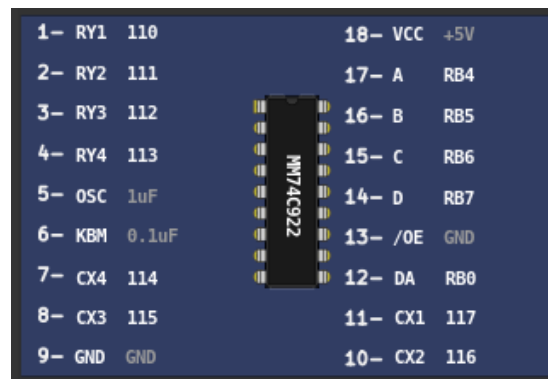


[Examples](#)

[Datasheet](#)

9.4.6 IO MM74C922

It is a MM74C922 key encoder.

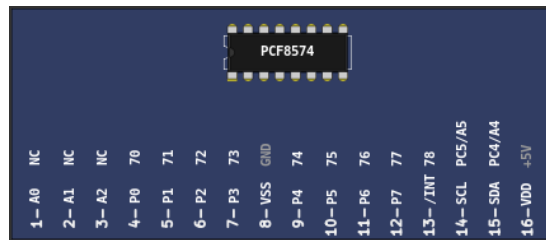


[Examples](#)

[Datasheet](#)

9.4.7 IO PCF8574

It is a PCF8574 serial I2C IO expander (Address 0x20 to 0x27).

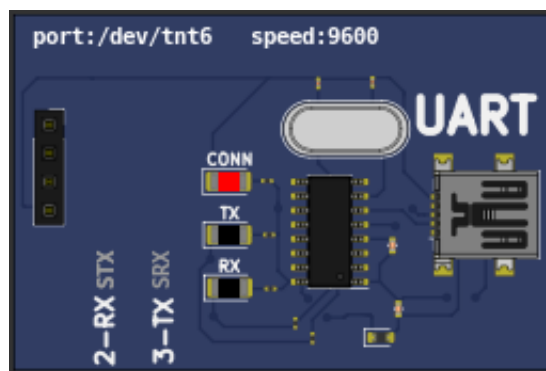


[Examples](#)

[Datasheet](#)

9.4.8 IO UART

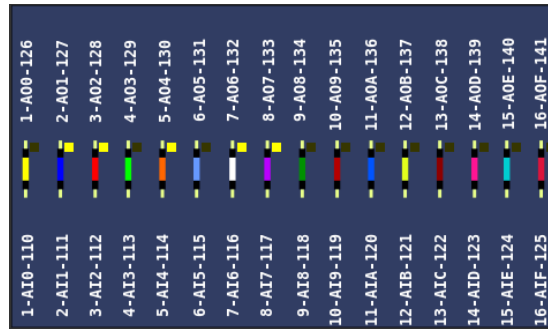
This part is a UART serial port. This part connects the hardware/software UART IO pins of microcontroller to one real/virtual PC serial port. To use virtual port is need to install a virtual port software, as described in Chapter: [Serial Communication](#). The serial communication uses the 8N1 format.



[Examples](#)

9.4.9 Jumper Wires

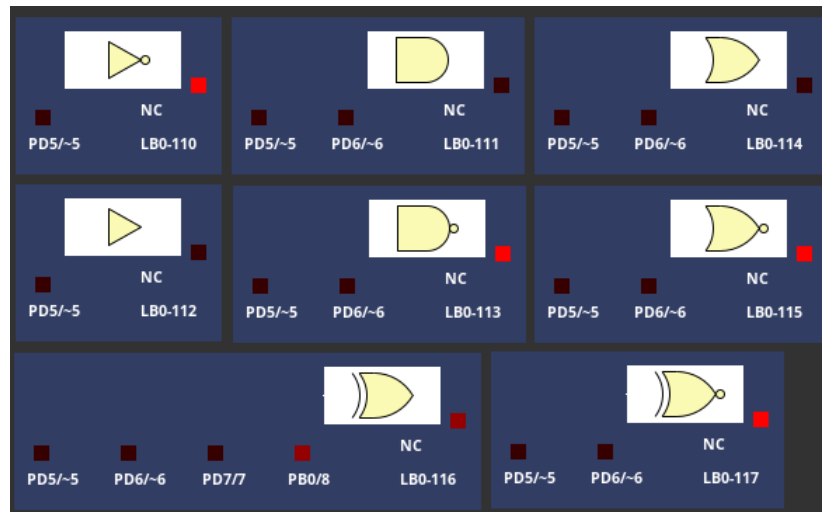
This part are formed by sixteen jumper wires. Each jumper has one input and one output. The jumper input must be connected to one pin output, the jumper output can be connected to multiple pin inputs. The jumper can be used to connect microcontroller pins or make connection between spare parts pins.



Examples

9.4.10 Logic Block

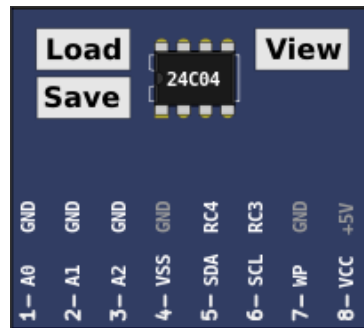
This is a logic gate block. They implements the NOT, BUFFER, AND, NAND, OR, NOR, XOR and XNOR logic gates. All gates have one simulation time step delay.



Examples

9.4.11 MEM 24CXXX

It is a 24CXXX serial I2C EEPROM part. There are support to the models 24C04 and 24C512.



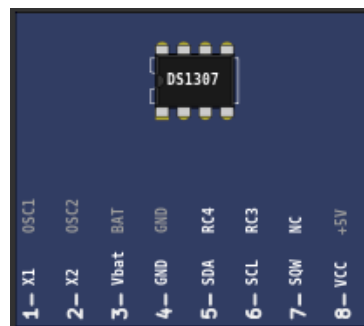
[Examples](#)

[Datasheet 24C04](#)

[Datasheet 24C512](#)

9.4.12 RTC ds1307

This part is a ds1307 real time clock with serial I2C interface (Address 0x68).

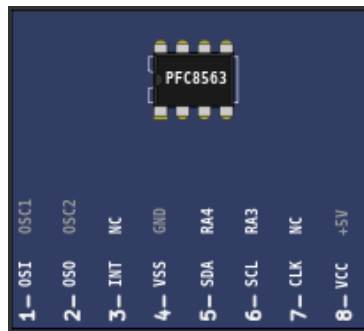


[Examples](#)

[Datasheet](#)

9.4.13 RTC pfc8563

This part is a pfc8563 real time clock with serial I2C interface (Address 0x51).



[Examples](#)

[Datasheet](#)

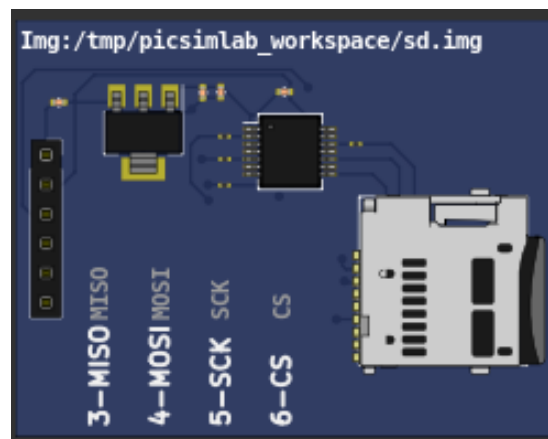
9.4.14 SD Card

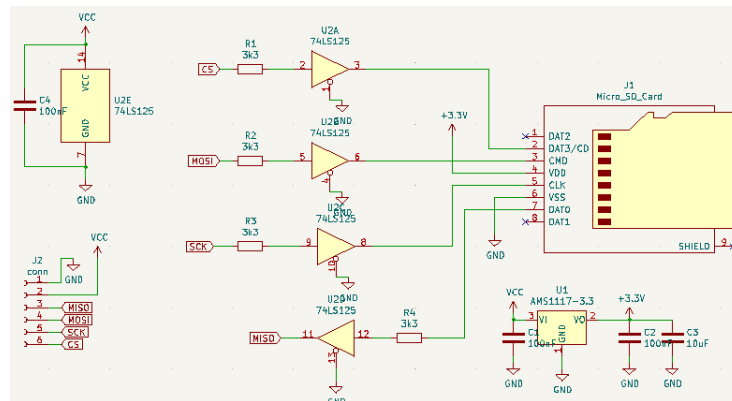
This part is a SD Card shield. It's necessary set one sd card file image before use it. (Click on SD card connector to open file dialog)

On Linux one empty image can be created with this command:

```
dd if=/dev/zero of=sd.img bs=1M count=32
```

This empty image can be used with raw SD card access, to work with FAT file system the image need to be formatted before the use. (using [SdFormatter.ino](#) for example)

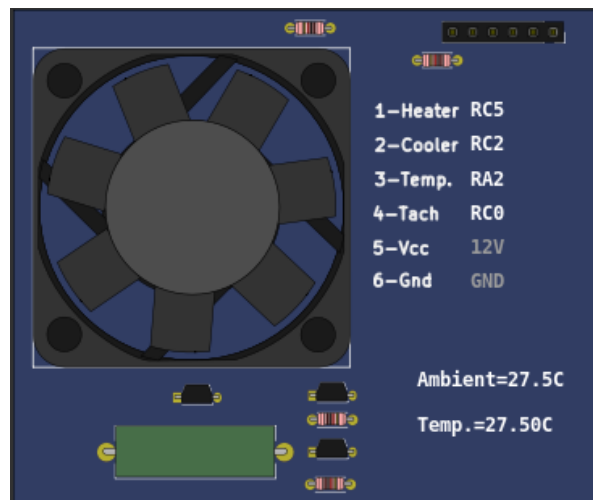




Examples

9.4.15 Temperature System

This part is a temperature control system. The temperature control system consists of a heating resistor, an LM35 temperature sensor, a cooler and an infrared tachometer.

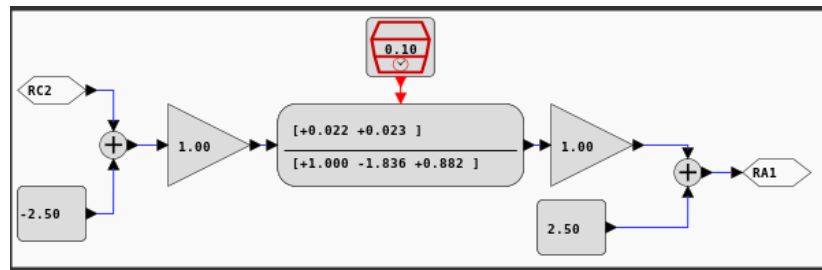


Examples

9.5 Virtual

9.5.1 D. Transfer Function

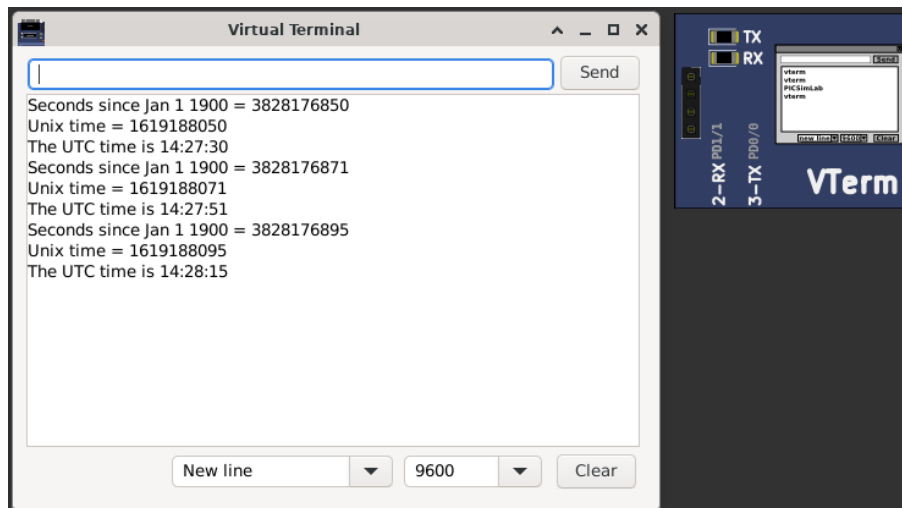
This is a discrete transfer function mathematical model.



Examples

9.5.2 IO Virtual Term

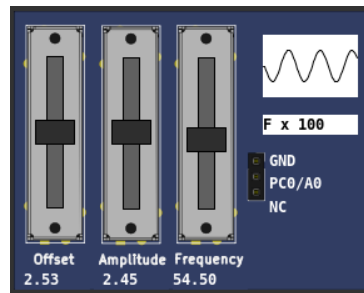
This part is a virtual serial terminal. This part can be used to read and write RX/TX pins UART signals. To use this part are don't need to use or install one virtual serial ports on computer. Click on terminal picture to open the terminal window. The serial communication uses the 8N1 format.



Examples

9.5.3 Signal Generator

This part is a virtual signal generator with support for sine, square and triangular waves generation with amplitude and frequency adjustment.



Examples

9.5.4 Text Box

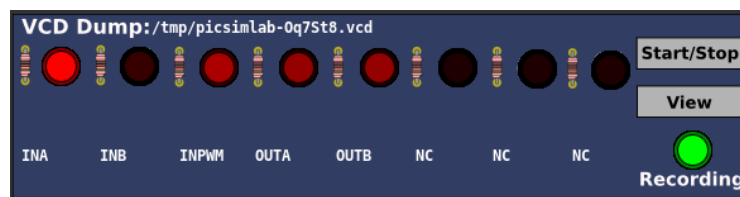
This part is static multiline text box.

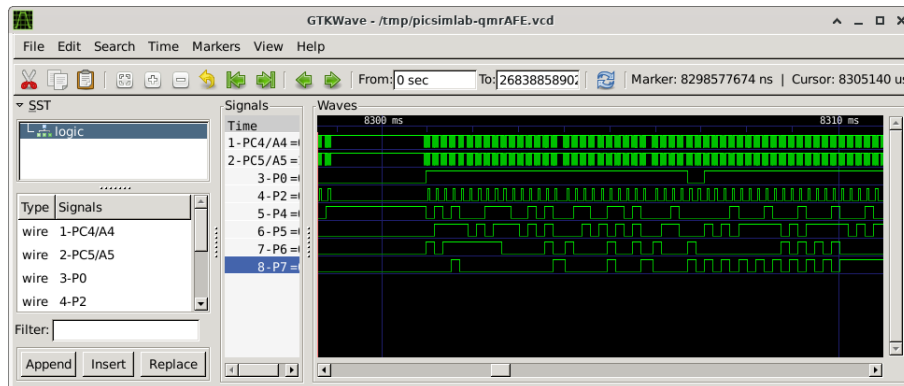


Examples

9.5.5 VCD Dump

This part is a digital value file dump (VCD) recorder. The VCD generated file can be visualized with one external viewer like [gtkwave](#) or [pulseview](#).

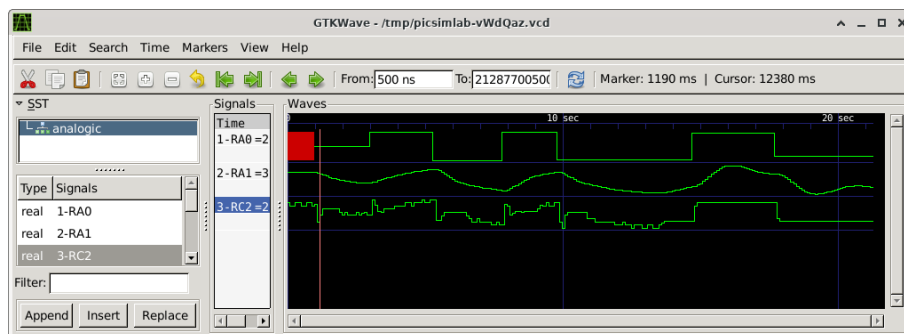
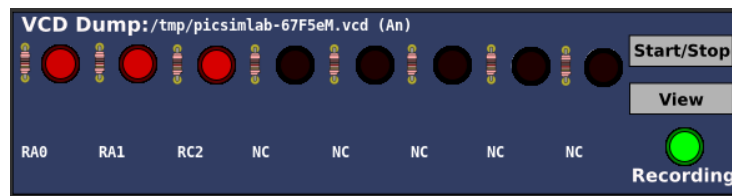




Examples

9.5.6 VCD Dump (Analogic)

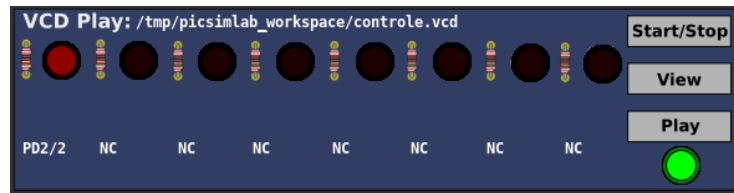
This part is a analog value file dump (VCD) recorder. The VCD generated file can be visualized with one external viewer like [gtkwave](#) or [pulseview](#).



Examples

9.5.7 VCD Play

This part play a VCD file generated from VCD Dump part.



Examples

Chapter 10

Troubleshooting

The simulation in PICSimLab consists of 3 parts:

- The microcontroller program
- Microcontroller simulation (made by [picsim](#) and [simavr](#))
- Simulation of boards and parts

When a problem occurs it is important to detect where it is occurring.

One of the most common problems is the error in the microcontroller program. Before creating an issue, test your code on a real circuit (even partially) to make sure the problem is not there.

Errors in the microcontroller simulation can be detected using code debugging. Any instruction execution or peripheral behavior outside the expected should be reported in the project of simulator used ([picsim](#) or [simavr](#)).

If the problem is not in either of the previous two options, the problem is probably in PICSimLab. A good practice is to send a source code together with a PICSimLab workspace (.pzw file) to open the issue about the problem.

Chapter 11

License

Copyright © 2024 Luis Claudio Gambôa Lopes <lcgamboa@yahoo.com>

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

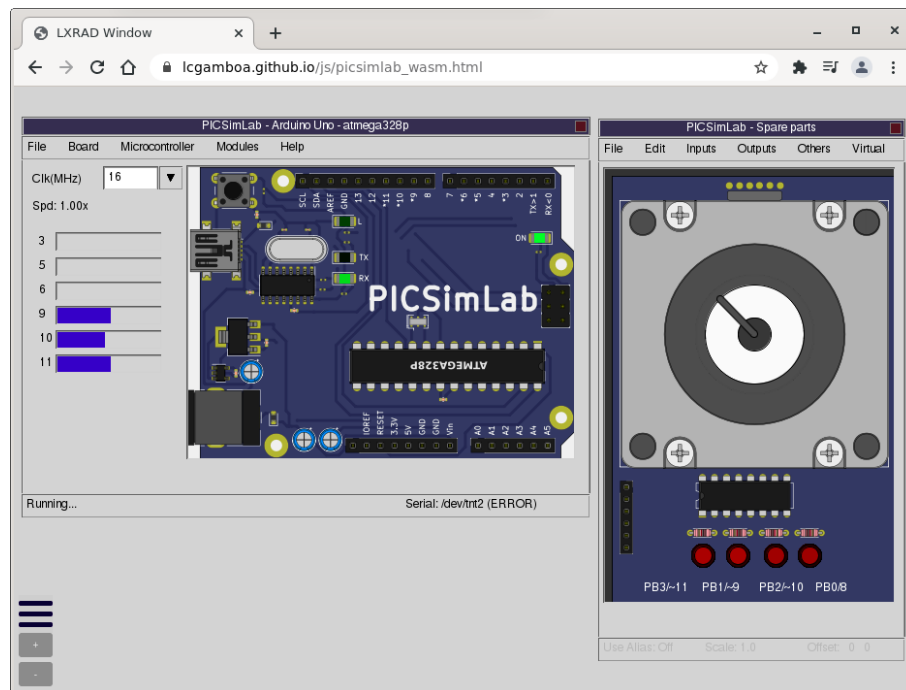
This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA 02111-1307, USA.

Appendix A

Online Simulator

The online version of PICSimLab has the same source code as the desktop version compiled using [Emscripten](#). The online version does not have the Tools menu, support for debugging and serial communication (only for **IO Vterm**).

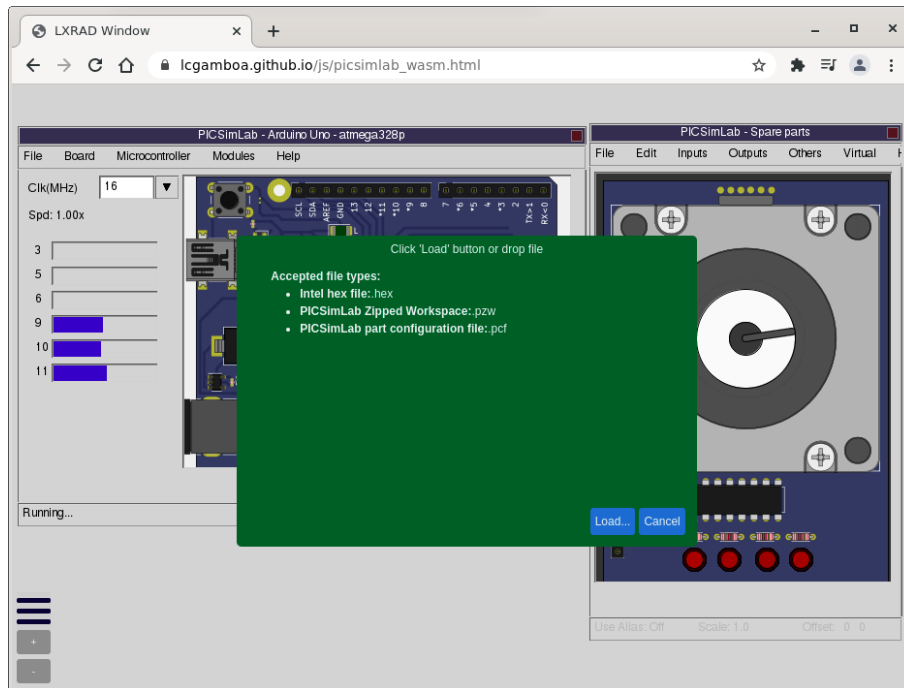


There are three versions generated with Emscripten:

- **WASM with multithread** - Fast speed (50% of desktop version) and worse browser compatibility (currently only tested and work in Chrome and Firefox desktop)

- [WASM](#) - Good speed and good browser compatibility
- [ASM.JS](#) - Slow speed and better browser compatibility

Clicking on the three-bar menu (bottom left) or any loading option in the menus to open the file loading window. Files can be loaded by drag and drop or by the load button.



The simulator can also be accessed from the [examples page](#) for online viewing of most examples (View Online link).

Due to the limitations of the online version, it is advisable to use the desktop version which has more resources and higher simulation speed, especially above 8Mhz clocks.

Appendix B

Use with MPLABX

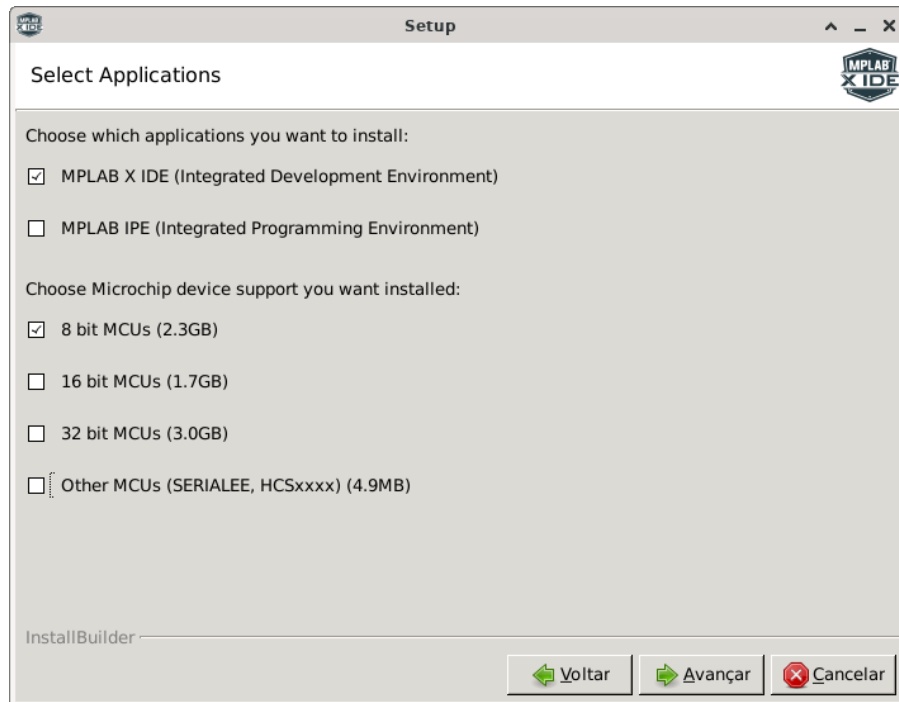
Use with MPLABX to program and Debug

B.1 Installing the Necessary Tools

B.1.1 Install MPLABX IDE and XC8 Compiler

Links for download [MPLABX IDE](#) and [XC8 Compiler](#) installers. Download and install.

For PICSimLab only MPLAB X IDE and 8 bit MCU support needs to be installed.

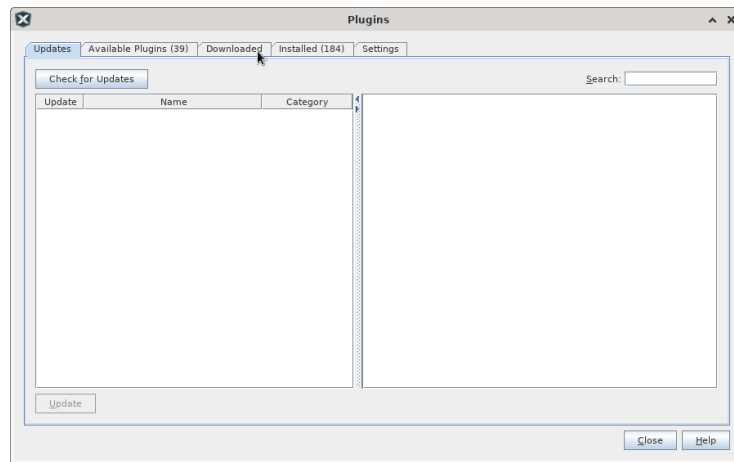
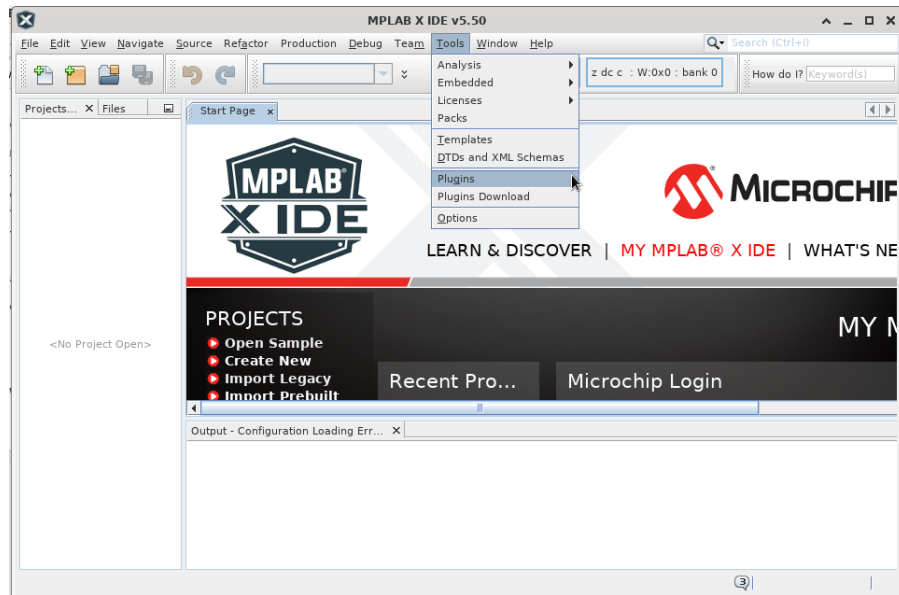


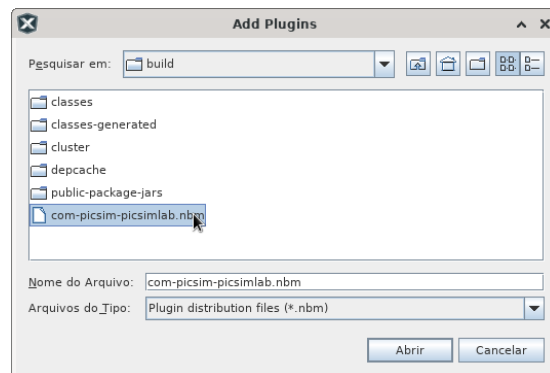
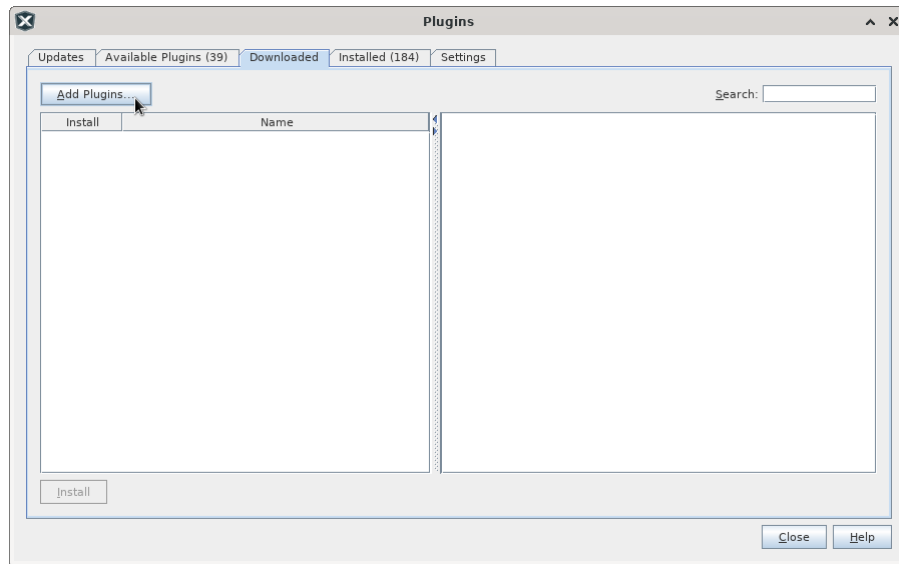
B.1.2 Install PICsimLab

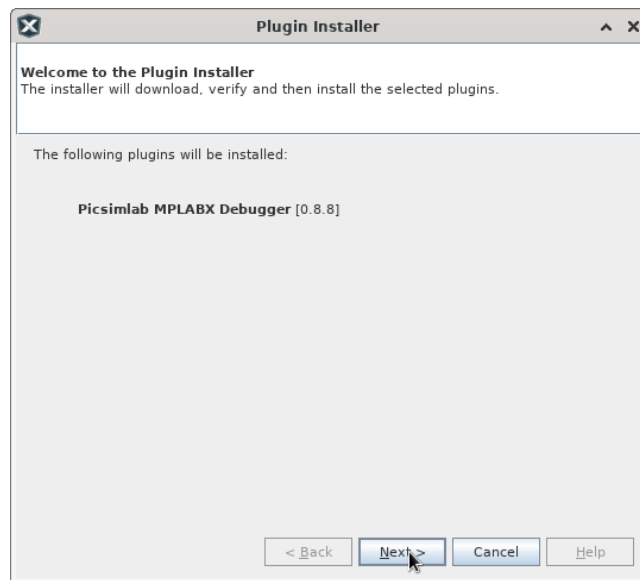
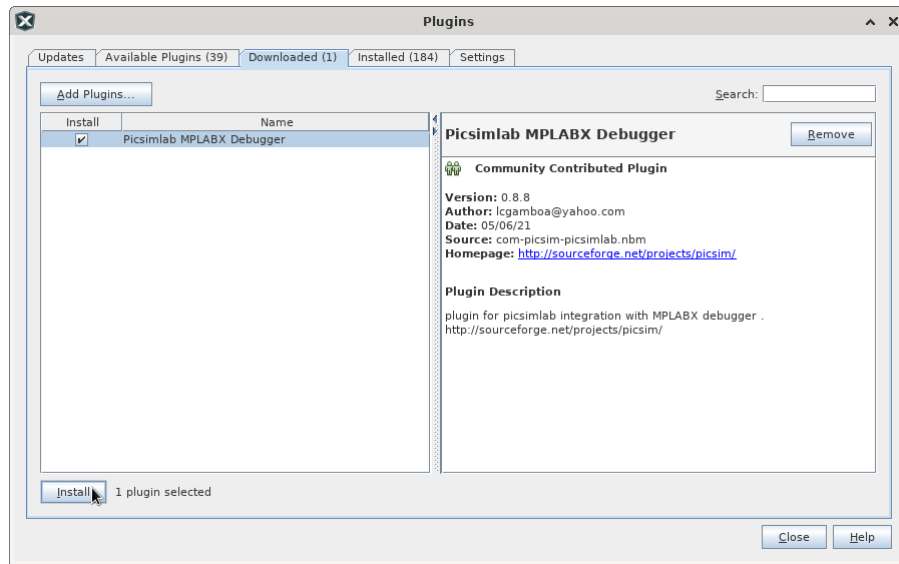
Link for download [PICSimLab](#) installer. Download and install

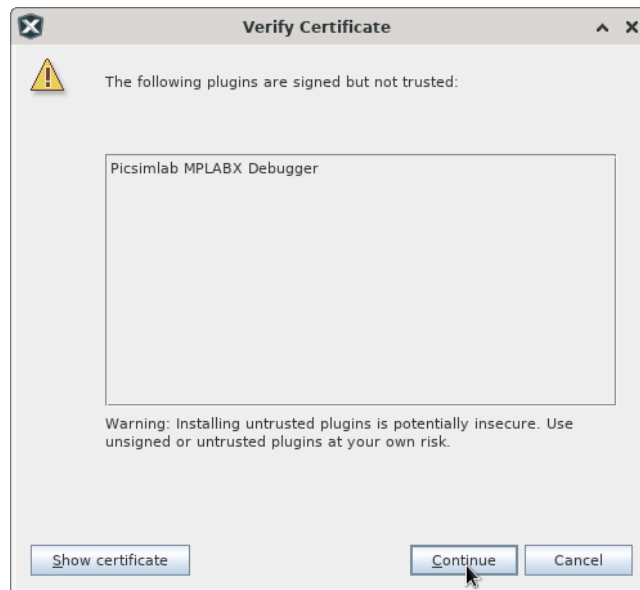
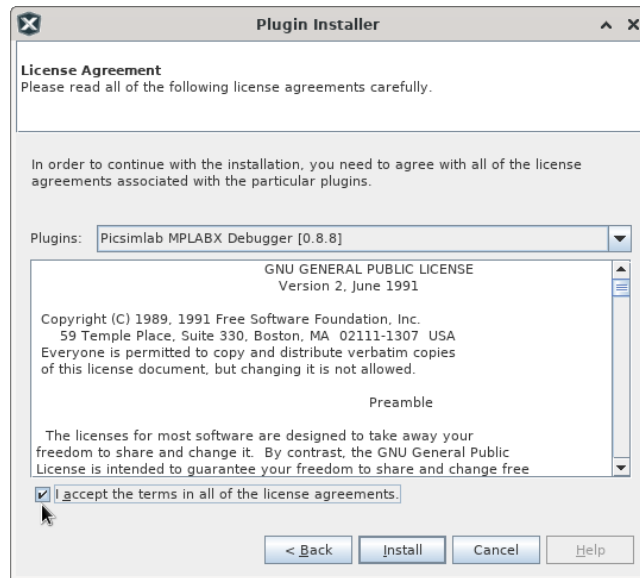
B.1.3 How to Install PICSimLab MPLABX Debugger plugin

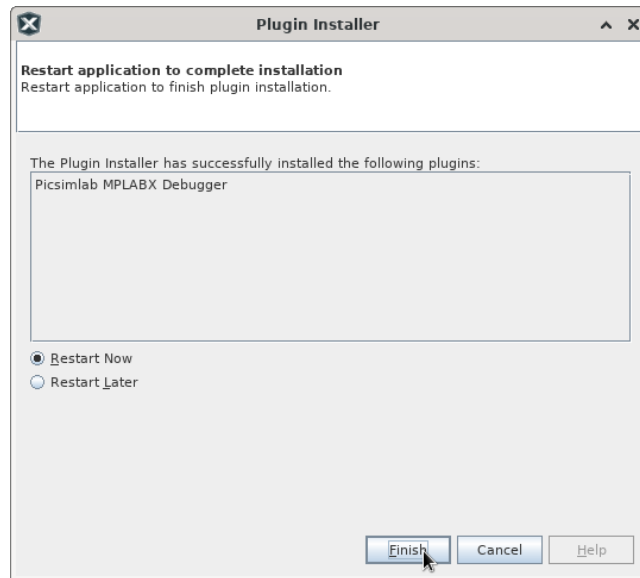
Link for download [PicsimLab MPLABX Debugger plugin \(com-picsim-picsimlab.nbm\)](#)





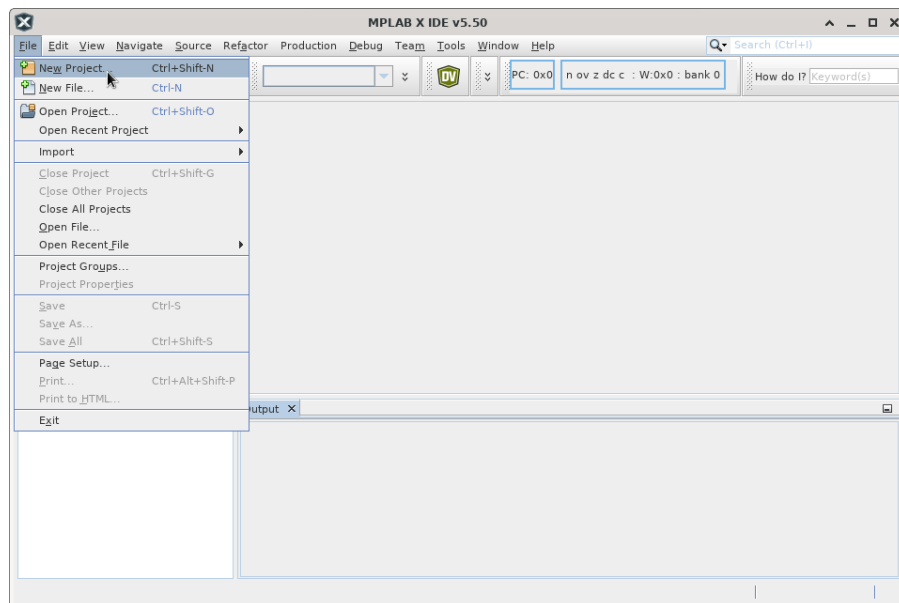


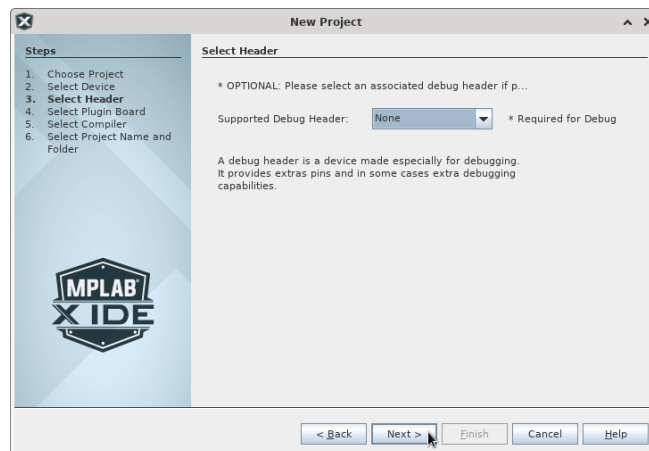
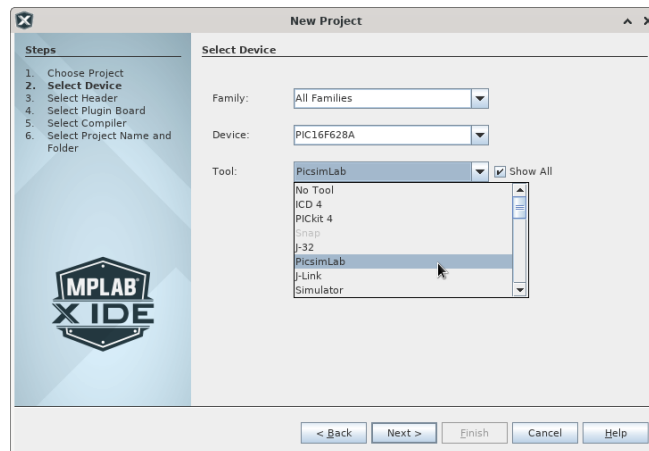
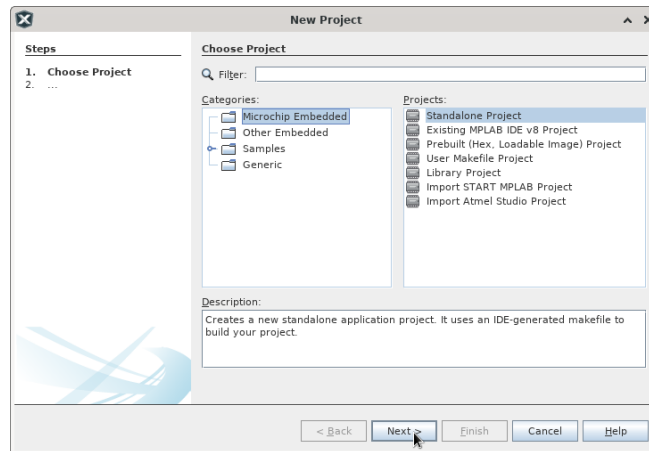


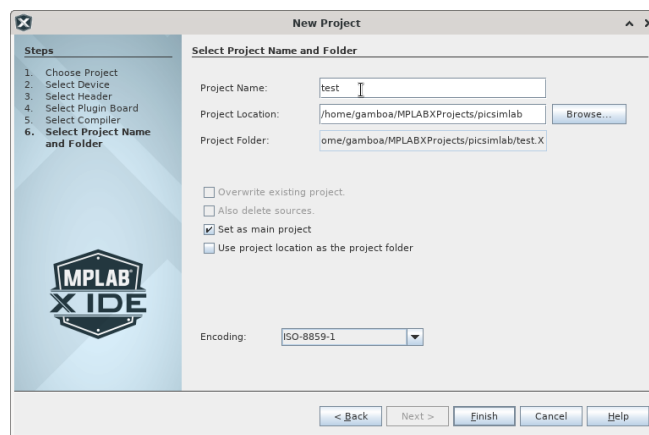
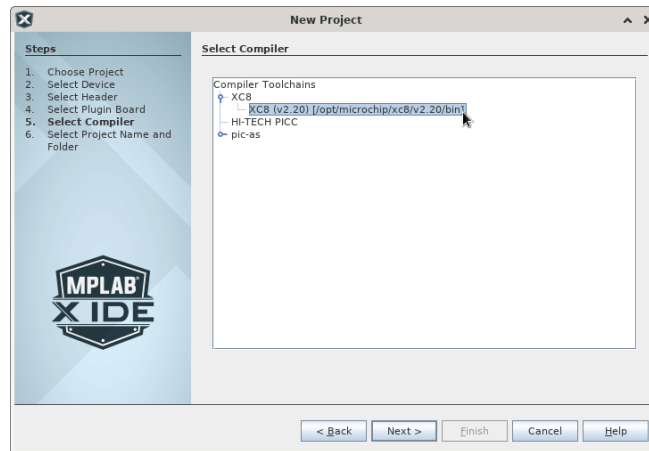


B.2 Configuring a New Project in MPLABX

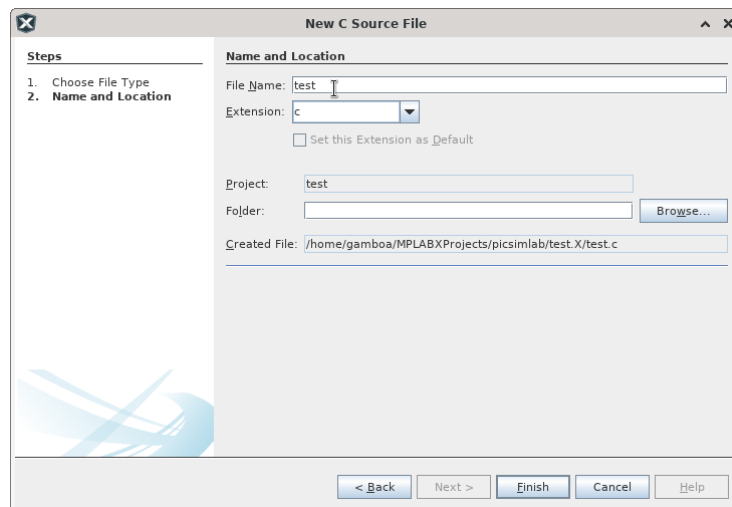
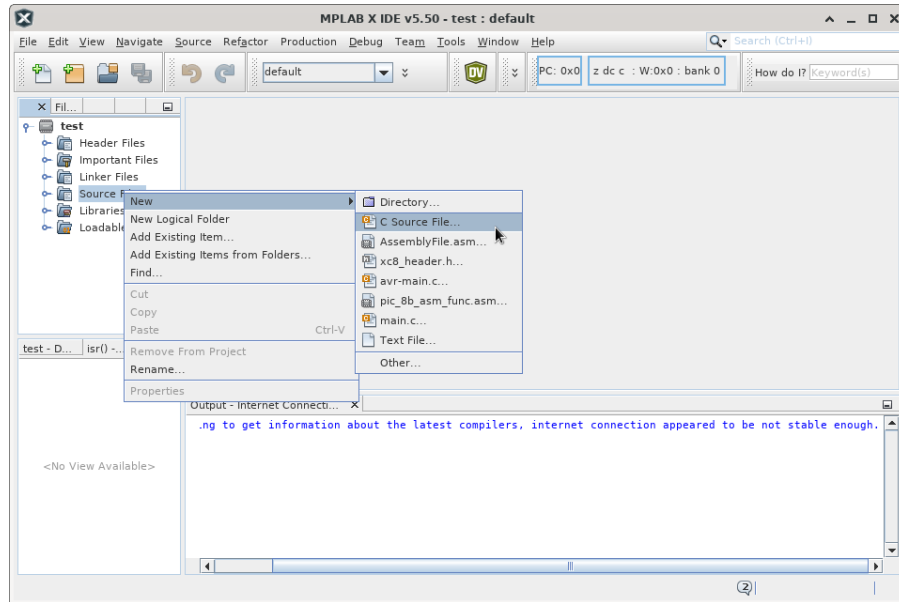
B.2.1 Project Creation



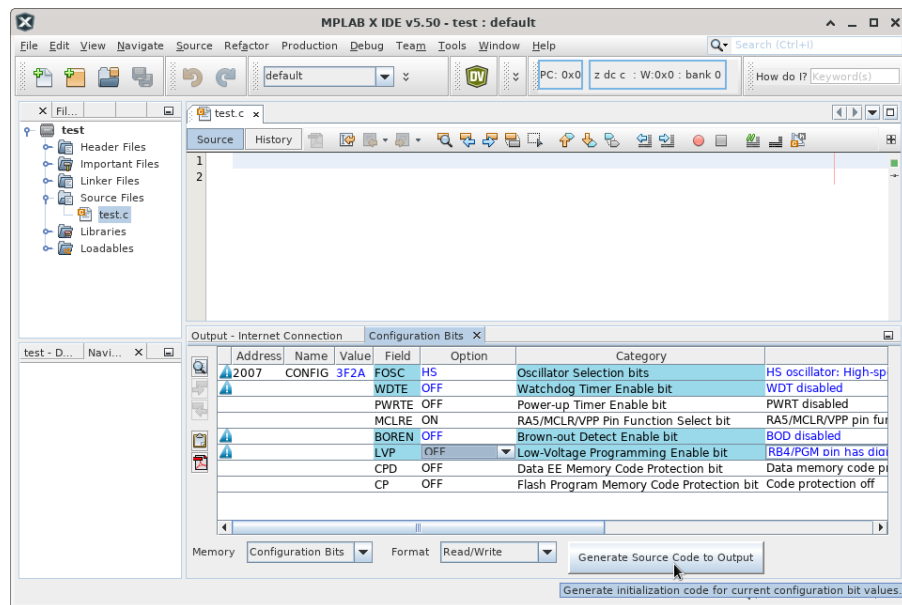
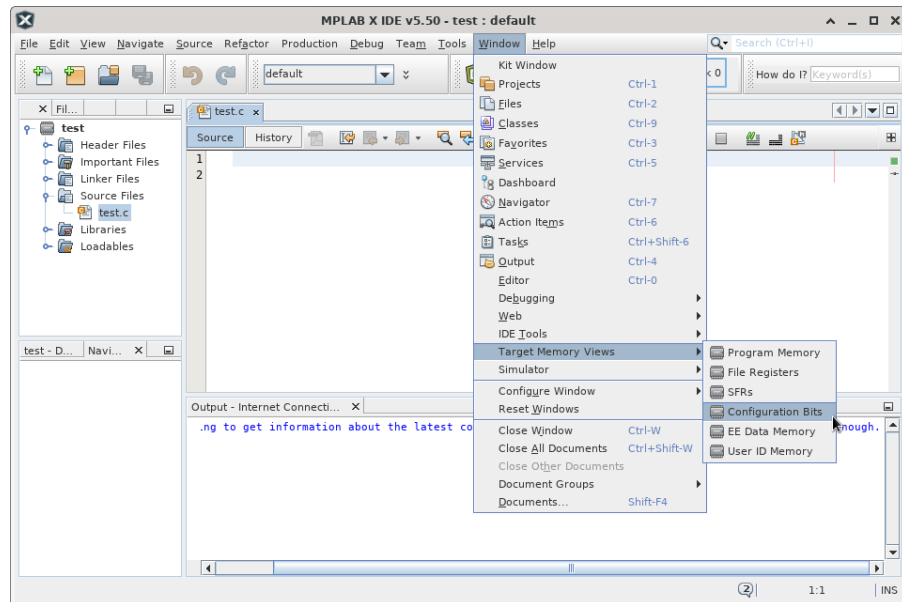


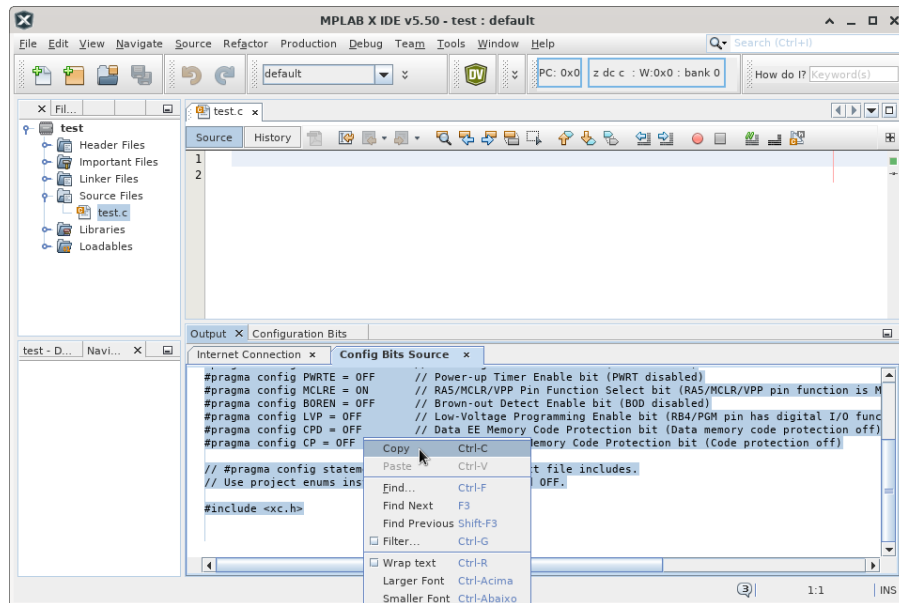


B.2.2 File Creation



B.2.3 PIC Configuration Bits

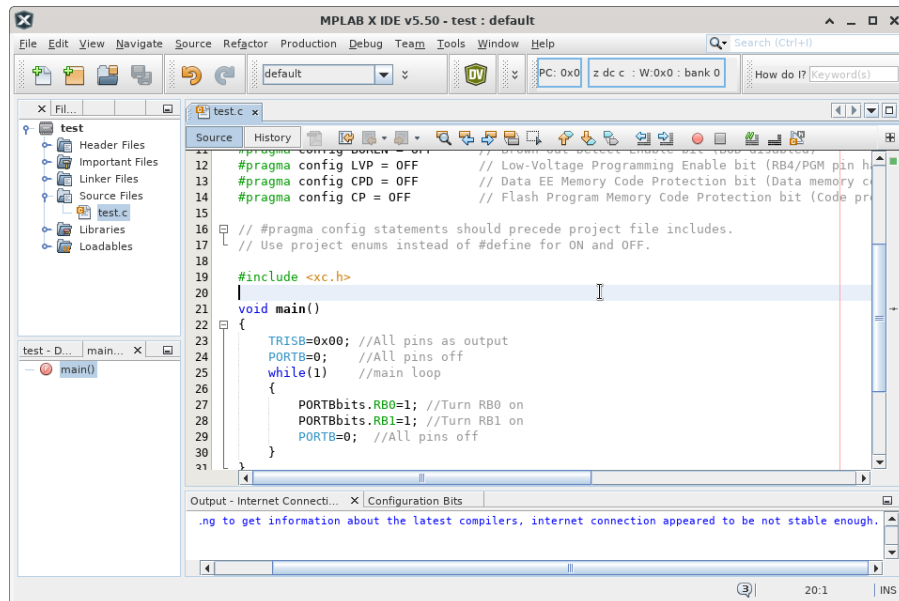




B.2.4 Code Example

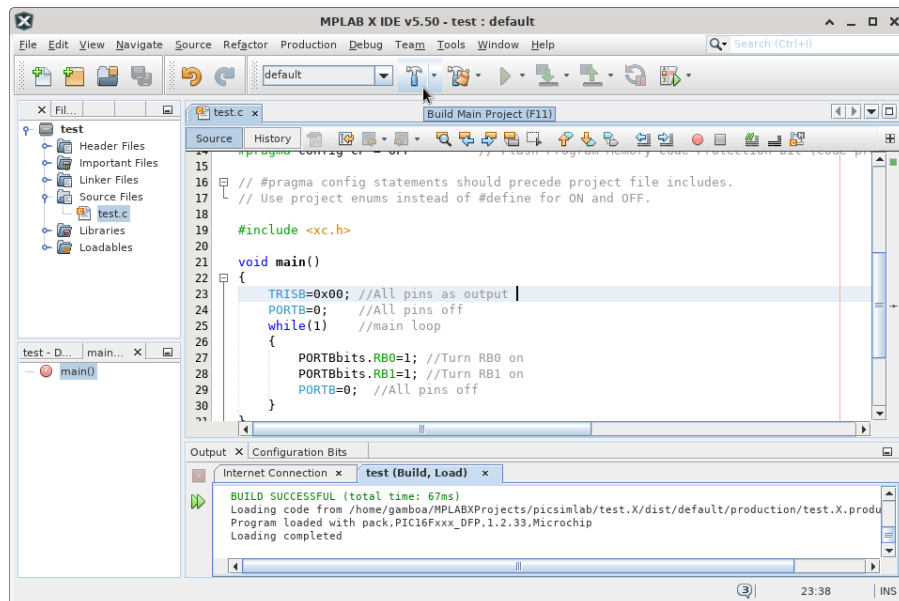
Paste the configuration and this simple code example in test.c:

```
void main()
{
    TRISB=0x00; //All pins as output
    PORTB=0;    //All pins off
    while(1)    //main loop
    {
        PORTBbits.RB0=1; //Turn RB0 on
        PORTBbits.RB1=1; //Turn RB1 on
        PORTB=0;    //All pins off
    }
}
```



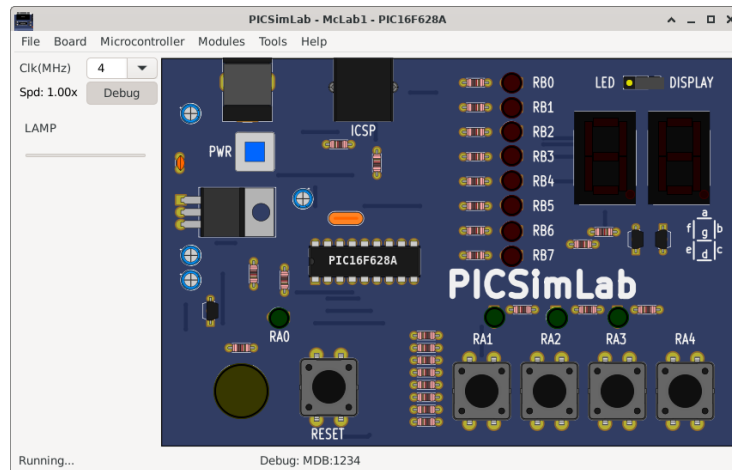
B.2.5 Building the Project

Use the **Build** button and wait for the message “**BUILD SUCCESSFUL**”.



B.3 Program and Debug PICsimLab With MPLABX

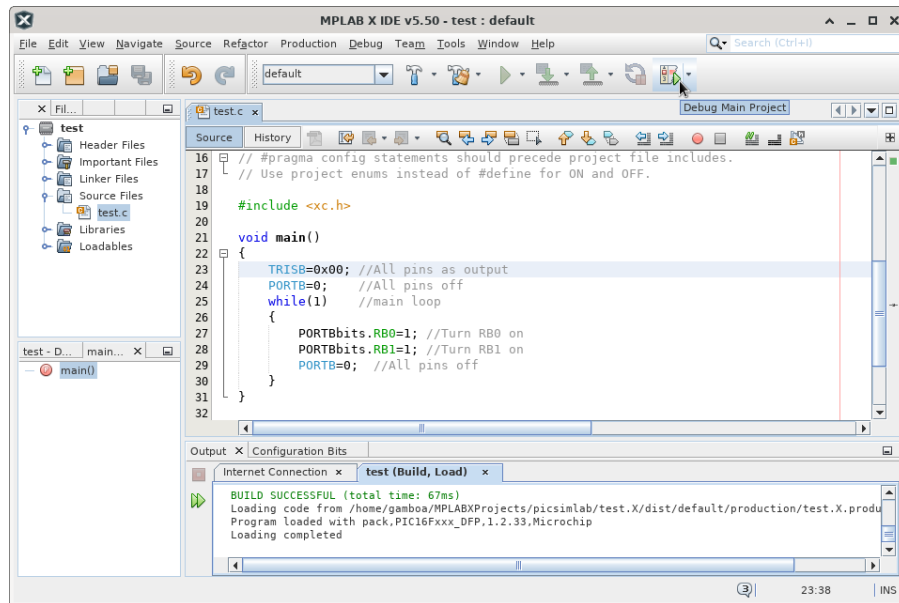
B.3.1 Starting PICsimLab



The plugin connect to Picsimlab through a TCP socket using port 1234, and you have to allow the access in the firewall. Verify in the PICsimLab statusbar the message “MplabxD: Ok”. It’s show debugger server state.

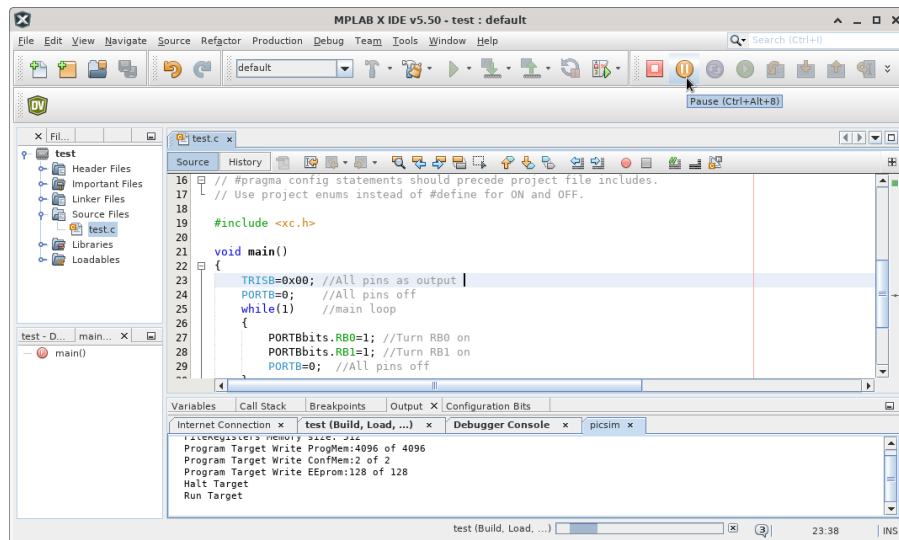
B.3.2 Programming PICsimLab

Use the **Debug** button to programming PICsimLab.



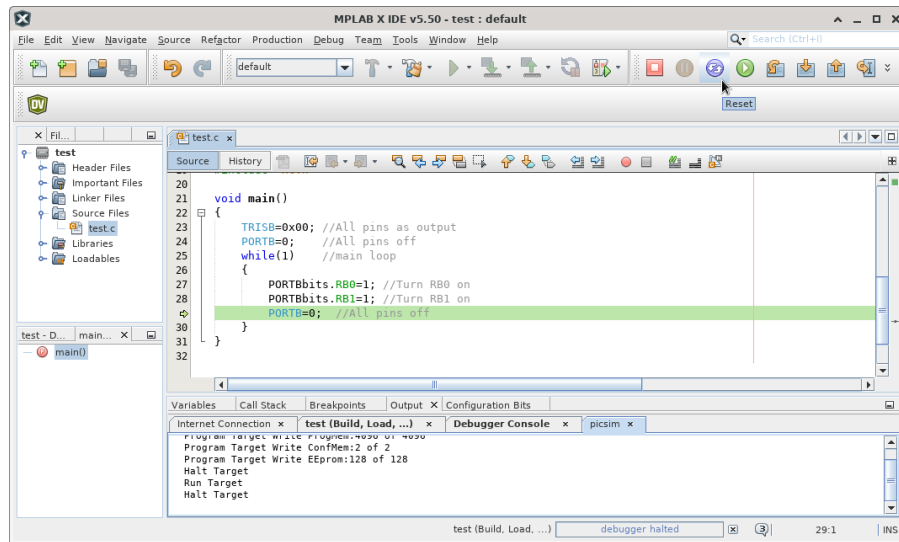
B.3.3 Pausing the Program

Use the **Pause** button to stop the program and inspect the code and memory.



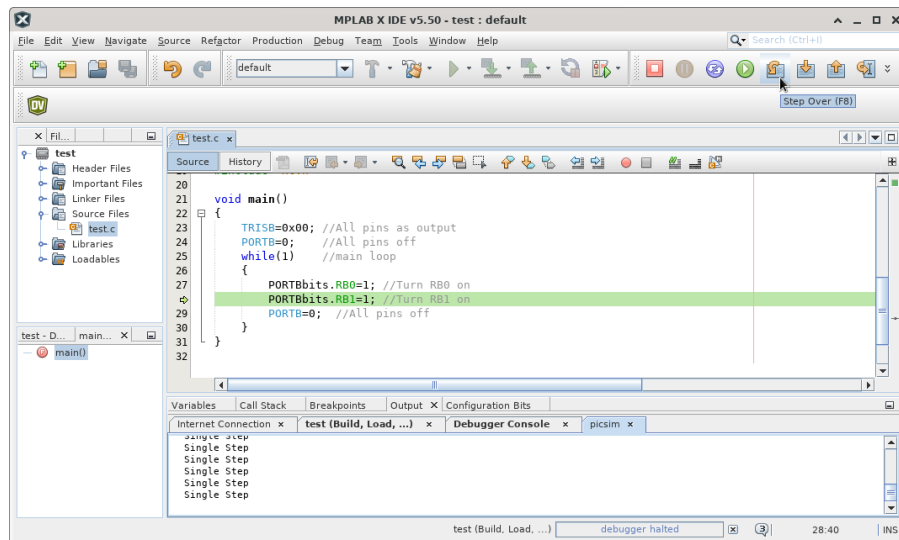
B.3.4 Restarting the Program

Use the **Restart** button to restart the program.

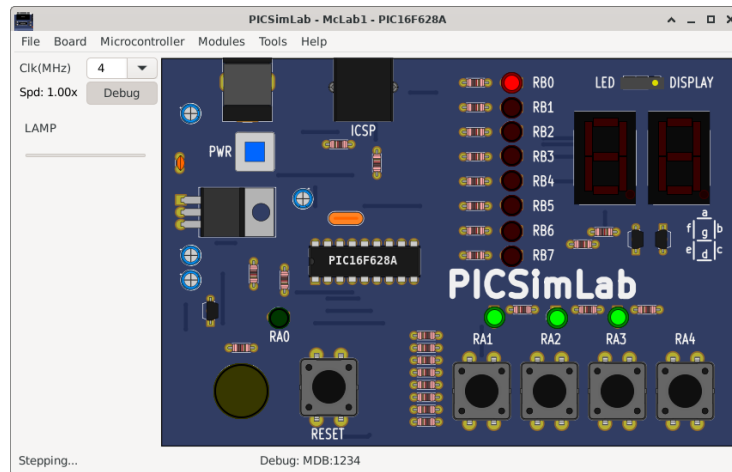


B.3.5 Running Step by Step

Use the **Step** or **Step Over** button to run the program step by step.

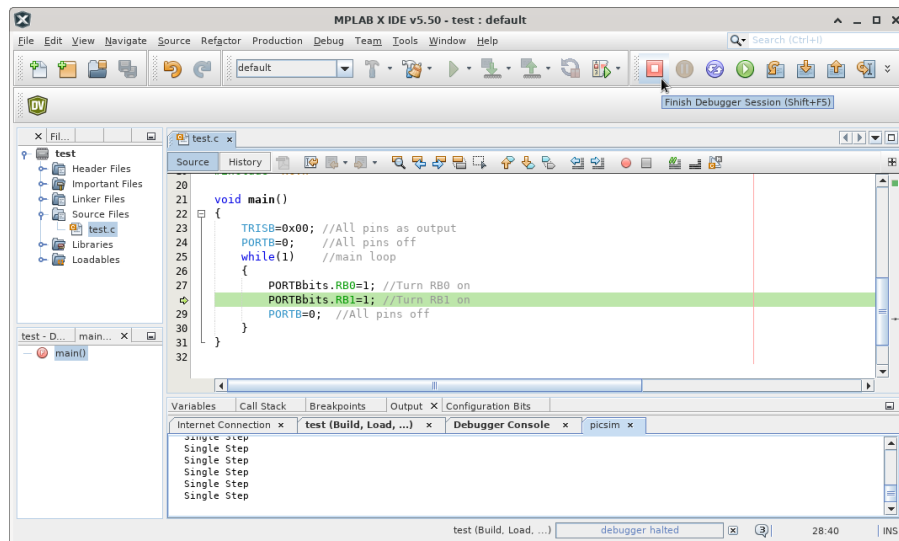


See in the PICsimLab the changes of each step.



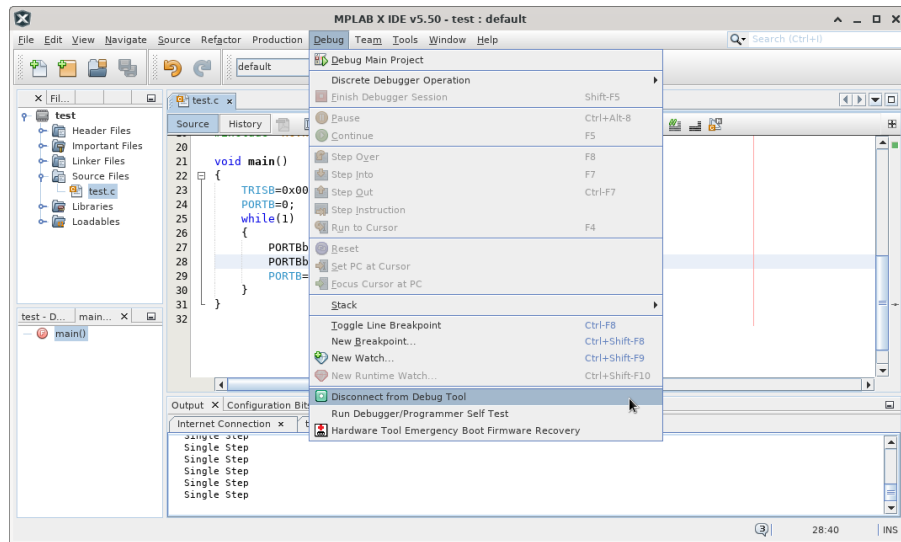
B.3.6 Stopping Debugger

Use the **Stop** button to turn off the MPLABX debugger.



B.3.7 Disconnect Debugger

Use the menu **Debug->Disconnect From Debug Tool** to disconnect the MPLABX debugger. The program continues running in PICsimLab after MPLABX debugger is disconnected.



B.4 This Tutorial in Video

Link for Youtube video version of this tutorial: [How to use MPLABX to program and debug PicsimLab 0.6](#)

Appendix C

Creating New Boards

First get the source code and compile as described in [Install from source](#).

C.1 Creating a New Board

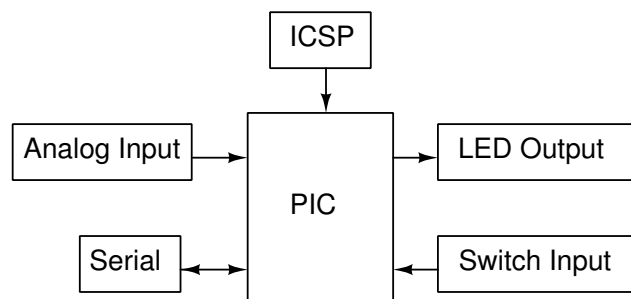
The first step is get the schematic and all information about the board hardware. The second step is the creation of four files in PICSimLab dir (consider replace the 'x' of board_x for a name of your board in your case):

- Board Picture (share/boards/X/board.svg) or (share/boards/X/board.png);
- Board map (share/boards/X/board.map);
- Board header (src/boards/board_x.h);
- Board C++ code (src/boards/board_x.cc);

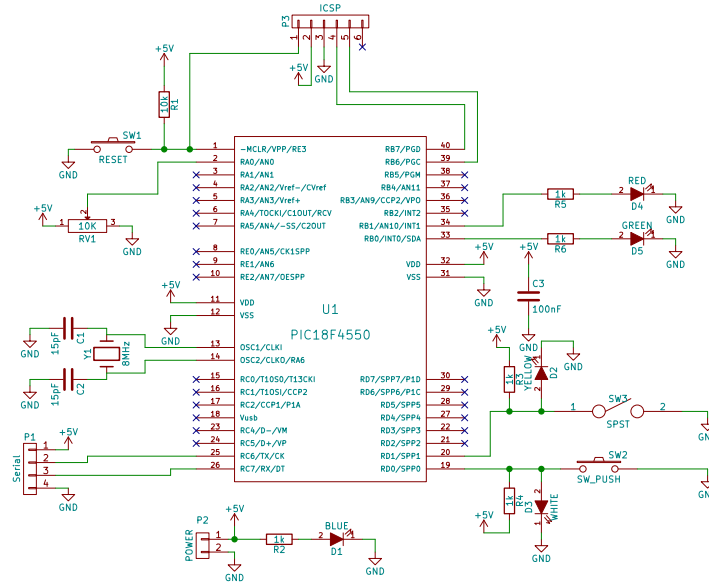
The third and last step is recompiling PICSimLab with new board support.

C.1.1 Board Hardware and Schematic

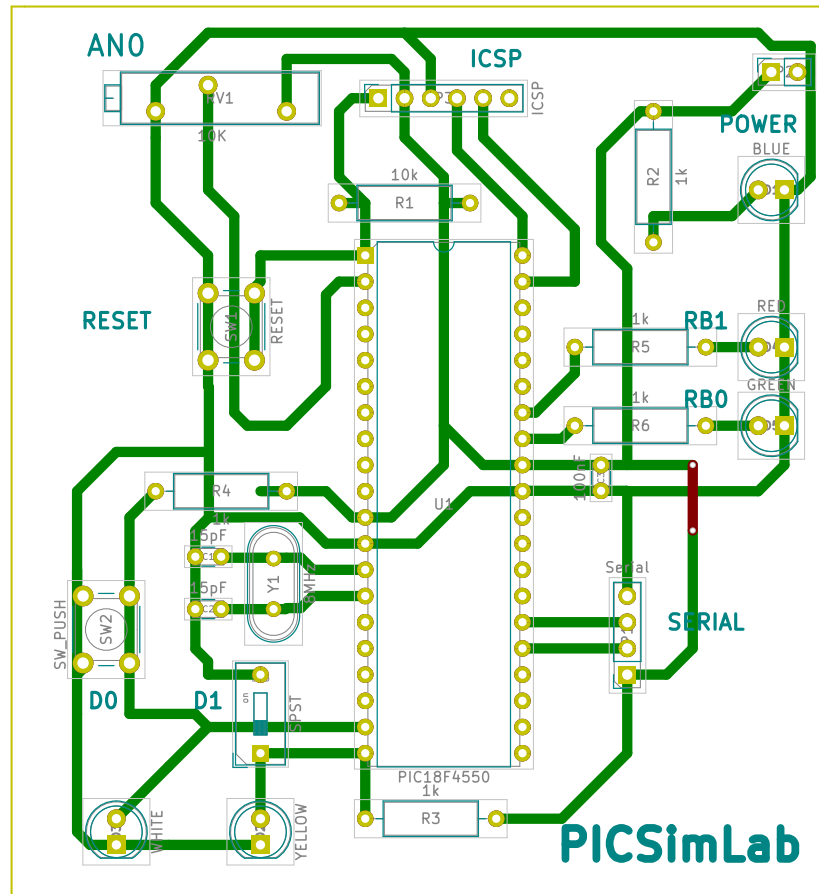
For this tutorial, the board created have the hardware shown in diagram below:



The schematic for the tutorial board made in [Kicad](#).

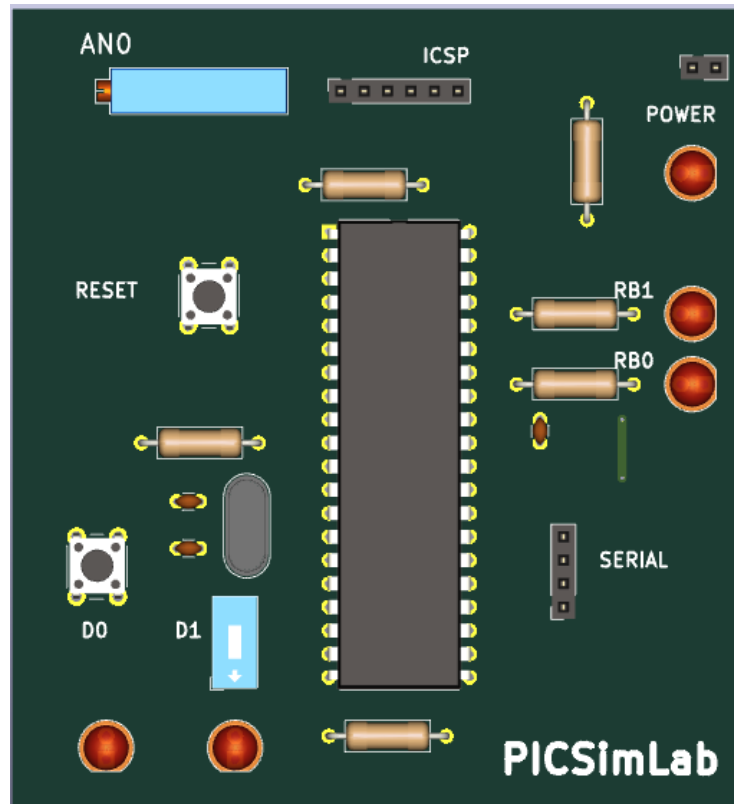


And the PCB layout was made in [Kicad](#) too. The PCB is not necessary if you have a real board.

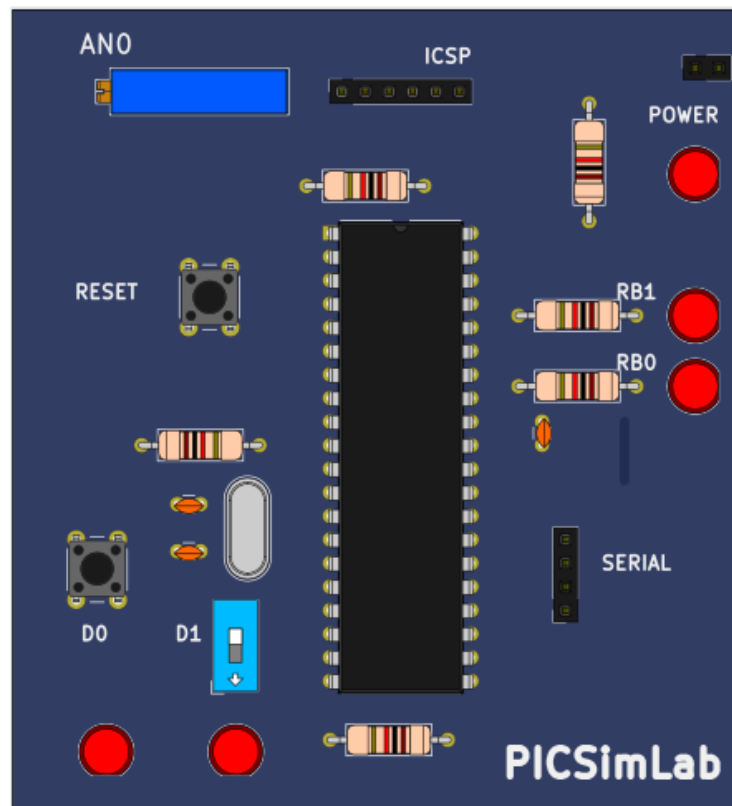


C.1.2 Board Picture

The PNG board picture was taken from [Kicad](#) 3D viewer. The picture image is saved as “share/board/X/board.png”.



It is also possible to use images in SVG format for better viewing quality. [PCBDraw](#) can be used to convert a Kicad PCB project to an SVG image using the [PICSimLab PcbDraw Library](#). The picture image is saved as “share/board/X/board.svg”.



C.1.3 Picture map

The PICSimLab use one picture image map for inputs and outputs.

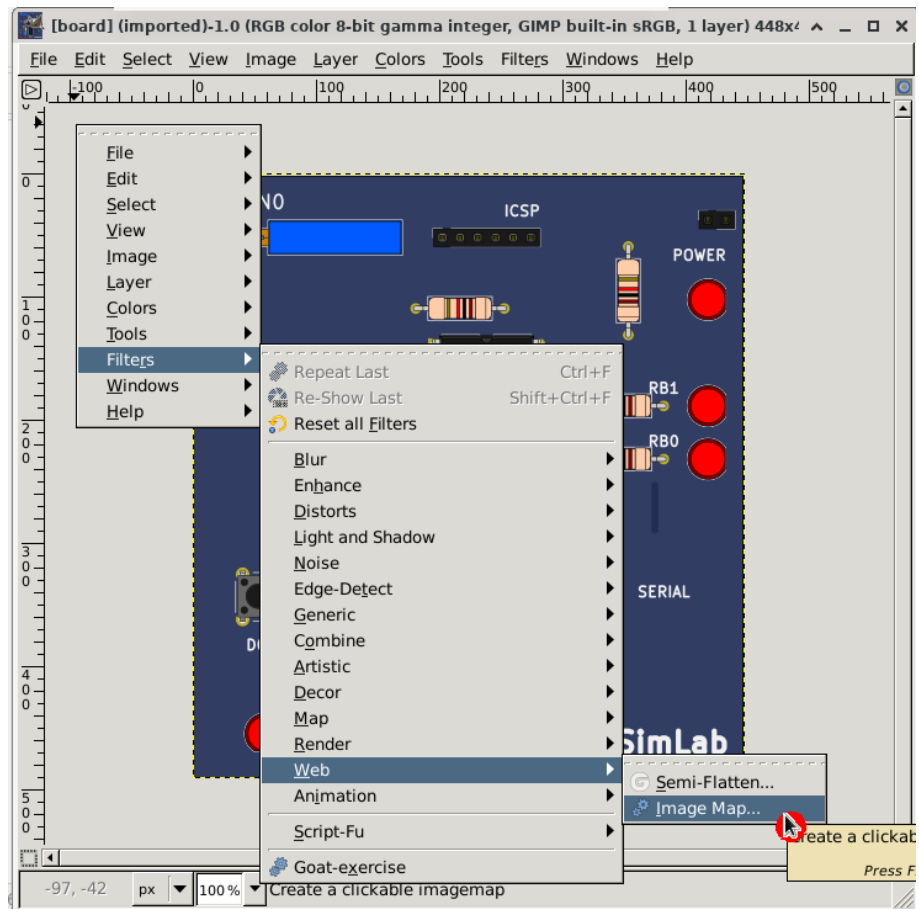
The inputs are the areas in board picture which user can interact (by mouse click) and start with letters “I_”.

The output are the areas in board picture to be redraw according simulator status and start with letters “O_”.

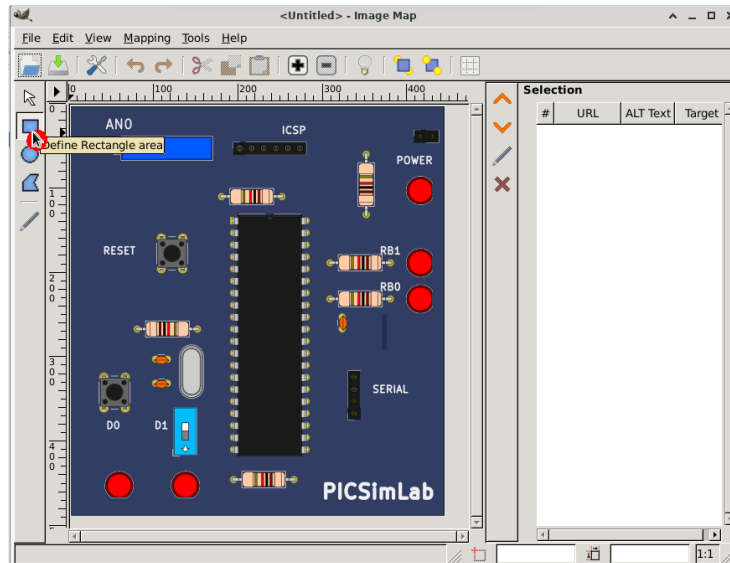
The bidirectional areas in board picture which user can interact and need to be redraw according simulator status are started with letter “B_”.

The picture map used for PICSimLab are normal HTML image-map. They can be made by hand or using any software which can handle image maps. The original PICSimLab maps are made using [Gimp image editor](#).

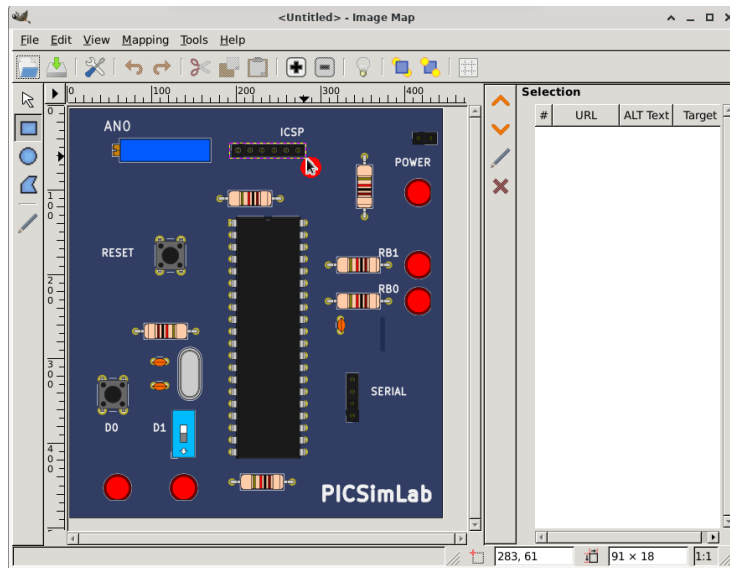
To start, in the GIMP, use the Filters->Web->Image Map to open image map editor window.



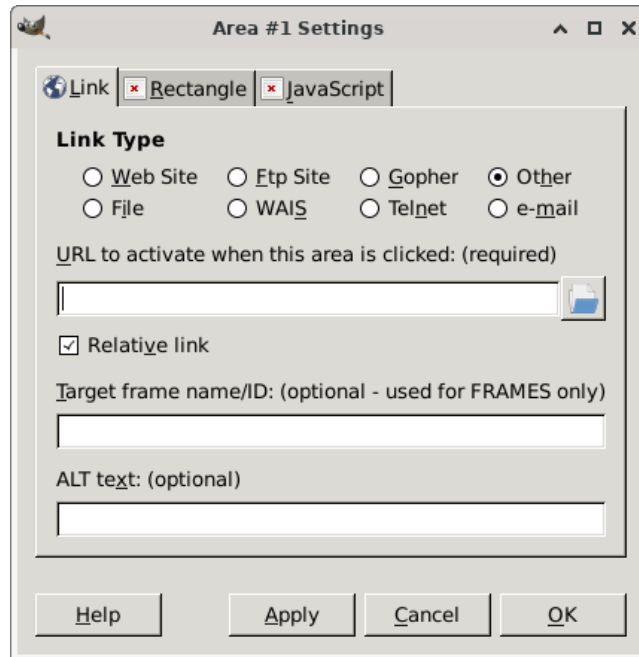
Then select rectangle or circle map on toolbar.



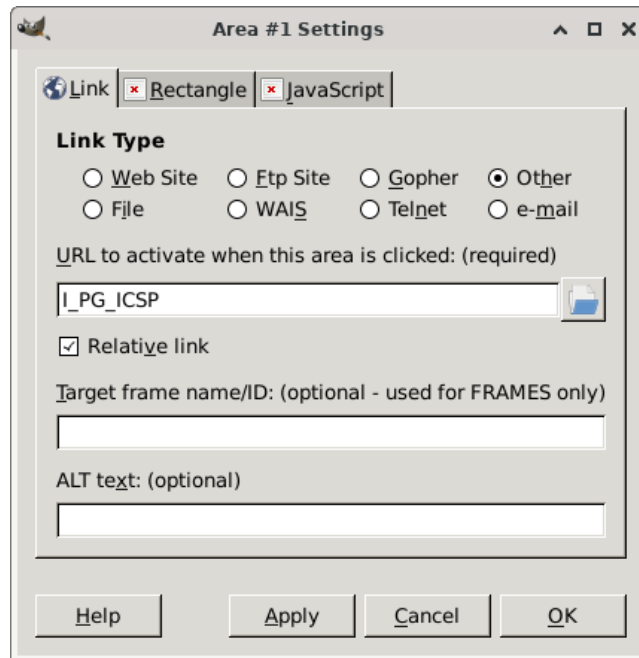
And mark the area in picture.



After area is select, in the settings windows select the link type for “Other”.



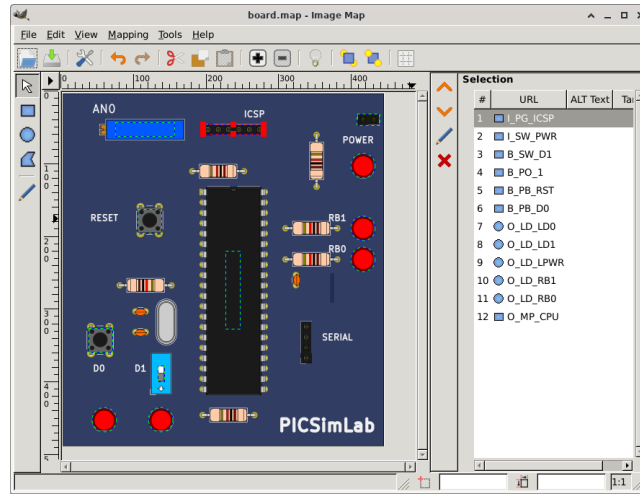
And write the name of area. The name must describe the area function on the board and follow the **Picture Map Reference**.



Board map

For this tutorial board, twelve areas are marked:

- I_PG_ICSP - where user click to load hexfile.
- I_SW_PWR - where user click to turn on/off the board.
- B_SW_D1 - Switch connected in RD1.
- B_PO_1 - Potentiometer connected to RA0.
- B_PB_RST - Button to reset board.
- B_PB_D0 - Button connected in RD0.
- O_LD_LD0 - draw LED connected in push button D0.
- O_LD_LD1 - draw LED connected in switch D1.
- O_LD_LPWR - draw power LED indicator.
- O_LD_RB1 - draw LED connected in RB1.
- O_LD_RB0 - draw LED connected in RB0.
- O_IC_CPU - draw microcontroller name.



Board map generated by Gimp image map editor and saved as “share/boards/X/board.map”.

```

1 
2
3 <map name="map">
4 <!-- $:-Image map file created by GIMP Image Map plug-in -->
5 <!-- $:-GIMP Image Map plug-in by Maurits Rijk -->
6 <!-- $:-Please do not edit lines starting with "$" -->
7 <!-- $VERSION:2.3 -->
8 <!-- $AUTHOR:lcgamboa@yahoo.com -->
9 <area shape="rect" coords="196,45,280,58" href="I_PG_ICSP" />
10 <area shape="rect" coords="409,30,441,46" href="I_SW_PWR" />
11 <area shape="rect" coords="133,379,142,401" href="B_SW_D1" />
12 <area shape="rect" coords="74,42,156,61" href="B_PO_1" />
13 <area shape="rect" coords="105,162,138,195" href="B_PB_RST" />
14 <area shape="rect" coords="37,327,70,360" href="B_PB_D0" />
15 <area shape="circle" coords="59,454,17" href="O_LD_LD0" />
16 <area shape="circle" coords="137,454,17" href="O_LD_LD1" />
17 <area shape="circle" coords="418,102,17" href="O_LD_LPWR" />
18 <area shape="circle" coords="418,189,17" href="O_LD_RB1" />
19 <area shape="circle" coords="418,232,17" href="O_LD_RB0" />
20 <area shape="rect" coords="227,220,247,328" href="O_IC_CPU" />
21 </map>

```

The kicad project files can be download from github [PICSimLab repository](#).

C.1.4 Board code

The header file and c++ code file with comments are listed in the next two subsections. This files control the behavior of board in simulator.

board_x.h

[board_x.h online file.](#)

[board_x.h online doxygen version.](#)

```

1  /* #####
2
3  PICSimLab - Programmable IC Simulator Laboratory
4
5  #####
6
7  Copyright (c) : 2015-2025 Luis Claudio Gambôa Lopes <lcgamboa@yahoo.com>
8
9  This program is free software; you can redistribute it and/or modify
10 it under the terms of the GNU General Public License as published by
11 the Free Software Foundation; either version 2, or (at your option)
12 any later version.
13
14 This program is distributed in the hope that it will be useful,
15 but WITHOUT ANY WARRANTY; without even the implied warranty of
16 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
17 GNU General Public License for more details.
18
19 You should have received a copy of the GNU General Public License
20 along with this program; if not, write to the Free Software
21 Foundation, Inc., 675 Mass Ave, Cambridge, MA 02139, USA.
22
23 For e-mail suggestions : lcgamboa@yahoo.com
24 ##### */
25
26 #ifndef BOARD_x_H
27 #define BOARD_x_H
28
29 #include "../devices/swbounce.h"
30 #include "../sim_backend/bsim_picsim.h"
31
32 #define BOARD_x_Name "X"
33
34 // new board class must be derived from board class defined in board.h
35 class cboard_x : public bsim_picsim {
36 private:
37     unsigned char p_BT1; // first board push button in RD0
38     unsigned char p_BT2; // second board switch in RD1
39
40     // value of potentiometer
41     unsigned char pot1;
42     // flag to control if potentiometer is active
43     unsigned char active;
44
45     // Register controls for remote interface called once on board creation
46     void RegisterRemoteControl(void) override;
47
48     SWBounce_t bounce;
49
50 public:
51     // Constructor called once on board creation
52     cboard_x(void);

```

```

53 // Destructor called once on board destruction
54 ~cboard_x(void);
55 // Return the board name
56 std::string GetName(void) override { return BOARD_x_Name; };
57 // Return the about info of board
58 std::string GetAboutInfo(void) override { return "L.C. Gamboa \n
↳ <lcgamboa@yahoo.com>"; };
59 // Called ever 100ms to draw board
60 void Draw(void) override;
61 void Run_CPU(void) override;
62 // Return a list of board supported microcontrollers
63 std::string GetSupportedDevices(void) override { return
↳ "PIC16F877A,PIC18F4550,PIC18F4620,"; };
64 // Reset board status
65 void Reset(void) override;
66 // Event on the board
67 void EvMouseButtonPress(unsigned int button, unsigned int x, unsigned int y,
↳ unsigned int state) override;
68 // Event on the board
69 void EvMouseButtonRelease(unsigned int button, unsigned int x, unsigned int y,
↳ unsigned int state) override;
70 // Event on the board
71 void EvMouseMove(unsigned int button, unsigned int x, unsigned int y, unsigned
↳ int state) override;
72 // Event on the board
73 void EvKeyPress(unsigned int key, unsigned int mask) override;
74 // Event on the board
75 void EvKeyRelease(unsigned int key, unsigned int mask) override;
76 // Called ever 1s to refresh status
77 void RefreshStatus(void) override;
78 // Called to save board preferences in configuration file
79 void WritePreferences(void) override;
80 // Called whe configuration file load preferences
81 void ReadPreferences(char* name, char* value) override;
82 // return the input ids numbers of names used in input map
83 unsigned short GetInputId(char* name) override;
84 // return the output ids numbers of names used in output map
85 unsigned short GetOutputId(char* name) override;
86 };
87
88 #endif /* BOARD_x_H */

```

board_x.cc[board_x.cc online file.](#)[board_x.cc online doxygen version.](#)

```

1  /* #####
2
3  PICSIMLab - Programmable IC Simulator Laboratory
4
5  #####
6
7  Copyright (c) : 2015-2025 Luis Claudio Gambôa Lopes <lcgamboa@yahoo.com>
8
9  This program is free software; you can redistribute it and/or modify
10 it under the terms of the GNU General Public License as published by
11 the Free Software Foundation; either version 2, or (at your option)
12 any later version.
13
14 This program is distributed in the hope that it will be useful,
15 but WITHOUT ANY WARRANTY; without even the implied warranty of
16 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
17 GNU General Public License for more details.
18
19 You should have received a copy of the GNU General Public License
20 along with this program; if not, write to the Free Software
21 Foundation, Inc., 675 Mass Ave, Cambridge, MA 02139, USA.
22
23 For e-mail suggestions : lcgamboa@yahoo.com
24 ##### */
25
26 // include files
27 #include "board_x.h"
28 #include <math.h>
29 #include "../lib/oscilloscope.h"
30 #include "../lib/picsimlab.h"
31 #include "../lib/spareparts.h"
32
33 /* ids of inputs of input map*/
34 enum {
35     I_POT1, // potentiometer
36     I_ICSP, // ICSP connector
37     I_PWR, // Power button
38     I_RST, // Reset button
39     I_BD0, // RD0 push button
40     I_SD1 // RD1 switch
41 };
42
43 /* ids of outputs of output map*/
44 enum {
45     O_POT1, // potentiometer
46     O_RST, // Reset button
47     O_SD1, // switch position (On/Off)
48     O_LD0, // LED on RD0 push button
49     O_LD1, // LED on RD1 switch
50     O_LPWR, // Power LED
51     O_RB0, // LED on RB0 output
52     O_RB1, // LED on RB1 output

```

```

53     O_BD0,    // RD1 switch
54     O_CPU    // CPU name
55 };
56
57 enum { PWM0 = 1, PWM1 };
58
59 // return the input ids numbers of names used in input map
60
61 unsigned short cboard_x::GetInputId(char* name) {
62     if (strcmp(name, "PG_ICSP") == 0)
63         return I_ICSP;
64     if (strcmp(name, "SW_PWR") == 0)
65         return I_PWR;
66     if (strcmp(name, "PB_RST") == 0)
67         return I_RST;
68     if (strcmp(name, "PB_D0") == 0)
69         return I_BD0;
70     if (strcmp(name, "SW_D1") == 0)
71         return I_SD1;
72     if (strcmp(name, "PO_1") == 0)
73         return I_POT1;
74
75     printf("Error input '%s' don't have a valid id! \n", name);
76     return INVALID_ID;
77 }
78
79 // return the output ids numbers of names used in output map
80
81 unsigned short cboard_x::GetOutputId(char* name) {
82     if (strcmp(name, "SW_D1") == 0)
83         return O_SD1;
84     if (strcmp(name, "LD_LD0") == 0)
85         return O_LD0;
86     if (strcmp(name, "LD_LD1") == 0)
87         return O_LD1;
88     if (strcmp(name, "LD_LPWR") == 0)
89         return O_LPWR;
90     if (strcmp(name, "LD_RB1") == 0)
91         return O_RB1;
92     if (strcmp(name, "LD_RB0") == 0)
93         return O_RB0;
94     if (strcmp(name, "PB_D0") == 0)
95         return O_BD0;
96     if (strcmp(name, "PO_1") == 0)
97         return O_POT1;
98     if (strcmp(name, "PB_RST") == 0)
99         return O_RST;
100    if (strcmp(name, "IC_CPU") == 0)
101        return O_CPU;
102
103    printf("Error output '%s' don't have a valid id! \n", name);
104    return INVALID_ID;
105 }
106
107 // Constructor called once on board creation
108
109 cboard_x::cboard_x(void) {

```

```

110 Proc = "PIC18F4550"; // default microcontroller if none defined in
    ↪ preferences
111 ReadMaps(); // Read input and output board maps
112
113 pot1 = 100;
114
115 active = 0;
116
117 PICSimLab.UpdateGUI(PWM0, GT_GAUGE, GA_ADD, (void*)"*RB0");
118 PICSimLab.UpdateGUI(PWM1, GT_GAUGE, GA_ADD, (void*)"*RB1");
119
120 SWBounce_init(&bounce, 2);
121 }
122
123 // Destructor called once on board destruction
124
125 cboard_x::~cboard_x(void) {
126     PICSimLab.UpdateGUI(PWM0, GT_GAUGE, GA_DEL, NULL);
127     PICSimLab.UpdateGUI(PWM1, GT_GAUGE, GA_DEL, NULL);
128     SWBounce_end(&bounce);
129 }
130
131 // Reset board status
132
133 void cboard_x::Reset(void) {
134     MReset(1);
135
136     p_BT1 = 1; // set push button in default state (high)
137
138     // write button state to pin 19 (RD0)
139     MSetPin(19, p_BT1);
140     // write switch state to pin 20 (RD1)
141     MSetPin(20, p_BT2);
142
143     // verify serial port state and refresh status bar
144     PICSimLab.UpdateStatus(PS_SERIAL, GetUARTStrStatus(0));
145
146     if (use_spare)
147         SpareParts.Reset();
148
149     RegisterRemoteControl();
150 }
151
152 // Register variables to be controled by remote control
153
154 void cboard_x::RegisterRemoteControl(void) {
155     // register inputa
156     input_ids[I_BD0]->status = &p_BT1;
157     input_ids[I_SD1]->status = &p_BT2;
158     input_ids[I_POT1]->status = &pot1;
159
160     // register output to be updated on input change
161     input_ids[I_BD0]->update = &output_ids[O_BD0]->update;
162     input_ids[I_SD1]->update = &output_ids[O_SD1]->update;
163     input_ids[I_POT1]->update = &output_ids[O_POT1]->update;
164
165     // register outputa

```

```

166     output_ids[O_RB0]->status = MGetPinOAVPtr(33);
167     output_ids[O_RB1]->status = MGetPinOAVPtr(34);
168     output_ids[O_LD0]->status = MGetPinOAVPtr(19);
169     output_ids[O_LD1]->status = MGetPinOAVPtr(20);
170 }
171
172 // Called ever 1s to refresh status
173
174 void cboard_x::RefreshStatus(void) {
175     // verify serial port state and refresh status bar
176     PICSimLab.UpdateStatus(PS_SERIAL, GetUARTStrStatus(0));
177 }
178
179 // Called to save board preferences in configuration file
180
181 void cboard_x::WritePreferences(void) {
182     // write selected microcontroller of board_x to preferences
183     PICSimLab.SavePrefs("X_proc", Proc);
184     // write switch state of board_x to preferences
185     PICSimLab.SavePrefs("X_bt2", std::to_string(p_BT2));
186     // write microcontroller clock to preferences
187     PICSimLab.SavePrefs("X_clock", FloatStrFormat("%2.1f", PICSimLab.GetClock()));
188     // write potentiometer position to preferences
189     PICSimLab.SavePrefs("X_pot1", std::to_string(pot1));
190 }
191
192 // Called whe configuration file load preferences
193
194 void cboard_x::ReadPreferences(char* name, char* value) {
195     // read switch state of board_x of preferences
196     if (!strcmp(name, "X_bt2")) {
197         if (value[0] == '0')
198             p_BT2 = 0;
199         else
200             p_BT2 = 1;
201     }
202     // read microcontroller of preferences
203     if (!strcmp(name, "X_proc")) {
204         Proc = value;
205     }
206     // read microcontroller clock
207     if (!strcmp(name, "X_clock")) {
208         PICSimLab.SetClock(atof(value));
209     }
210
211     // read potentiometer position
212     if (!strcmp(name, "X_pot1")) {
213         pot1 = atoi(value);
214     }
215 }
216
217 // Event on the board
218
219 void cboard_x::EvKeyPress(unsigned int key, unsigned int mask) {
220     // if keyboard key 1 is pressed then activate button (state=0)
221     if (key == '1') {
222         p_BT1 = 0;

```

```

223     output_ids[O_BD0]->update = 1;
224 }
225
226 // if keyboard key 2 is pressed then toggle switch state
227 if (key == '2') {
228     p_BT2 ^= 1;
229     output_ids[O_SD1]->update = 1;
230 }
231 }
232
233 // Event on the board
234
235 void cboard_x::EvKeyRelease(unsigned int key, unsigned int mask) {
236     // if keyboard key 1 is pressed then deactivate button (state=1)
237     if (key == '1') {
238         p_BT1 = 1;
239         output_ids[O_BD0]->update = 1;
240     }
241 }
242
243 // Event on the board
244
245 void cboard_x::EvMouseButtonPress(unsigned int button, unsigned int x, unsigned
↵ int y, unsigned int state) {
246     int i;
247
248     // search for the input area which owner the event
249     for (i = 0; i < inputc; i++) {
250         if (((input[i].x1 <= x) && (input[i].x2 >= x)) && ((input[i].y1 <= y) &&
↵ (input[i].y2 >= y))) {
251             switch (input[i].id) {
252                 // if event is over I_ISCP area then load hex file
253                 case I_ICSP:
254                     PICSimLab.OpenLoadHexFileDialog();
255                     ;
256                     break;
257                 // if event is over I_PWR area then toggle board on/off
258                 case I_PWR:
259                     if (PICSimLab.GetMcuPwr()) // if on turn off
260                     {
261                         PICSimLab.SetMcuPwr(0);
262                         Reset();
263                         p_BT1 = 1;
264                     } else // if off turn on
265                     {
266                         PICSimLab.SetMcuPwr(1);
267                         Reset();
268                     }
269                     output_ids[O_LPWR]->update = 1;
270                     break;
271                 // if event is over I_RST area then turn off and reset
272                 case I_RST:
273                     if (PICSimLab.GetMcuPwr() && MReset(-1)) // if powered
274                     {
275                         PICSimLab.SetMcuPwr(0);
276                         PICSimLab.SetMcuRst(1);
277                     }

```

```

278         p_RST = 0;
279         output_ids[O_RST]->update = 1;
280         break;
281         // if event is over I_D0 area then activate button (state=0)
282     case I_BD0:
283         p_BT1 = 0;
284         output_ids[O_BD0]->update = 1;
285         break;
286         // if event is over I_D1 area then toggle switch state
287     case I_SD1:
288         p_BT2 ^= 1;
289         output_ids[O_SD1]->update = 1;
290         break;
291     case I_POT1: {
292         active = 1;
293         pot1 = (x - input[i].x1) * 2.77;
294         if (pot1 > 199)
295             pot1 = 199;
296         output_ids[O_POT1]->update = 1;
297     } break;
298     }
299 }
300 }
301 }
302
303 // Event on the board
304
305 void cboard_x::EvMouseMove(unsigned int button, unsigned int x, unsigned int y,
↪ unsigned int state) {
306     int i;
307
308     for (i = 0; i < inputc; i++) {
309         switch (input[i].id) {
310             case I_POT1:
311                 if (((input[i].x1 <= x) && (input[i].x2 >= x)) && ((input[i].y1 <=
↪ y) && (input[i].y2 >= y))) {
312                     if (active) {
313                         pot1 = (x - input[i].x1) * 2.77;
314                         if (pot1 > 199)
315                             pot1 = 199;
316                         output_ids[O_POT1]->update = 1;
317                     }
318                 } else {
319                     active = 0;
320                 }
321                 break;
322             }
323         }
324     }
325
326 // Event on the board
327
328 void cboard_x::EvMouseButtonRelease(unsigned int button, unsigned int x, unsigned
↪ int y, unsigned int state) {
329     int i;
330
331     // search for the input area which owner the event

```

```

332     for (i = 0; i < inputc; i++) {
333         if (((input[i].x1 <= x) && (input[i].x2 >= x)) && ((input[i].y1 <= y) &&
334             ⇨ (input[i].y2 >= y))) {
335             switch (input[i].id) {
336                 // if event is over I_RST area then turn on
337                 case I_RST:
338                     if (PICSimLab.GetMcuRst()) // if powered
339                     {
340                         PICSimLab.SetMcuPwr(1);
341                         PICSimLab.SetMcuRst(0);
342
343                         if (MReset(-1)) {
344                             Reset();
345                         }
346                         p_RST = 1;
347                         output_ids[O_RST]->update = 1;
348                         break;
349                         // if event is over I_D0 area then deactivate button (state=1)
350                 case I_BD0:
351                     p_BT1 = 1;
352                     output_ids[O_BD0]->update = 1;
353                     break;
354                 case I_POT1: {
355                     active = 0;
356                     output_ids[O_POT1]->update = 1;
357                 } break;
358             }
359         }
360     }
361 }
362
363 // Called ever 100ms to draw board
364 // This is the critical code for simulator running speed
365
366 void cboard_x::Draw(void) {
367     int update = 0; // verify if updated is needed
368     int i;
369     const picpin* pins = MGetPinsValues();
370
371     // board_x draw
372     for (i = 0; i < outputc; i++) // run over all outputs
373     {
374         if (output[i].update) // only if need update
375         {
376             output[i].update = 0;
377
378             if (!update) {
379                 PICSimLab.CanvasCmd({.cmd = CC_INIT, .Init{Scale, Scale, 0}});
380                 PICSimLab.CanvasCmd({.cmd = CC_SETFONTWEIGHT,
381                     ⇨ .SetFontWeight{CC_FONTWEIGHT_BOLD}});
382             }
383             update++; // set to update buffer
384
385             if (!output[i].r) // if output shape is a rectangle
386             {
387                 if (output[i].id == O_SD1) // if output is switch

```

```

387 {
388     // draw a background white rectangle
389     PICSimLab.CanvasCmd({.cmd = CC_SETBGCOLOR, .SetBgColor{255,
390         ↪ 255, 255}});
391     PICSimLab.CanvasCmd({.cmd = CC_RECTANGLE,
392         ↪ .Rectangle{1, output[i].x1, output[i].y1,
393         ↪ output[i].x2 - output[i].x1,
394         ↪ output[i].y2 -
395         ↪ output[i].y1}});
396
397     if (!p_BT2) // draw switch off
398     {
399         // draw a grey rectangle
400         PICSimLab.CanvasCmd({.cmd = CC_SETBGCOLOR, .SetBgColor{70,
401             ↪ 70, 70}});
402         PICSimLab.CanvasCmd(
403             {.cmd = CC_RECTANGLE,
404             ↪ .Rectangle{1, output[i].x1, output[i].y1 +
405             ↪ (int)((output[i].y2 - output[i].y1) * 0.35)),
406             ↪ output[i].x2 - output[i].x1,
407             ↪ ((output[i].y2 - output[i].y1) *
408             ↪ 0.65f)}});
409     } else // draw switch on
410     {
411         // draw a grey rectangle
412         PICSimLab.CanvasCmd({.cmd = CC_SETBGCOLOR, .SetBgColor{70,
413             ↪ 70, 70}});
414         PICSimLab.CanvasCmd({.cmd = CC_RECTANGLE,
415             ↪ .Rectangle{1, output[i].x1,
416             ↪ output[i].y1, output[i].x2 -
417             ↪ output[i].x1,
418             ↪ ((output[i].y2 -
419             ↪ output[i].y1) *
420             ↪ 0.65f)}});
421     }
422 } else if (output[i].id == O_BD0) {
423     PICSimLab.CanvasCmd({.cmd = CC_SETCOLOR, .SetColor{102, 102,
424         ↪ 102}});
425     PICSimLab.CanvasCmd({.cmd = CC_CIRCLE, .Circle{1,
426         ↪ output[i].cx, output[i].cy, 10}});
427     if (p_BT1) {
428         PICSimLab.CanvasCmd({.cmd = CC_SETCOLOR, .SetColor{15, 15,
429             ↪ 15}});
430     } else {
431         PICSimLab.CanvasCmd({.cmd = CC_SETCOLOR, .SetColor{55, 55,
432             ↪ 55}});
433     }
434     PICSimLab.CanvasCmd({.cmd = CC_CIRCLE, .Circle{1,
435         ↪ output[i].cx, output[i].cy, 8}});
436 } else if (output[i].id == O_RST) {
437     PICSimLab.CanvasCmd({.cmd = CC_SETCOLOR, .SetColor{102, 102,
438         ↪ 102}});
439     PICSimLab.CanvasCmd({.cmd = CC_CIRCLE, .Circle{1,
440         ↪ output[i].cx, output[i].cy, 10}});
441
442     if (p_RST) {

```

```

424 PICSimLab.CanvasCmd({.cmd = CC_SETCOLOR, .SetColor{15, 15,
425     ↳ 15}});
426 } else {
427     PICSimLab.CanvasCmd({.cmd = CC_SETCOLOR, .SetColor{55, 55,
428     ↳ 55}});
429 }
430 PICSimLab.CanvasCmd({.cmd = CC_CIRCLE, .Circle{1,
431     ↳ output[i].cx, output[i].cy, 8}});
432 } else if (output[i].id == O_POT1) {
433     PICSimLab.CanvasCmd({.cmd = CC_SETCOLOR, .SetColor{0, 50,
434     ↳ 215}});
435     PICSimLab.CanvasCmd({.cmd = CC_RECTANGLE,
436     ↳ .Rectangle{1, output[i].x1, output[i].y1,
437     ↳ output[i].x2 - output[i].x1,
438     ↳ output[i].y2 -
439     ↳ output[i].y1}});
440     PICSimLab.CanvasCmd({.cmd = CC_SETCOLOR, .SetColor{250, 250,
441     ↳ 250}});
442     PICSimLab.CanvasCmd(
443     ↳ {.cmd = CC_RECTANGLE, .Rectangle{1, output[i].x1 + pot1 /
444     ↳ 2.77f, output[i].y1 + 2, 10, 15}});
445 } else if (output[i].id == O_CPU) {
446     PICSimLab.CanvasCmd({.cmd = CC_SETFONTSIZE,
447     ↳ .SetFontSize{10}});
448     float x, y;
449     int w, h;
450     PICSimLab.CanvasCmd({.cmd = CC_SETCOLOR, .SetColor{26, 26,
451     ↳ 26}});
452     PICSimLab.CanvasCmd({.cmd = CC_RECTANGLE,
453     ↳ .Rectangle{1, output[i].x1, output[i].y1,
454     ↳ output[i].x2 - output[i].x1,
455     ↳ output[i].y2 -
456     ↳ output[i].y1}});
457     PICSimLab.CanvasCmd({.cmd = CC_SETCOLOR, .SetColor{230, 230,
458     ↳ 230}});
459     w = output[i].x2 - output[i].x1;
460     h = output[i].y2 - output[i].y2;
461     x = output[i].x1 + (w / 2) + 7;
462     y = output[i].y1 + (h / 2) + (Proc.length());
463     PICSimLab.CanvasCmd({.cmd = CC_ROTATEDTEXT,
464     ↳ .RotatedText{Proc.c_str(), x, y, 270}});
465 }
466 } else // if output shape is a circle
467 {
468     PICSimLab.CanvasCmd({.cmd = CC_SETFGCOLOR, .SetFgColor{0, 0, 0}});
469     ↳ // black
470
471     switch (output[i].id) // search for color of output
472     {
473         case O_LD0: // White using pin 19 mean value (RD0)
474             PICSimLab.CanvasCmd({.cmd = CC_SETBGCOLOR,
475             ↳ .SetBgColor{(unsigned
476             ↳ int)pins[18].oavalue, (unsigned
477             ↳ int)pins[18].oavalue,
478             ↳ (unsigned
479             ↳ int)pins[18].oavalue}},

```

```

463         break;
464         case O_LD1: // Yellow using pin 20 mean value (RD1)
465             PICSimLab.CanvasCmd(
466                 {.cmd = CC_SETBGCOLOR,
467                  .SetBgColor{(unsigned int)pins[19].oavalue, (unsigned
468                      ↪ int)pins[19].oavalue, 0}});
469             break;
470         case O_LPWR: // Blue using mcupwr value
471             PICSimLab.CanvasCmd({.cmd = CC_SETBGCOLOR,
472                                  .SetBgColor{0, 0, (unsigned int)(200
473                  ↪ * PICSimLab.GetMcuPwr() + 55)}});
474             break;
475         case O_RB0: // Green using pin 33 mean value (RB0)
476             PICSimLab.CanvasCmd({.cmd = CC_SETBGCOLOR, .SetBgColor{0,
477                  ↪ (unsigned int)pins[32].oavalue, 0}});
478             break;
479         case O_RB1: // Red using pin 34 mean value (RB1)
480             PICSimLab.CanvasCmd({.cmd = CC_SETBGCOLOR,
481                                  ↪ .SetBgColor{(unsigned int)pins[33].oavalue, 0, 0}});
482             break;
483     }
484     DrawLED(PICSimLab.CanvasCmd, &output[i]);
485 }
486 // end draw
487 if (update) {
488     PICSimLab.CanvasCmd({.cmd = CC_END});
489 }
490
491 int value = (pins[33].oavalue - 55) / 2; // RB0 mean value
492 PICSimLab.UpdateGUI(PWM0, GT_GAUGE, GA_SET, (void*)&value);
493 value = (pins[32].oavalue - 55) / 2; // RB1 mean value
494 PICSimLab.UpdateGUI(PWM1, GT_GAUGE, GA_SET, (void*)&value);
495 }
496
497 void cboard_x::Run_CPU(void) {
498     int i;
499     int j;
500     unsigned char pi;
501     const picpin* pins;
502     unsigned int alm[40];
503     int bret;
504
505     const int JUMPSTEPS = PICSimLab.GetJUMPSTEPS(); // number of steps skipped
506     const long int NSTEP = PICSimLab.GetNSTEP(); // number of steps in 100ms
507     const float RNSTEP = 200.0 * MGetPinCount() / NSTEP;
508
509     // reset pins mean value
510     memset(alm, 0, 40 * sizeof(unsigned int));
511
512     // read pins to a local variable to speed up
513     pins = MGetPinsValues();
514
515     // Spare parts window pre process

```

```

516     if (use_spare)
517         SpareParts.PreProcess();
518
519     SWBounce_prepare(&bounce, PICSimLab.GetBoard()->MGetInstClockFreq());
520
521     unsigned char p_BT1_ = p_BT1;
522     unsigned char p_BT2_ = p_BT2;
523
524     if ((pins[19 - 1].dir == PD_IN) && (pins[19 - 1].value != p_BT1_)) {
525         SWBounce_bounce(&bounce, 0);
526     }
527     if ((pins[20 - 1].dir == PD_IN) && (pins[20 - 1].value != p_BT2_)) {
528         SWBounce_bounce(&bounce, 1);
529     }
530
531     j = JUMPSTEPS; // step counter
532     pi = 0;
533     if (PICSimLab.GetMcuPwr()) // if powered
534         for (i = 0; i < NSTEP; i++) // repeat for number of steps in 100ms
535             {
536                 if (j >= JUMPSTEPS) // if number of step is bigger than step to skip
537                     {
538                         MSetPin(MGetResetPin(), p_RST);
539                         if (!bounce.do_bounce) {
540                             MSetPin(19, p_BT1_); // Set pin 19 (RD0) with button state
541                             MSetPin(20, p_BT2_); // Set pin 20 (RD1) with switch state
542                         }
543                     }
544
545                 if (bounce.do_bounce) {
546                     bret = SWBounce_process(&bounce);
547                     if (bret) {
548                         if (bounce.bounce[0]) {
549                             if (bret == 1) {
550                                 MSetPin(19, !pins[19 - 1].value);
551                             } else {
552                                 MSetPin(19, p_BT1_);
553                             }
554                         }
555                         if (bounce.bounce[1]) {
556                             if (bret == 1) {
557                                 MSetPin(20, !pins[20 - 1].value);
558                             } else {
559                                 MSetPin(20, p_BT2_);
560                             }
561                         }
562                     }
563                 }
564
565                 // verify if a breakpoint is reached if not run one instruction
566                 if (!mplabxd_testbp())
567                     MStep();
568                 ioupdated = MGetIOUpdated();
569                 InstCounterInc();
570                 // Oscilloscope window process
571                 if (use_oscscope)
572                     Oscilloscope.SetSample();

```

```

573 // Spare parts window process
574 if (use_spare)
575     SpareParts.Process();
576
577 // increment mean value counter if pin is high
578 alm[pi] += pins[pi].value;
579 pi++;
580 if (pi == MGetPinCount())
581     pi = 0;
582
583 if (j >= JUMPSTEPS) // if number of step is bigger than steps to skip
584 {
585     // set analog pin 2 (AN0) with value from scroll
586     MSetAPin(2, (5.0 * pot1 / 199));
587
588     j = -1; // reset counter
589 }
590 j++; // counter increment
591 MClearIOUpdated();
592 }
593
594 // calculate mean value
595 for (pi = 0; pi < MGetPinCount(); pi++) {
596     MSetPinOAV(pi + 1, (alm[pi] * RNSTEP) + 55);
597 }
598
599 // Spare parts window pre post process
600 if (use_spare)
601     SpareParts.PostProcess();
602
603 // verify if LEDS need update
604 if (output_ids[O_LD0]->value != pins[18].oavalue) {
605     output_ids[O_LD0]->value = pins[18].oavalue;
606     output_ids[O_LD0]->update = 1;
607 }
608 if (output_ids[O_LD1]->value != pins[19].oavalue) {
609     output_ids[O_LD1]->value = pins[19].oavalue;
610     output_ids[O_LD1]->update = 1;
611 }
612 if (output_ids[O_RB0]->value != pins[32].oavalue) {
613     output_ids[O_RB0]->value = pins[32].oavalue;
614     output_ids[O_RB0]->update = 1;
615 }
616 if (output_ids[O_RB1]->value != pins[33].oavalue) {
617     output_ids[O_RB1]->value = pins[33].oavalue;
618     output_ids[O_RB1]->update = 1;
619 }
620 }
621
622 // Register the board in PICSimLab
623 board_init(BOARD_x_Name, cboard_x);

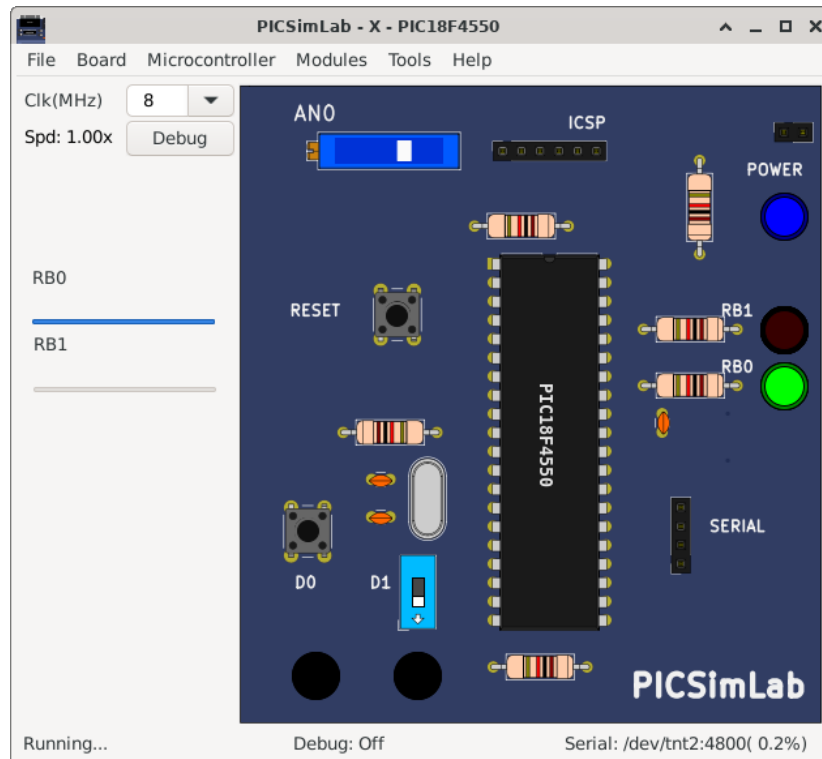
```

C.1.5 Integration with PICsimLab

After include the four files created for new board, the PICSimLab can be recompiled, as described in [Install from source](#).

C.1.6 Final Result

The PICSimLab board created for this tutorial are shown in the figure below.



The sample program below can be used to test new board, this code is write for XC8 compiler:

```

1  #include <xc.h>;
2
3  #include "config_4550.h"
4  #include "adc.h"
5  #include "serial.h"
6  #include "itoa.h"
7
8  void main()
9  {
10     unsigned int  val;
11     char buffer[10];
12
13     ADCON1=0x02;
14     TRISA=0xFF;
15     TRISB=0xFC;
16     TRISC=0xBF;

```

```
17  TRISD=0xFF;
18  TRISE=0x0F;
19
20  adc_init();
21  serial_init();
22
23
24  while(1)
25  {
26      val=adc_amostra(0);
27
28      if(PORTBbits.RD1)
29      {
30          if(val > 340)
31              PORTBbits.RB0=1;
32          else
33              PORTBbits.RB0=0;
34
35          if(val > 680)
36              PORTBbits.RB1=1;
37          else
38              PORTBbits.RB1=0;
39      }
40      else
41      {
42          if(PORTBbits.RD0)
43          {
44              PORTBbits.RB0=1;
45              PORTBbits.RB1=0;
46          }
47          else
48          {
49              PORTBbits.RB0=0;
50              PORTBbits.RB1=1;
51          }
52      }
53  }
54
55  serial_tx_str(itoa(val,buffer));
56  serial_tx_str("\r\n");
57  }
58
59 }
```